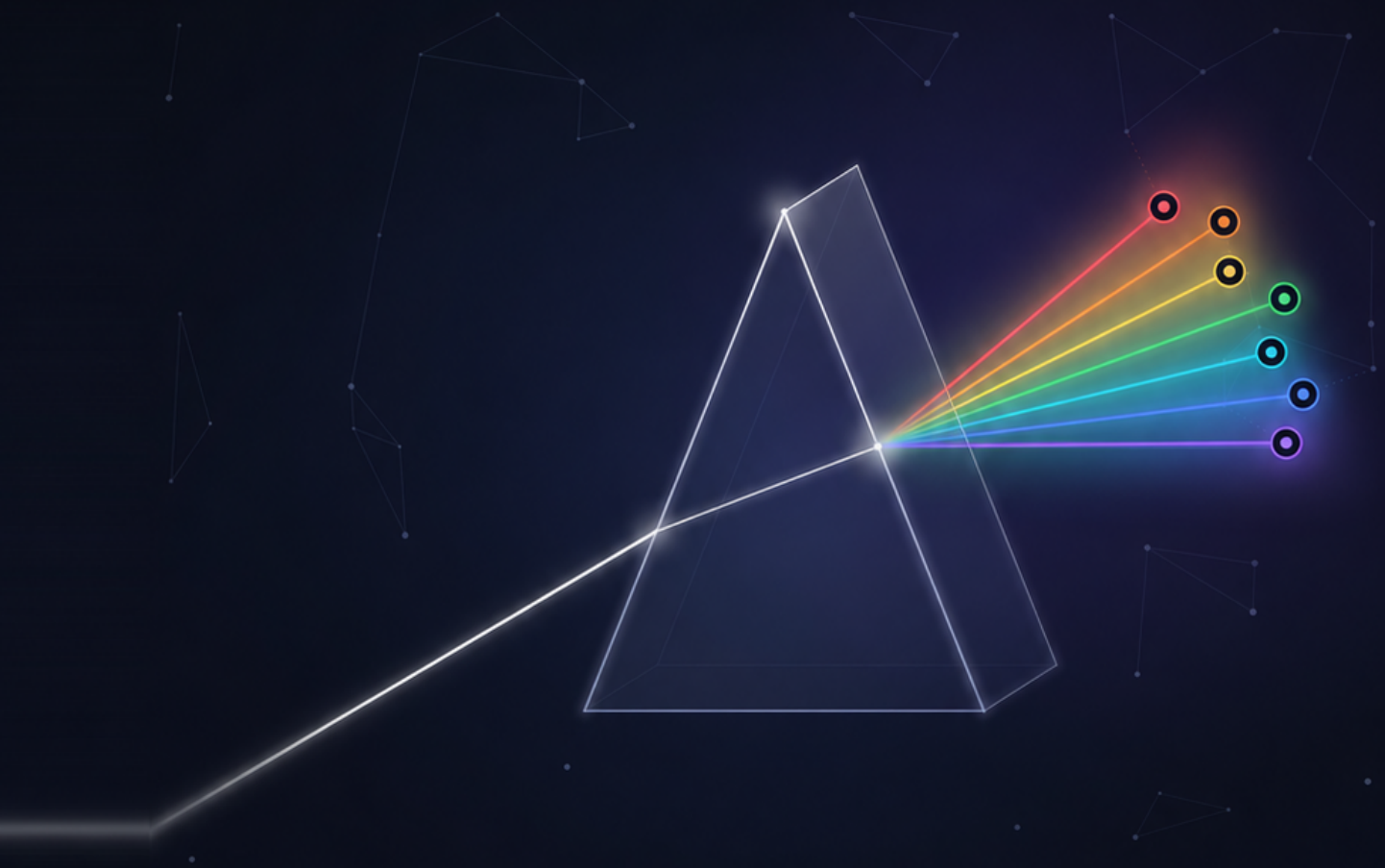


DOMINA IDENTUS

did:prism

Manual para desarrolladores de identidad descentralizada

JON BAUER & ROBERTO CARVAJAL

Domina Identus: manual para desarrolladores

Jon Bauer, Roberto Carvajal

30 de sept. de 2026

Tabla de contenidos

Bienvenida 1

Derechos de autor 2

Licencia 3
- Contenido del libro: . . . 3
- Aplicaciones: . . . 3

Dedicatoria 4

Prefacio 5

Sección I 6

Introducción 7

Fundamentos de SSI 8
- Qué significa SSI en este libro . . . 8
- La interacción SSI . . . 8
- DIDs y documentos DID . . . 10
- Credenciales verificables . . . 10
- Presentaciones verificables . . . 11
- Los roles del triángulo de confianza . . . 11
- Registros de confianza y gobernanza . . . 12
- Protocolos de intercambio . . . 12
- Cómo se aplica esto a Identus . . . 13
- Ideas clave para desarrolladores . . . 13

DIDs y documentos DID 15
- Descripción general . . . 15
- Sintaxis de los DIDs y métodos DID . . . 15
- Sujetos y controladores de DIDs . . . 16
- Estructura del documento DID . . . 17
- Ejemplo de documento DID PRISM . . . 18
- Ciclo de vida del DID . . . 19
- Resolvedores . . . 20
- Controladores . . . 21
- Flujo de verificación . . . 21
- Comprobaciones de privacidad . . . 22

Conceptos de Identus **24**
PRISM Node . . . 25
Cloud Agent . . . 25
Bloques de construcción . . . 26
Apollo: criptografía . . . 26
Castor: DID . . . 27
Pollux: credenciales verificables . . . 27
Mercury: DIDComm . . . 27
Edge Agent . . . 28
Mediador . . . 28
Registro de datos verificables (VDR) . . . 29

Sección II: primeros pasos **31**

Instalación: entorno local **32**
Descripción general . . . 32
Selección de versiones . . . 32
Requisitos previos . . . 33
Clonar el repositorio de Cloud Agent . . . 34
Configurar las instancias locales del agente . . . 34
Iniciar el Cloud Agent del emisor . . . 36
Iniciar el Cloud Agent del verificador . . . 36
Detener las instancias locales . . . 37
Modelo SSI local . . . 38

API REST del agente **39**
La API de Cloud Agent . . . 39
Especificación OpenAPI . . . 41
Pasarela APISIX . . . 42
Swagger UI . . . 42
Postman . . . 44
Tutoriales . . . 44

Mediador **46**
Descripción general . . . 46
Protocolos disponibles . . . 46
Selección de versión . . . 49
Requisitos previos . . . 49
Clonar el repositorio del mediador . . . 49
Archivos de configuración local . . . 49
Iniciar el mediador . . . 52
Comprobar el mediador en ejecución . . . 53
Notas de configuración . . . 54
Detener el mediador . . . 55

Sección III: construcción **56**

Proyecto de ejemplo **57**
Descripción general: billetera de billetes de avión . . . 57

Billeteras **58**
Descripción general . . . 58
Responsabilidades de la billetera . . . 58
Billeteras de Edge Agent . . . 59
Almacenamiento Pluto . . . 60
Billeteras de Cloud Agent . . . 60
Billeteras de varios inquilinos . . . 61
Material de claves de Cloud Agent . . . 61
Elegir la custodia Edge o Cloud . . . 62
Flujo del billete de avión . . . 63
Notas de implementación . . . 64

DIDComm **66**
Descripción general . . . 66
Características principales . . . 66
Estructura del mensaje . . . 67
Protocolos . . . 68
Interacción de ejemplo . . . 69
Ventajas . . . 69
DIDComm en Identus . . . 70

Conexiones **71**
Descripción general . . . 71
Conexiones en la identidad autosoberana . . . 71
DIDs de pares . . . 72
Invitaciones fuera de banda . . . 74
Conectar dos pares . . . 76
Presentación y emisión de credenciales sin conexión previa . . . 83

Credenciales verificables **85**
Descripción general . . . 85
Formatos . . . 85
Credenciales verificables JWT . . . 86
Credenciales verificables SD-JWT . . . 86
AnonCreds . . . 86
OID4VCI . . . 87
Contexto de W3C VC 2.0 . . . 87
Esquemas . . . 87
Especificaciones de esquemas . . . 87
Esquema de ejemplo . . . 88
Crear esquemas en Identus . . . 88
Versiones . . . 89

Emisión . . . 89
Requisitos previos . . . 90
Estados de la credencial . . . 90
Flujo del emisor . . . 92
Flujo del titular . . . 93
Emisión sin conexión previa . . . 94
Emisión OpenID4VCI . . . 95
Presentación y verificación . . . 95
Revocación . . . 96
Mecanismo de revocación . . . 96
Revocar una credencial . . . 97
Comprobar el estado de revocación . . . 97

Verificación **99**
Presentar una prueba . . . 99
Verificación y aceptación . . . 101
Verificación directa de credenciales . . . 101
Políticas de verificación . . . 102
Divulgación selectiva . . . 102

Sección IV: despliegue **105**

Instalación: entorno de producción **106**
Alcance . . . 106
Selección de versiones . . . 106
Modelo de despliegue . . . 107
Requisitos de hardware y datos . . . 110
Preparar la configuración básica de Identus . . . 110
Crear el archivo de entorno de preproducción . . . 112
Ejemplo Compose con características de producción . . . 114
Configuración y seguridad de producción . . . 120
Refuerzo de Docker y PostgreSQL . . . 121
SSL y proxy inverso . . . 122
Administración de claves con HashiCorp Vault . . . 122
Operación con varios inquilinos . . . 123
IAM externo con Keycloak . . . 125
Operaciones de Cardano Wallet . . . 128
Operaciones de Cardano DB Sync . . . 131
Selección de red preprod y mainnet . . . 133
Copiar la configuración de red Cardano . . . 133
Iniciar preprod . . . 134
Comprobaciones de producción . . . 135

Mantenimiento **137**
Reinicio y limpieza . . . 137

Observabilidad . . . 138
Comprobaciones de estado y servicios . . . 138
Administración de nodos y memoria . . . 138
Pruebas de rendimiento . . . 139
Análisis con BlockTrust Analytics . . . 139
Actualización de agentes . . . 139
Selección de versiones y compatibilidad . . . 139
Procedimiento de actualización y notas de compatibilidad . . . 139
Reducir el tiempo de inactividad . . . 140
HashiCorp Vault y administración de claves . . . 140
Administración de claves . . . 140
Copias de seguridad y restauración . . . 140
Rotación de claves y credenciales . . . 141

Sección V: adenda **145**

Registros de confianza **146**
Qué es un registro de confianza . . . 146
Posición del registro de confianza en la arquitectura . . . 147
Trust Registry Query Protocol (TRQP) 2.0 de ToIP . . . 149
Registros de confianza e Identus hoy . . . 150

Continúa tu recorrido **151**

Apéndices **152**

Erratas **153**

Glosario **154**

Bienvenida

¡Te damos la bienvenida al libro!

Derechos de autor

IdentusBook.com

Copyright © 2024-2026 Jon Bauer y Roberto Carvajal

Todos los derechos reservados

Consulta nuestra Licencia para obtener más información sobre los usos permitidos.

Licencia

Contenido del libro:

Publicamos el contenido del libro (texto, código impreso e imágenes) bajo la licencia Creative Commons Attribution-NonCommercial-ShareAlike 4.0.

Queremos que cualquier persona pueda usar el libro o extraer fragmentos, siempre que cite la fuente. En general, aceptamos que otras personas compartan nuestro trabajo con fines no comerciales. Protegemos los derechos de autor de este libro para poder publicar versiones y revisiones oficiales, y para mantener la posibilidad de una edición impresa si surge la oportunidad o si tiene sentido.

Si quieres reproducir contenido de este libro con fines comerciales, ponte en contacto con nosotros.

Aplicaciones:

Publicamos el código de ejemplo que acompaña a este libro bajo una licencia MIT.

Esta licencia permite que cualquier persona use, modifique o redistribuya el código, siempre que también le asigne la licencia MIT.

Dedicatoria

Dedicamos este libro a las numerosas personas de Input/Output, Hyperledger, LF Decentralized Trust y de toda la comunidad que han contribuido a que este software sea lo que es hoy.

Valoramos su trabajo. ¡Gracias!

Prefacio

La identidad sustenta innumerables sectores y productos, y la privacidad debe ser un derecho humano. Proteger la privacidad de quienes usan productos basados en la identidad exige mucho trabajo, pero merece el esfuerzo.

Nos acercamos a la identidad digital por una posibilidad: la privacidad real en las transacciones entre personas. En *Code: And Other Laws of Cyberspace* (1999), Lawrence Lessig advirtió que los gobiernos ejercen una presión constante para controlar y regular las tecnologías que nacen en libertad, como Internet. Sin límites a esa presión, el control sustituye a la libertad y la vigilancia sustituye a la privacidad.

Los sistemas centralizados permiten ejercer control, y el poder absoluto corrompe de forma absoluta. Las tecnologías descentralizadas, como las cadenas de bloques y la identidad autosoberana, rechazan la idea de que una sola entidad deba observar toda la información. El cifrado, la criptografía y la arquitectura descentralizada demuestran que podemos contar con identidad verificable, comercio y confianza sin renunciar a la privacidad.

Identus expresa esos principios. Con Identus puedes crear aplicaciones útiles sin alimentar la vigilancia. Quizá quieras mantener conversaciones, acuerdos y compras en privado. Quizá solo quieras saber que las personas con las que tratas en Internet son quienes dicen ser. La identidad autosoberana nos devuelve la libertad y la privacidad que merecemos para crear y conectar. Es lo correcto.

Escribimos este libro para mostrar a otros desarrolladores cómo usar este potente conjunto de herramientas. Esperamos que te impulse a crear algo valioso para todos. Tenemos muchas ganas de ver lo que construyes.

Jon Bauer y Roberto Carvajal

Sección I

Introducción

Este libro se dirige a desarrolladores que quieren crear y lanzar aplicaciones de identidad autosoberana con Identus. Queremos mostrarte cómo configurar, construir y desplegar una idea compleja desde cero.

Para aprender sobre la identidad autosoberana, ya cuentas con excelentes recursos. Si todavía no conoces SSI o Identus, te recomendamos consultar los siguientes recursos antes de leer este libro.

- Self-Sovereign Identity, de Alex Preukschat, Drummond Reed y otros autores. Este libro ofrece una referencia fundamental sobre SSI y los temas de su ecosistema.
- Documentación de Identus.

Identus todavía es una tecnología nueva. Cuando escribimos este libro, había pocas prácticas recomendadas y, de hecho, pocas experiencias de uso. Un pequeño grupo de desarrolladores se ha aventurado a crear aplicaciones en la plataforma y a compartir sus experiencias. Queremos compartir lo que hemos aprendido para ampliar ese conocimiento, ayudarte a evitar los errores comunes y llevar tus ideas al mercado.

Esperamos que cualquier persona que sienta curiosidad por crear aplicaciones de identidad digital descentralizada pueda leer este libro.

Esperamos que quienes empiezan puedan evitar los errores y malentendidos comunes, y llevar sus ideas al mercado más rápido.

Nos alegra que nos acompañes :)

Fundamentos de SSI

Qué significa SSI en este libro

La identidad autosoberana (Self-Sovereign Identity, SSI) describe sistemas de identidad que usan credenciales en manos de sus titulares y pruebas verificables, en lugar de un único proveedor de identidad que guarda el perfil de la cuenta del usuario. Una persona, organización, dispositivo o agente de software puede recibir una credencial de un emisor, guardarla en una billetera o agente y presentar una prueba a un verificador. El verificador comprueba la prueba y aplica su propia política sin crear una integración de cuentas específica con cada emisor.

SSI no hace que las afirmaciones del titular sean confiables por sí solas. Un verificador comprueba quién emitió la credencial, qué afirma, si la prueba es válida, si la credencial sigue vigente y si acepta al emisor para esa transacción. SSI cambia la custodia y el intercambio de datos de identidad; la gobernanza, la ley, los contratos y la reputación siguen influyendo en las decisiones de confianza.

Este capítulo usa los términos de la Recomendación del W3C sobre identificadores descentralizados (DIDs) v1.0 y la Recomendación del W3C sobre el modelo de datos de credenciales verificables v2.0. DID Core define los DIDs como identificadores de identidad digital verificable y descentralizada que pueden funcionar sin depender de registros centralizados, proveedores de identidad ni autoridades de certificación. El modelo de datos de VC define credenciales, presentaciones, emisores, titulares, sujetos y verificadores. La guía de inicio rápido de Identus describe Identus como bibliotecas centrales para las interacciones SSI entre emisores, titulares y verificadores.

La interacción SSI

Un intercambio típico tiene cuatro pasos.

1. Un emisor hace afirmaciones sobre un sujeto y las reúne en una credencial.
2. Un titular recibe la credencial y la guarda en una billetera o agente.
3. Un verificador pide al titular una prueba que responda a una solicitud específica.
4. El titular devuelve la presentación y el verificador comprueba la criptografía, el estado y la política de negocio.

Ejemplo: una universidad emite una credencial de título académico a Alice. Alice recibe la credencial en su billetera y actúa como titular. El título describe a Alice, por lo que ella también es el sujeto. Un empleador actúa como verificador y pide una prueba de que

Alice tiene un título en informática de una universidad acreditada. La billetera de Alice crea una presentación a partir de la credencial. El software del empleador comprueba que la universidad firmó la credencial, que Alice puede demostrar el control de las claves del titular que exige el formato de credencial, que el emisor no ha suspendido ni revocado la credencial y que la política de contratación del empleador acepta a esa universidad.

La evaluación de la política es validación. La verificación criptográfica puede demostrar que la clave de un emisor protegió la credencial y que nadie la modificó después de su emisión. No puede demostrar que el emisor sea aceptable para una decisión concreta. Un diploma de un proveedor de formación desconocido puede superar la verificación criptográfica y aun así incumplir la política del empleador. La especificación de VC del W3C describe la validación como la comprobación de reglas de negocio del verificador para determinar si una credencial resulta adecuada para un uso concreto.



Figura 1: El emisor entrega la credencial; el titular entrega la presentación. El empleador decide si la acepta después de las comprobaciones técnicas.

DIDs y documentos DID

Un identificador descentralizado (DID) es una URI. Incluye el esquema `did:`, un método DID y un identificador específico del método. En `did:prism:4a9bce8d72e4c30017c42f2b`, el método es `prism`; el último segmento contiene los datos del identificador que define ese método.

El método DID indica al software cómo crear, resolver, actualizar y desactivar esa clase de DID. DID Core no exige una tecnología de almacenamiento concreta. Un método DID puede usar un libro mayor distribuido, una base de datos, una red entre pares u otro registro de datos verificables, siempre que el método defina las operaciones necesarias.

Al resolver un DID, el software obtiene un documento DID. Los documentos DID expresan material público de verificación, relaciones de verificación y servicios. Un documento DID puede indicar al software qué claves públicas puede usar para autenticación, afirmación, acuerdo de claves, invocación de capacidades o delegación de capacidades. Puede enumerar puntos de acceso de servicios para protocolos de comunicación como DIDComm. Debe excluir secretos, claves privadas y perfiles personales.

El sujeto del DID y su controlador pueden ser la misma entidad, pero también pueden ser entidades distintas. El sujeto es la persona, organización, dispositivo, modelo de datos u otra cosa que identifica el DID. El controlador es la entidad que puede cambiar el documento DID según el método DID. El DID de una empresa puede identificar a la empresa como sujeto, y la empresa puede delegar las operaciones de claves a un Cloud Agent bajo su control.

Los DIDs pueden generar un riesgo de correlación. DID Core advierte que los identificadores inequívocos a escala global pueden vincular actividades entre contextos. Los DIDs por pares reducen ese riesgo al dar a cada relación su propio DID seudónimo. Aplica la misma revisión de privacidad a los documentos DID: las claves reutilizadas, los puntos de acceso poco comunes o las propiedades descriptivas también pueden vincular actividades entre relaciones.

Credenciales verificables

Una credencial es un conjunto de afirmaciones de un emisor. Una credencial verificable (Verifiable Credential, VC) incluye un mecanismo de protección que permite al software detectar modificaciones y comprobar la autoría. La VC puede referirse a una persona, organización, dispositivo, cuenta, envío, artefacto de software o cualquier otro sujeto.

El emisor decide qué afirmar y firma la credencial o la protege con otro mecanismo. El titular guarda la credencial. El sujeto es la entidad que describen las afirmaciones. El titular y el sujeto suelen coincidir, pero no siempre. Una madre o un padre puede guardar una credencial sobre su hijo. Un representante de una empresa puede guardar una credencial sobre la empresa. Quien administra una flota de dispositivos puede guardar credenciales sobre esos dispositivos.

El modelo de datos y el formato de credencial son conceptos distintos. W3C VC 2.0 define el modelo. Las implementaciones pueden proteger y transportar credenciales mediante distintos

formatos y protocolos. Los capítulos posteriores sobre Identus usan JWT-VC, SD-JWT-VC y AnonCreds en los flujos de implementación, por lo que la elección depende de los formatos de credencial que admitan los agentes y las billeteras de ese flujo.

La verificación en producción incluye el estado de la credencial. W3C VC 2.0 define `credentialStatus` para descubrir información de estado, como la suspensión o la revocación. Los detalles dependen del método de estado. Un verificador que acepta una credencial sin comprobar su estado puede aceptar una credencial que el emisor ya no respalda.

Presentaciones verificables

Una presentación verificable (Verifiable Presentation, VP) contiene datos derivados de una o más credenciales que el titular comparte con un verificador concreto. Una presentación puede incluir una credencial, partes de una credencial o datos de varias credenciales, según el formato de credencial y el mecanismo de prueba.

Un titular usa presentaciones para responder a las solicitudes de los verificadores. El verificador pide los datos que necesita. La billetera muestra la solicitud al titular. El titular autoriza o rechaza la divulgación. La billetera crea entonces una presentación que se ajusta a la solicitud y al formato de credencial.

La divulgación selectiva depende del formato de credencial y del mecanismo de prueba. El modelo de datos de VC del W3C define la divulgación selectiva como la capacidad del titular para elegir con precisión qué compartir. La misma especificación recomienda minimizar los datos: los verificadores deben pedir la información mínima necesaria para una transacción. Un formato de credencial que admite divulgación selectiva puede permitir que Alice demuestre que tiene más de 21 años sin revelar su fecha de nacimiento completa. Un formato que carece de esa función puede exigir que comparta más datos.

Los roles del triángulo de confianza

Las conversaciones sobre SSI suelen usar el «triángulo de confianza» para describir al emisor, al titular y al verificador. Considera el triángulo como roles en una interacción, no como etiquetas permanentes para personas u organizaciones.

Un emisor crea una credencial y asume la responsabilidad de sus afirmaciones. Un departamento de vehículos motorizados puede emitir una credencial de licencia de conducir. Una universidad puede emitir una credencial de título académico. Un gimnasio puede emitir una credencial de membresía. El emisor obtiene su autoridad del contexto: la ley, la acreditación, un contrato, las reglas de la comunidad, la reputación o la gobernanza.

Un titular recibe credenciales y crea presentaciones a partir de ellas. El titular puede ser una persona que usa una billetera móvil, una empresa que usa un Cloud Agent o software que administra credenciales para dispositivos. El control del titular significa que este decide si presenta una credencial en una interacción concreta, dentro de los límites del diseño de la billetera, el modelo de custodia, la ley y la política.

Un verificador recibe una presentación y decide si la acepta. La verificación suele comprobar la sintaxis, las pruebas criptográficas, las claves del emisor, la vinculación con el titular, el estado de la credencial y el esquema. La validación comprueba las reglas de negocio. Para un bar, puede bastar con «mayor de 21 años y credencial emitida por un departamento estatal de vehículos motorizados reconocido». Para la credencial de un empleado de hospital, el verificador puede necesitar comprobar el rol, el centro, el estado de la licencia y los registros de confianza.

Los roles dependen de la interacción. La misma entidad puede guardar una credencial, verificar otra y emitir una tercera.

Registros de confianza y gobernanza

Un DID puede demostrar el control de claves. Una credencial puede demostrar que un emisor hizo una afirmación. El verificador todavía necesita decidir si ese emisor tiene la autoridad adecuada para su contexto. Los registros de confianza publican esa información de autorización para un ecosistema definido.

Trust over IP Foundation describe una consulta a un registro de confianza en términos sencillos: «¿Tiene la entidad X la autorización Y dentro del marco de gobernanza Z del ecosistema?». El anuncio del protocolo de consultas a registros de confianza de ToIP v2.0 presenta TRQP como un mecanismo de solo lectura para descubrir quién tiene autorización para hacer qué dentro de un ecosistema de confianza digital. LF Decentralized Trust describe un registro de confianza como un sistema que conserva información autorizada que las partes que confían en ella usan para tomar decisiones de confianza.

Un registro de confianza no crea autoridad por sí solo. La autoridad procede del marco de gobernanza que lo respalda. Para una credencial inmobiliaria, el registro puede enumerar agentes con licencia y los tipos de credencial que pueden emitir o verificar. Para un título universitario, el registro puede enumerar instituciones acreditadas. Para una credencial de cadena de suministro, puede enumerar auditores que acepta un grupo industrial concreto.

Un verificador debe comprobar por separado la verificación técnica y la política de aceptación. La verificación comprueba pruebas, claves, sintaxis y estado. La política de aceptación comprueba si el emisor y el tipo de credencial cumplen las reglas de la parte que confía en ellos.

Protocolos de intercambio

Los DIDs y las VC definen modelos de datos y material de verificación. Las aplicaciones también necesitan protocolos para intercambiar mensajes, emitir credenciales y solicitar presentaciones.

DIDComm Messaging v2.1 define formatos de mensajes cifrados que usan muchos sistemas SSI para la comunicación entre agentes. Los mensajes cifrados de DIDComm ocultan el contenido a quienes carecen de autorización, revelan y demuestran la identidad del remitente

a los destinatarios autorizados en el modo autenticado, y garantizan la integridad. Identus usa DIDComm V2 para la comunicación entre Cloud Agents y para muchos flujos de agentes.

OpenID Foundation define protocolos basados en OAuth para el intercambio de credenciales. OpenID4VCI 1.0 define una API para emitir credenciales verificables. OpenID4VP 1.0 define un protocolo para solicitar y presentar credenciales. Los ecosistemas de producción pueden admitir DIDComm, OpenID4VCI, OpenID4VP, transferencia mediante códigos QR, presentaciones sin conexión o una combinación de estas vías.

Cómo se aplica esto a Identus

Identus ofrece componentes para estos roles SSI. El código de las aplicaciones puede usar agentes, SDK y API en lugar de volver a implementar el conjunto de estándares.

El Identus Cloud Agent puede emitir, guardar y verificar VC; administrar DIDs y conexiones basadas en DIDs; exponer una API REST; y usar DIDComm V2 para la comunicación entre agentes. Sus usos habituales incluyen billeteras con custodia, emisores empresariales, servicios de verificación y flujos de backend.

La documentación de administración de DIDs de Identus explica que Cloud Agent administra DIDs PRISM y DIDs de pares (Peer DIDs). Los DIDs PRISM sirven para la identidad pública del emisor y la resolución respaldada por un libro mayor. Los DIDs de pares sirven para las actividades DIDComm en las que los identificadores específicos de cada relación reducen la correlación.

La guía de inicio rápido de Identus describe PRISM Node como el componente de registro de datos verificables que ancla información de claves para la emisión y la verificación. También describe a los mediadores como servicios que guardan y retransmiten mensajes entre Cloud Agents y SDK de billeteras. PRISM Node admite la resolución pública de DIDs. Los mediadores retransmiten mensajes para agentes que carecen de conexión permanente, sin leer el contenido cifrado de DIDComm.

Ideas clave para desarrolladores

SSI permite que emisores, titulares y verificadores intercambien afirmaciones con pruebas criptográficas y participación del titular. Los DIDs identifican y resuelven material de verificación. Los documentos DID publican claves públicas y servicios, y excluyen perfiles de identidad privados. Las credenciales verificables contienen afirmaciones del emisor. Las presentaciones verificables permiten que los titulares respondan a las solicitudes de los verificadores. La verificación comprueba pruebas y estado. La validación aplica políticas. Los registros de confianza y la gobernanza ayudan a los verificadores a decidir qué emisores y tipos de credencial aceptan en un ecosistema concreto.

En Identus, estos conceptos corresponden a componentes: PRISM Node para la resolución pública de DIDs, Cloud Agent para el trabajo de emisores, titulares y verificadores en

servidores, SDK de Edge Agent para billeteras, DIDComm para mensajes seguros entre agentes y mediadores para el enrutamiento cuando los agentes de borde están sin conexión.

DIDs y documentos DID

Descripción general

Un **identificador descentralizado (DID)** es una URI para un sujeto. El sujeto puede ser una persona, organización, dispositivo, agente de software, modelo de datos u otra entidad que elija el controlador del DID. La especificación DID Core del W3C define un DID como una URI con tres partes: el esquema `did:`, un nombre de método y un identificador específico del método.

El DID en sí es solo el identificador. Un resolvedor resuelve el DID y devuelve un resultado de resolución. Si la resolución tiene éxito, el resultado incluye un **documento DID**, metadatos de resolución y metadatos del documento DID. La especificación DID Resolution del W3C define esta interfaz del resolvedor y deja los pasos de obtención específicos al método DID.

Un **documento DID** publica la información pública que el software necesita para interactuar con el sujeto del DID: métodos de verificación, relaciones de verificación y servicios. Un documento DID debe excluir claves privadas, secretos y datos de perfiles personales. DID Core advierte que los documentos DID públicos pueden generar riesgos de privacidad y correlación. Por eso, los datos personales deben mantenerse en credenciales, intercambios entre pares o puntos de acceso de servicios controlados, fuera del documento público.

Los desarrolladores de Identus usan `did:prism` y `did:peer` para tareas distintas. `did:prism` sirve para las identidades públicas de emisores y verificadores que necesitan resolución respaldada por un libro mayor. `did:peer` sirve para las actividades DIDComm específicas de cada relación. Una billetera, un Cloud Agent o un mediador puede usar DIDs de pares distintos para reducir la correlación entre relaciones. La documentación de administración de DIDs de Identus describe cómo Cloud Agent admite DIDs PRISM y DIDs de pares administrados.

Sintaxis de los DIDs y métodos DID

La sintaxis genérica es:

```
did:<method>:<method-specific-id>
```

Para `did:prism`, la especificación del método DID PRISM define esta estructura:

```
did:prism:<initial-hash>[:<encoded-state>]
```

La forma corta contiene el prefijo `did:prism:` y un hash inicial de 64 caracteres. La forma larga añade el estado inicial codificado después de otros dos puntos. PRISM usa la forma larga antes de que el controlador ancle el DID. Cuando el controlador publica la operación DID y el VDR PRISM la acepta, el software puede usar la forma corta como DID PRISM publicado.

El estado codificado de un DID PRISM en forma larga no es un documento DID JSON. Es un estado específico del método que PRISM codifica para que un resolvedor pueda construir el documento DID inicial antes de que aparezca una operación pública de creación en el VDR.

Un **método DID** define cómo el software crea, resuelve, actualiza y desactiva DIDs y documentos DID para ese método. DID Core define el modelo de datos común. La especificación del método define el registro, las operaciones del protocolo, la codificación, las reglas de validación y el ciclo de vida.

Para `did:prism`, el método PRISM define operaciones de creación, actualización y desactivación. Los nodos PRISM leen las operaciones del VDR y mantienen el estado actual necesario para construir documentos DID. La especificación del VDR PRISM describe las entradas SSI como cadenas de eventos con eventos de creación, actualización y desactivación.

Sujetos y controladores de DIDs

El DID identifica al sujeto. El método DID autoriza al controlador a cambiar el documento DID. Una misma entidad puede asumir ambos roles, aunque también pueden asumirlos entidades distintas.

Por ejemplo, una universidad puede ser el sujeto de `did:prism:...`. La universidad puede ejecutar un Identus Cloud Agent que guarde las claves del controlador y publique actualizaciones del DID mediante PRISM Node. La universidad sigue siendo el emisor en el intercambio de credenciales; el Cloud Agent constituye la infraestructura que actúa en su nombre.

La propiedad `controller` de nivel superior en un documento DID identifica uno o más DIDs de controladores. DID Core trata esta autorización por separado de `authentication`. Un verificador que comprueba una prueba de autenticación revisa la relación `authentication`. El software que procesa la autoridad para actualizar un DID sigue las reglas del controlador que define el método DID.

La propiedad `controller` dentro de un `verificationMethod` tiene otro alcance. Identifica al controlador de ese método de verificación. Puede coincidir con el `id` del documento DID o apuntar a otro DID cuando un sistema delega el control de claves.

Estructura del documento DID

Los documentos DID comparten un modelo de datos y pueden tener distintas representaciones. Los ejemplos de DID Core y PRISM usan JSON-LD con `@context`; las representaciones JSON pueden omitir el contexto JSON-LD.

Estas son las propiedades habituales de un documento DID:

Propiedad	Para qué la usa el software
`id`	El DID del sujeto que describe el documento DID.
`controller`	Uno o más DIDs con autorización para controlar el documento DID según las reglas del método DID.
`verificationMethod`	Material público de verificación, como JWK, junto con un `id`, un `type` y el controlador del método.
`authentication`	Métodos de verificación autorizados para autenticar al sujeto del DID mediante desafío y respuesta.
`assertionMethod`	Métodos de verificación autorizados para hacer afirmaciones, incluida la emisión de credenciales verificables.
`keyAgreement`	Métodos de verificación autorizados para el acuerdo de claves, incluida la preparación del cifrado DIDComm.
`capabilityInvocation`	Métodos de verificación autorizados para invocar capacidades de objetos.
`capabilityDelegation`	Métodos de verificación autorizados para delegar capacidades de objetos.
`service`	Puntos de acceso o descriptores de servicios para interactuar, como la mensajería DIDComm.
`@context`	Contexto JSON-LD que usan las representaciones JSON-LD.

Una relación de verificación puede incluir métodos de verificación completos o referirse a ellos mediante una URL DID. Los ejemplos de PRISM definen cada método una sola vez en `verificationMethod` y lo referencian desde `authentication`, `assertionMethod` o `keyAgreement`.

Ejemplo de documento DID PRISM

Este ejemplo usa valores ficticios y abreviados para que la estructura resulte fácil de leer. Estos identificadores y claves no son datos PRISM que puedas resolver.


```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/suites/jws-2020/v1",
    "https://didcomm.org/messaging/contexts/v2"
  ],
  "id": "did:prism:example-university",
  "verificationMethod": [
    {
      "id": "did:prism:example-university#authentication-key-1",
      "type": "JsonWebKey2020",
      "controller": "did:prism:example-university",
      "publicKeyJwk": {
        "kty": "OKP",
        "crv": "Ed25519",
        "x": "fake-authentication-public-key"
      }
    },
    {
      "id": "did:prism:example-university#issuing-key-1",
      "type": "JsonWebKey2020",
      "controller": "did:prism:example-university",
      "publicKeyJwk": {
        "kty": "OKP",
        "crv": "Ed25519",
        "x": "fake-issuing-public-key"
      }
    },
    {
      "id": "did:prism:example-university#key-agreement-key-1",
      "type": "JsonWebKey2020",
      "controller": "did:prism:example-university",
      "publicKeyJwk": {
        "kty": "OKP",
        "crv": "X25519",
        "x": "fake-key-agreement-public-key"
      }
    }
  ],
  "authentication": [
    "did:prism:example-university#authentication-key-1"
```

```
  ],
  "assertionMethod": [
    "did:prism:example-university#issuing-key-1"
  ],
  "keyAgreement": [
    "did:prism:example-university#key-agreement-key-1"
  ],
  "service": [
    {
      "id": "did:prism:example-university#didcomm-1",
      "type": "DIDCommMessaging",
      "serviceEndpoint": [
        {
          "uri": "https://agent.example.edu/didcomm",
          "accept": ["didcomm/v2"],
          "routingKeys": ["did:peer:example-mediator#key-1"]
        }
      ]
    }
  ]
}
```

En el ejemplo, el software del emisor puede usar `#issuing-key-1` para firmar una credencial verificable, y un verificador puede comprobar que esa clave aparece en `assertionMethod`. Una implementación DIDComm puede usar `#key-agreement-key-1` para preparar mensajes cifrados y el servicio `DIDCommMessaging` para conocer el perfil DIDComm aceptado, la URI del punto de acceso y las claves de enrutamiento.

Ciclo de vida del DID

Cada método DID define sus propias reglas de ciclo de vida. Las operaciones habituales son crear, resolver, actualizar y desactivar. Los desarrolladores de Identus acceden a esas operaciones mediante las API de Cloud Agent, las API de resolvedores, la configuración de resolvedores del SDK y el comportamiento de PRISM Node.

Para un DID PRISM administrado en Cloud Agent, el controlador crea el DID a partir de una plantilla de documento. Cloud Agent deriva el material de claves PRISM de una semilla de billetera y una ruta de derivación, guarda la ruta de derivación y puede reconstruir el material de claves a partir de la semilla durante la ejecución. La guía de inicio rápido de Identus crea un DID PRISM en forma larga, lo publica mediante Cloud Agent y luego usa la forma corta como `publishedPrismDID` para los flujos de esquemas y credenciales.

Las actualizaciones de PRISM publican operaciones del método que cambian el estado del DID, como añadir o eliminar métodos de verificación o servicios. La máquina de estados

del VDR trata la desactivación como un estado final. Después de la desactivación, los resolvedores deben dejar de tratar al DID como activo.

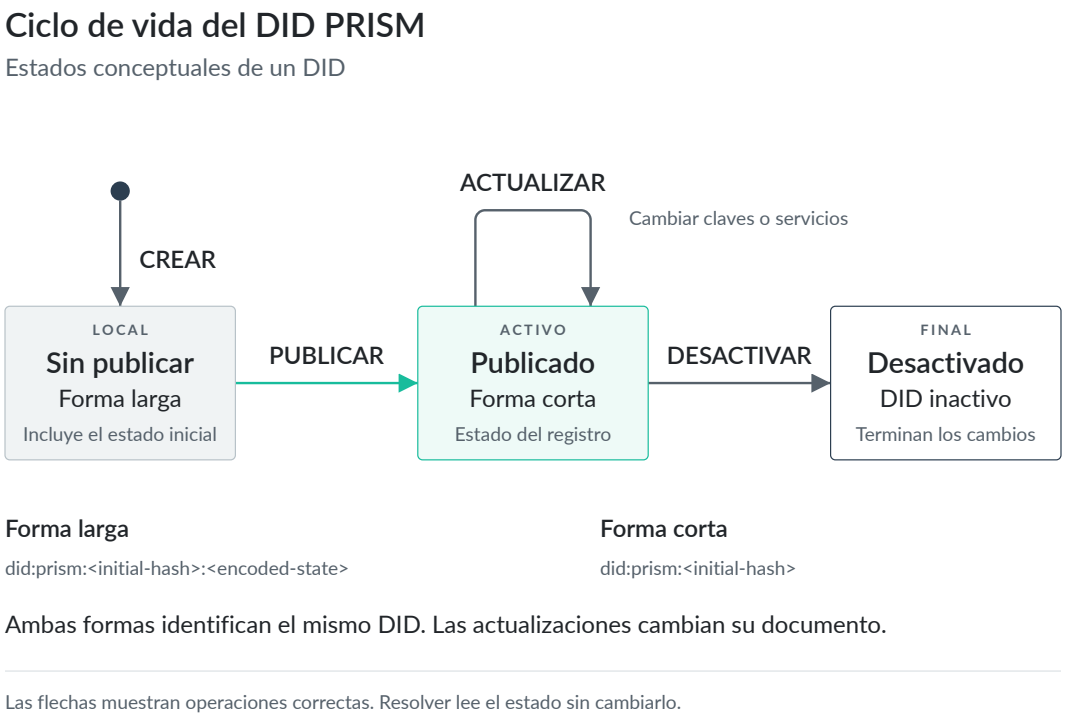


Figura 1: El controlador firma las operaciones del registro. La publicación, las actualizaciones y la desactivación surten efecto cuando el registro acepta las operaciones confirmadas. La desactivación termina el ciclo de vida del DID.

Para un DID de pares administrado, Cloud Agent genera material de claves aleatorio y lo guarda en el almacenamiento de secretos. Los DIDs de pares no sirven como identificadores públicos globales de emisores. Sirven para las relaciones DIDComm y la mediación, donde cada relación puede usar identificadores y claves distintos.

Resolvedores

Un resolvedor recibe un DID y devuelve el documento DID junto con sus metadatos. El resolvedor debe comprender el método DID. Un resolvedor genérico puede dirigir distintos métodos a drivers específicos, pero el driver de cada método sigue realizando la obtención y la validación propias de ese método.

Para `did:prism`, un resolvedor necesita el estado PRISM actual del DID. Identus documenta estas vías de resolución: Cloud Agent con PRISM Node, la compatibilidad de Universal Resolver con DIDs PRISM, la configuración de resolvedores del SDK e indexadores de la

comunidad. La documentación del resolvedor DID PRISM de Identus describe Cloud Agent y PRISM Node como una solución para crear, actualizar, desactivar y resolver DIDs PRISM.

La elección del resolvedor es una decisión de confianza. Un verificador puede usar un resolvedor alojado, ejecutar su propio resolvedor o resolver contra un estado indexado local. El software del verificador debe tratar la respuesta del resolvedor como una entrada de la verificación, no como toda la decisión de confianza. La verificación de credenciales sigue comprobando la prueba, la relación de verificación, el estado de la credencial, el esquema y la política del verificador.

Controladores

Los controladores son entidades con autorización para cambiar el documento DID según las reglas del método DID. Un controlador puede ser una persona que usa una billetera, una organización que ejecuta Cloud Agent o un servicio bajo el control operativo de una organización. El documento DID registra a los controladores como DIDs; la autoridad legal o de gobernanza que respalda a esos controladores existe fuera del documento DID.

En `did:prism`, el controlador demuestra su autoridad mediante las operaciones firmadas del método. Las reglas de validación del VDR PRISM exigen que la clave maestra del DID firme las operaciones de creación y que una clave maestra vigente firme los eventos de actualización o desactivación. Las API de DIDs PRISM administrados de Cloud Agent realizan este trabajo de protocolo para la aplicación, pero el modelo de controladores sigue determinando quién puede rotar claves, añadir servicios o desactivar el DID.

Flujo de verificación

Un verificador que comprueba una credencial emitida desde un DID PRISM usa este flujo.

1. El verificador recibe una presentación de la billetera o del Edge Agent del titular.
2. El verificador extrae el DID del emisor y el método de verificación que referencia la prueba de la credencial.
3. El resolvedor resuelve el DID del emisor y obtiene un documento DID.
4. El verificador obtiene el método de verificación referenciado y comprueba que aparece en `assertionMethod`.
5. El verificador comprueba la prueba criptográfica con la clave pública del método de verificación.
6. El verificador comprueba el estado de la credencial, el esquema y la política, incluida la confianza en el emisor para la transacción solicitada.

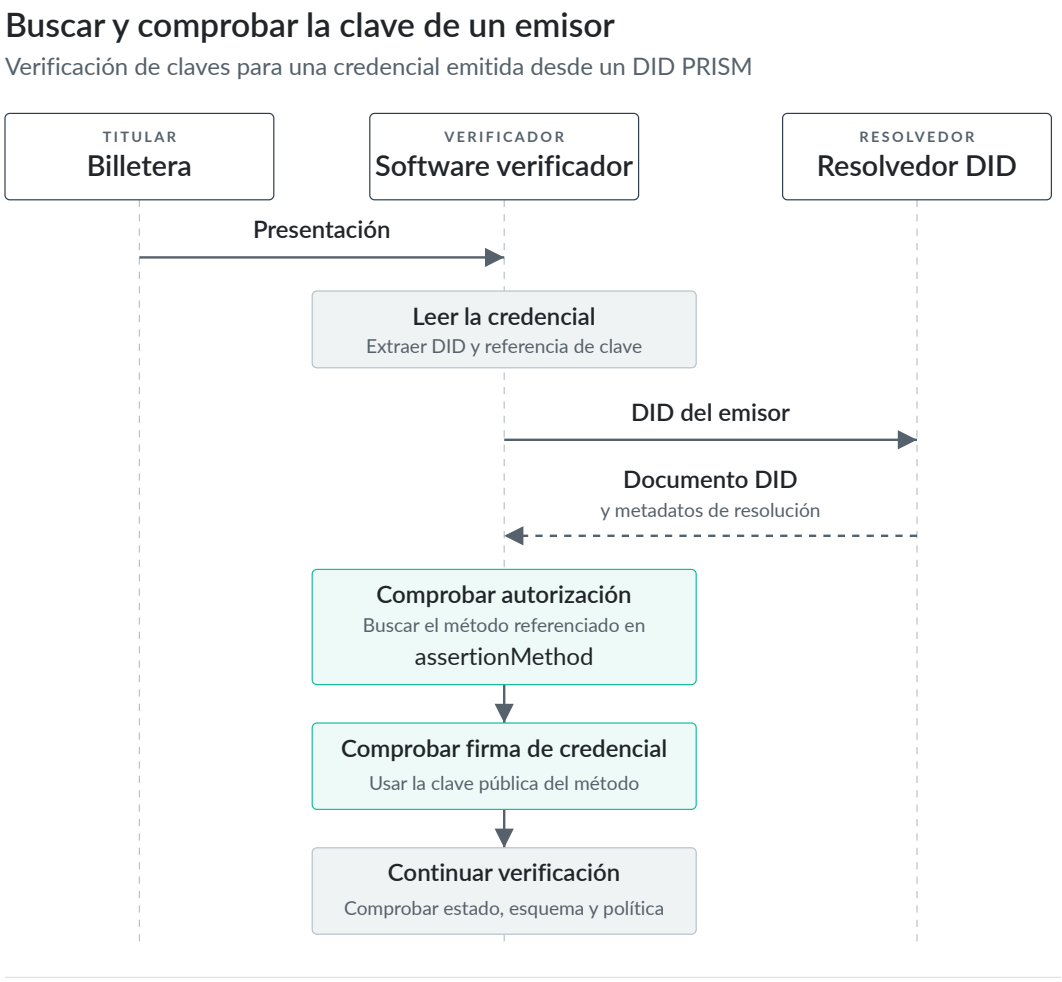


Figura 2: El resolvedor entrega el documento DID. El verificador comprueba la autorización de la clave del emisor y la firma de la credencial, y después aplica las demás comprobaciones de la credencial y la política de confianza.

Las relaciones de verificación del DID impiden reutilizar claves para distintos fines de prueba. Una clave que solo aparece en `authentication` no puede emitir una credencial. Una clave que solo aparece en `assertionMethod` no puede participar en el acuerdo de claves de DIDComm. El software del verificador debe comprobar la relación que corresponde a la operación que valida.

Comprobaciones de privacidad

Los DIDs públicos sirven para emisores, verificadores, esquemas y registros de confianza, pero generan correlación. El DID público de un emisor debe mantenerse estable para que los

verificadores puedan resolver sus claves y los sistemas de políticas puedan referirse al emisor. La billetera de un titular debe usar DIDs por pares o específicos de cada relación para las conexiones DIDComm, salvo que el titular quiera permitir la correlación pública.

Antes de publicar un documento DID, el controlador debe comprobar que solo contiene el material público necesario para el método DID y los flujos del protocolo:

1. Excluir claves privadas, semillas, tokens de portador, credenciales y atributos de perfiles personales.
2. Excluir URL de puntos de acceso que incluyan un nombre de usuario, identificador de cliente, número de cuenta u otro valor que permita correlacionar actividades.
3. Evitar que las relaciones que deben mantenerse desvinculadas reutilicen un método de verificación.
4. Excluir propiedades `type` descriptivas o propiedades personalizadas que revelen la categoría del sujeto sin que el protocolo lo necesite.

Los DIDs se centran en la verificación pública y el enrutamiento. Las credenciales y las presentaciones contienen afirmaciones. Los agentes y las billeteras gestionan la divulgación a verificadores concretos.

Conceptos de Identus

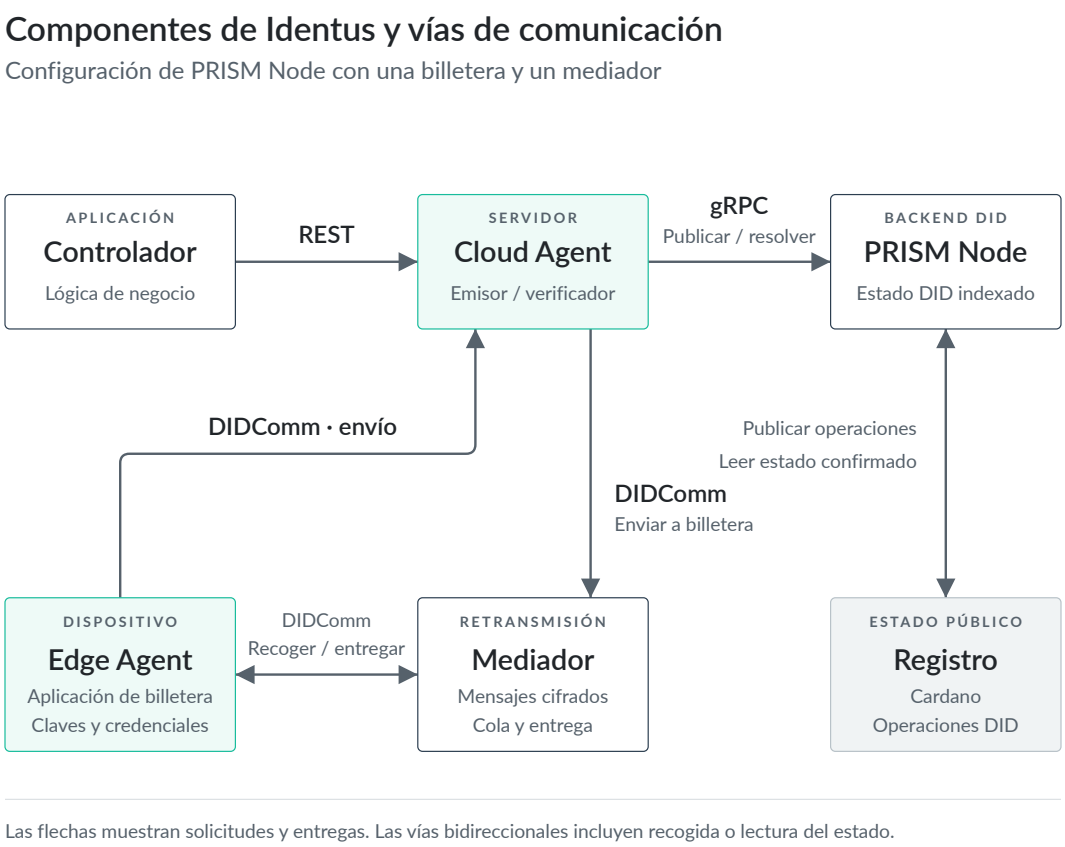


Figura 1: El controlador ejecuta la lógica de negocio mediante la API de Cloud Agent. La billetera intercambia mensajes DIDComm con Cloud Agent y obtiene los mensajes entrantes del mediador. PRISM Node publica operaciones DID y resuelve el estado de los DIDs mediante el registro.

Identus reúne varios componentes de código abierto. Puedes usar cada componente por separado o crear tu propia versión, aunque sus autores los diseñaron para que funcionen juntos.

PRISM Node

PRISM Node implementa el método `did:prism` y actúa como un nodo de segunda capa para el libro mayor distribuido. Actualmente solo admite la cadena de bloques Cardano o una base de datos local, pero en el futuro actuará como una interfaz integral para varios VDR (registros de datos verificables). El nodo puede resolver DIDs PRISM y escribir transacciones en una cadena de bloques o una base de datos. Mantiene un estado interno indexado que sincroniza con la cadena de bloques subyacente para realizar búsquedas eficientes.

PRISM Node constituye un componente esencial del ecosistema Identus y ofrece una plataforma segura y confiable para almacenar y administrar identificadores descentralizados. Gestiona la creación, la actualización, la resolución y la desactivación de DIDs PRISM: genera transacciones con la información de operación necesaria, verifica y valida estas operaciones, y las publica en la cadena de bloques. Cuando la cadena confirma las transacciones, el nodo actualiza su estado interno.

La arquitectura de PRISM Node permite que los usuarios:

- Creen DIDs sin publicar mediante el bloque de construcción Apollo, con la opción de anunciarlos públicamente más adelante. No todos los DIDs necesitan publicación en un VDR. Por ejemplo, un titular puede mantener su DID fuera del ámbito público, pero Cloud Agent exige que el emisor publique y ancle su DID de emisión en un VDR. PRISM Node realiza esa operación.
- Actualicen documentos DID mediante operaciones de actualización en la cadena.
- Desactiven DIDs mediante operaciones de desactivación en la cadena.
- Resuelvan DIDs mediante consultas a los cambios históricos en la cadena.
- Consulten el estado de las operaciones enviadas al nodo.

Este enfoque de segunda capa permite que los DIDs sean escalables y eficientes. Ofrece procesamiento y almacenamiento fuera de la cadena y usa la seguridad y la inmutabilidad de la cadena de bloques subyacente. PRISM Node debe permanecer en línea para prestar un servicio confiable.

Cloud Agent

Los autores escribieron Cloud Agent en Scala. Se ejecuta en un servidor y se comunica con clientes y pares mediante una API REST. Constituye un componente esencial de una aplicación Identus: puede administrar billeteras de identidad y sus operaciones, además de emitir credenciales verificables. Cloud Agent debe permanecer en línea.

Cloud Agent ofrece servicios completos de identidad autosoberana con un diseño escalable, robusto y conforme a los estándares. Admite los estándares del W3C, DIDCommV2 y los protocolos de Hyperledger Aries para facilitar la interoperabilidad en el ecosistema SSI. Sus principales capacidades incluyen:

- Compatibilidad con varios roles de agente: emisor, titular y verificador.

- Administración de credenciales verificables del estándar W3C (formatos JSON y JSON-LD codificados como JWT), SD-JWT y AnonCreds.
- Implementación de DIF Presentation Exchange para solicitudes y envío de credenciales.
- Compatibilidad con los métodos DID `did:prism` y `did:peer` (versión 2).
- Implementación completa de la mensajería y los protocolos DIDCommV2.
- Compatibilidad con las RFC de Hyperledger Aries, incluidos el intercambio de DIDs, el protocolo fuera de banda, la emisión de credenciales y la presentación de pruebas.

La API REST de Cloud Agent permite que los desarrolladores creen controladores en cualquier lenguaje de programación sin conocer en profundidad los estándares SSI subyacentes. Esta arquitectura separa la lógica de negocio de la infraestructura de identidad. Así facilita el desarrollo de aplicaciones especializadas que usan las capacidades de la identidad descentralizada.

En un despliegue, Cloud Agent se comunica con PRISM Node mediante el protocolo gRPC. Lo usa como registro de datos verificables para anclar DIDs en un libro mayor distribuido y ofrecer alta seguridad y disponibilidad.

Bloques de construcción

Identus divide las operaciones SSI importantes en bibliotecas especializadas que llama «bloques de construcción». Puedes combinar y configurar estos componentes modulares para distintos casos de uso y requisitos de producto. Esta arquitectura ofrece flexibilidad y opciones de personalización para implementar soluciones de identidad descentralizada que respondan a tus necesidades.

Apollo: criptografía

Apollo reúne primitivas criptográficas que Identus usa para garantizar la integridad, la autenticidad y la confidencialidad de los datos. Ofrece la base para la comunicación segura y la protección de datos en el ecosistema Identus.

Apollo usa funciones hash criptográficas para crear huellas digitales de los datos y detectar modificaciones no autorizadas. Por ejemplo, cuando un verificador recibe una credencial, las funciones criptográficas de Apollo pueden comprobar que nadie la haya modificado desde su emisión.

Apollo también implementa firmas digitales para autenticar la identidad de remitentes y destinatarios, y usa algoritmos de cifrado para proteger los datos sensibles frente a accesos no autorizados. En un contexto sanitario, el cifrado de Apollo mantendría la confidencialidad de las credenciales de los pacientes cuando las comparten las partes autorizadas.

Castor: DID

Castor permite crear, administrar y resolver identificadores descentralizados (DIDs). Actualmente admite el método nativo `did:prism` y el método `did:peer`. El equipo debate una arquitectura más flexible que permitiría que cualquier persona escribiera complementos para admitir otros métodos DID.

Cuando un usuario crea una identidad digital en una aplicación Identus, Castor genera el DID y el material criptográfico asociado. Por ejemplo, una universidad que emite credenciales de estudiantes usaría Castor para crear y administrar DIDs de la institución y, posiblemente, de cada estudiante, con lo que establecería la base de relaciones digitales de confianza.

El componente resolvedor de Castor puede buscar un DID y obtener su documento DID asociado. Ese documento contiene las claves públicas, los mecanismos de autenticación y los puntos de acceso de servicios necesarios para interactuar de forma segura con esa identidad.

Pollux: credenciales verificables

Pollux gestiona todas las operaciones de credenciales verificables. Permite que los usuarios emitan, administren y verifiquen credenciales de forma que protejan la privacidad. Este bloque de construcción implementa la funcionalidad central del intercambio de credenciales en los sistemas de identidad autosoberana.

En un contexto laboral, una empresa podría usar Pollux para emitir credenciales de empleados que estos guardarían en sus billeteras digitales. Al solicitar un préstamo, un empleado podría revelar de forma selectiva la información laboral pertinente de esa credencial, sin revelar datos personales innecesarios. El banco podría usar las funciones de verificación de Pollux para confirmar la autenticidad y la validez de la credencial sin contactar directamente con el empleador.

Pollux también gestiona el estado de las credenciales. Permite que los emisores revoquen credenciales cuando sea necesario y que los verificadores comprueben si una credencial sigue siendo válida antes de aceptarla.

Mercury: DIDComm

Mercury ofrece una interfaz al protocolo DIDCommV2 que permite la comunicación segura y privada entre DIDs con independencia del mecanismo de transporte subyacente. Este bloque de construcción establece la base de la comunicación entre agentes en el ecosistema Identus.

Por ejemplo, cuando un ciudadano quiere compartir una credencial emitida por el gobierno con un proveedor de servicios, Mercury facilita el intercambio de mensajes cifrados y autenticados entre la billetera del ciudadano y el sistema de verificación del proveedor. Ambos se comunican entre pares y prescinden de intermediarios centralizados para ese intercambio.

El diseño de Mercury no depende del transporte. Estas comunicaciones seguras pueden usar distintos canales, incluidos HTTP, WebSockets y Bluetooth. Así permite distintos despliegues, desde aplicaciones web hasta dispositivos móviles.

La documentación de Identus ofrece más información sobre cada bloque de construcción.

Edge Agent

Los Edge Agents ofrecen capacidades de agente a las aplicaciones cliente, como sitios web, aplicaciones móviles y otro software orientado al usuario. Los Edge Agents pueden perder la conexión, por lo que envían y reciben todas las comunicaciones mediante un proxy en línea llamado mediador.

Los desarrolladores pueden integrar los SDK de Edge Agent en sus aplicaciones para acceder a un conjunto completo de funciones SSI en el cliente. Estos SDK gestionan operaciones esenciales, como:

- Crear y administrar DIDs.
- Guardar y administrar credenciales verificables.
- Comunicarse de forma segura mediante DIDComm.
- Realizar operaciones criptográficas de firma y verificación.
- Guardar datos de identidad de forma segura en el dispositivo.

Identus ofrece SDK de Edge Agent en varios lenguajes para distintas plataformas:

- **SDK de TypeScript**: para aplicaciones web.
- **SDK de Swift**: para aplicaciones iOS.
- **SDK de Kotlin Multiplatform**: para aplicaciones Android y desarrollo multiplataforma.

Cada SDK implementa los mismos bloques de construcción centrales que Cloud Agent: las interfaces de Apollo, Castor, Mercury, Pollux y Pluto. Esto ofrece funciones coherentes e interoperabilidad en todo el ecosistema Identus. Esta coherencia arquitectónica permite que los desarrolladores creen experiencias fluidas en las que las operaciones de identidad se ejecuten en el cliente o en el servidor, según el caso de uso.

Los Edge Agents suelen funcionar como billeteras digitales. Permiten que los usuarios mantengan el control de sus credenciales y sus datos de identidad en sus dispositivos personales. Este enfoque sigue los principios centrales de la identidad autosoberana: conserva el control de los datos en manos del usuario y reduce la dependencia de servicios centralizados.

Mediador

Los mediadores actúan como intermediarios entre DIDs de pares. Para que un agente envíe un mensaje a otro, debe conocer los DIDs `to` y `from` de cada mensaje. El remitente y el destinatario forman una conexión criptográfica llamada `DIDPair`. Los mediadores mantienen

colas de mensajes para cada `DIDPair`. Si un Edge Agent está sin conexión, el mediador guarda sus mensajes entrantes hasta que el agente vuelve a conectarse y puede recibirlos. Los mediadores pueden entregar mensajes cuando el agente los consulta o enviarlos mediante WebSockets. Deben permanecer en línea y ofrecer alta disponibilidad.

Identus admite de forma oficial su propia implementación de mediador y publica actualizaciones para ella. También existen implementaciones alternativas cercanas a la comunidad Identus. En teoría, cualquier mediador compatible con DIDCommV2 debería funcionar con Identus, por lo que esta lista no pretende incluir todas las opciones.

- PRISM Mediator.
- RootsID Mediator.
- Blocktrust Mediator.

RootsID y Blocktrust ofrecen instancias alojadas de sus mediadores con acceso público. Aunque resultan útiles durante el desarrollo, no recomendamos usarlas para despliegues Identus en producción: carecen de garantías de disponibilidad y no escalarán más allá de un pequeño número de usuarios simultáneos. El capítulo explica cómo ejecutar tu propio mediador.

Registro de datos verificables (VDR)

El registro de datos verificables (Verifiable Data Registry, VDR) aborda el reto de garantizar la autenticidad y la integridad de datos públicos en un entorno que genera y consume datos a una escala sin precedentes. Los sistemas tradicionales suelen depender de autoridades centralizadas. Esa dependencia introduce puntos únicos de fallo y debilita la confianza, en especial cuando varias partes independientes necesitan confiar en los mismos datos.

Los retos principales incluyen permitir la verificación descentralizada, garantizar la integridad y la autenticidad sin supervisión central, ofrecer interoperabilidad entre distintas tecnologías de almacenamiento (bases de datos, cadenas de bloques y cachés en memoria) y facilitar una colaboración escalable sin exigir confianza previa entre las partes.

El sistema VDR aborda estos problemas mediante una API unificada y un marco modular y extensible para almacenar, modificar, obtener y eliminar datos. Separa la capa de aplicación de los mecanismos de almacenamiento mediante dos componentes principales que admiten complementos:

- **Drivers**: son complementos que implementan la lógica real de almacenamiento y obtención de datos. Cada driver se adapta a un backend de almacenamiento concreto, como una base de datos, una cadena de bloques o almacenamiento en memoria. Los drivers ejecutan operaciones de creación, lectura, actualización y eliminación. También generan y verifican pruebas criptográficas (hashes para datos inmutables y firmas digitales para datos modificables) para garantizar la integridad.

- **Administradores de URL**: construyen y resuelven URL que hacen referencia a los datos almacenados. Incluyen los metadatos necesarios, como identificadores de drivers, familias de drivers y pruebas criptográficas, mediante parámetros de consulta estándar. Así, cuando recibe una URL, el sistema VDR puede seleccionar el driver correcto y verificar la integridad de los datos.

Esta abstracción permite que la capa VDR mantenga operaciones de datos coherentes y verificables con independencia de dónde o cómo guarde los datos. Emplea hashes criptográficos para datos inmutables y firmas digitales para datos modificables, incluidos en las URL o disponibles mediante el driver, para garantizar la integridad. Así, varias partes pueden verificar por sí mismas la autenticidad y la integridad sin confiar previamente unas en otras, lo que mejora la seguridad y la colaboración. El diseño del sistema VDR permite adaptarse a distintos backends de almacenamiento y mantener métodos coherentes de verificación y acceso a los datos.

Sección II: primeros pasos

Instalación: entorno local

Descripción general

Hyperledger Identus mantiene el código de sus componentes en repositorios distintos. El repositorio de Cloud Agent contiene el agente de servidor que usa este capítulo. Una aplicación controladora envía solicitudes HTTP a la API REST de Cloud Agent. Cloud Agent administra DIDs, mensajes DIDComm V2, emisión, verificación y almacenamiento de credenciales, y configuración de inquilinos. El README oficial de Cloud Agent lo describe como un agente en la nube W3C/Aries que se ejecuta en el servidor. Las aplicaciones de billetera de los titulares usan los SDK de Edge Agent.

El entorno local ejecuta Cloud Agent con una configuración de desarrollo. Docker inicia Cloud Agent junto con sus servicios de apoyo: PostgreSQL, HashiCorp Vault, APISIX y PRISM Node. La guía oficial de inicio rápido describe esta configuración como de un solo inquilino, con autenticación por clave de API desactivada y un libro mayor en memoria para almacenar DIDs publicados.

Este capítulo inicia dos instancias locales de Cloud Agent:

- `issuer` en el puerto `8000`, para crear DIDs de emisor, esquemas y ofertas de credenciales.
- `verifier` en el puerto `9000`, que usarás más adelante para los flujos de solicitud de pruebas.

Para el capítulo sobre la API REST, puedes ejecutar solo la instancia del emisor. La instancia del verificador resulta útil cuando el libro llega a los flujos de conexiones, emisión y verificación.

Selección de versiones

Hyperledger Identus no es un único programa con un único número de versión. Cada componente, como Cloud Agent, Mediator, NeoPRISM y cada SDK de Edge Agent, vive en su propio repositorio, evoluciona a su propio ritmo y publica versiones con su propia numeración. Una **versión de Identus Platform** es un conjunto con nombre de versiones concretas de los componentes que los mantenedores de Identus han probado juntas mediante pruebas de integración de extremo a extremo. El número de versión de la plataforma, como `v2.17`, no coincide con el número de versión de ningún componente.

El contenido de este libro es compatible con **Identus Platform `v2.17`**. En el momento de la publicación, esa versión incluía estas versiones de los componentes:

Tabla 1: Versiones de los componentes en Identus Platform `v2.17` en el momento de la publicación

Componente	Versión en Identus Platform `v2.17`
Cloud Agent	`2.2.0`
Mediator	`1.2.1`
NeoPRISM	`0.14.1`
SDK-TS (TypeScript)	`8.0.0`
SDK-Swift	`8.1.1`
SDK-KMP (Kotlin Multiplatform)	`5.0.0`

La versión de la plataforma no fija una versión de PRISM Node. Los ejemplos de este libro combinan Cloud Agent `2.2.0` con PRISM Node `2.6.1`.

Los componentes siguen publicando versiones entre una versión de la plataforma y la siguiente, así que el repositorio de un componente puede mostrar una versión más reciente que la de la tabla anterior. Antes de pasar a una versión más reciente, lee las notas de versión de Identus Platform y las notas de versión de ese componente. Las notas de la versión v2.17 también indican que SDK-KMP `5.0.0` no funciona y que se omitieron sus pruebas de integración, así que comprueba el estado actual del SDK de Kotlin antes de construir sobre él.

Este capítulo usa Cloud Agent `2.2.0` y PRISM Node `2.6.1`. Es posible que la guía oficial de inicio rápido todavía muestre un par de versiones anterior. Trata `AGENT_VERSION` y `PRISM_NODE_VERSION` como un par de versiones probado. Si cambias uno de los valores, consulta las notas de versión del componente y repite las comprobaciones de estado al final de este capítulo.

Requisitos previos

Instala Git, Docker y Docker Compose antes de ejecutar los servicios locales de Cloud Agent. Docker Desktop incluye la CLI de Docker y el complemento Compose en macOS y Windows. En Linux, puedes usar Docker Engine con el complemento Compose.

```
git --version
docker --version
docker compose version
```

Cada comando debe mostrar una versión. Si alguno falla, instala o repara esa herramienta antes de iniciar Cloud Agent.

En Windows, usa WSL 2 y ejecuta los comandos de Linux dentro de la distribución WSL. La guía de Microsoft sobre WSL explica cómo instalar una distribución, enumerar las distribuciones instaladas y comprobar si una distribución usa WSL 1 o WSL 2.

Clonar el repositorio de Cloud Agent

Clona el repositorio actual de Cloud Agent y conserva el nombre de directorio local que usa la guía oficial de inicio rápido:

```
git clone https://github.com/hyperledger-identus/cloud-agent
 ↪  identus-cloud-agent
cd identus-cloud-agent
```

Los materiales antiguos de Identus pueden usar `https://github.com/hyperledger/identus-cloud-agent`. Identus Platform `v2.15` documenta el traslado del repositorio a `hyperledger-identus/cloud-agent`.

Configurar las instancias locales del agente

El directorio `infrastructure/local` contiene los scripts y los archivos de entorno para las ejecuciones locales. El README local indica que los scripts descargan imágenes remotas, `.env` controla las versiones de las imágenes, `run.sh` inicia instancias con nombre y `stop.sh` las detiene.

Crea la configuración del emisor:

```
cat > ./infrastructure/local/.env-issuer <<'EOF'
API_KEY_ENABLED=false
AGENT_VERSION=2.2.0
PRISM_NODE_VERSION=2.6.1
PORT=8000
NETWORK=identus
VAULT_DEV_ROOT_TOKEN_ID=root
PG_PORT=5432
EOF
```

Crea la configuración del verificador:

```
cat > ./infrastructure/local/.env-verifier <<'EOF'
API_KEY_ENABLED=false
AGENT_VERSION=2.2.0
PRISM_NODE_VERSION=2.6.1
PORT=9000
NETWORK=identus
VAULT_DEV_ROOT_TOKEN_ID=root
PG_PORT=5433
EOF
```

> **Advertencia**
>
> `API_KEY_ENABLED=false` desactiva la autenticación por clave de API. Usa esta configuración solo para desarrollo local.

Los valores de `PORT` exponen las dos API REST en puertos distintos del equipo anfitrión. Los valores de `PG_PORT` impiden que los dos contenedores PostgreSQL usen el mismo puerto del anfitrión. Conservamos `NETWORK=identus` de la configuración oficial de inicio rápido; el archivo Compose local actual crea la red de ejecución a partir del nombre del proyecto Compose.

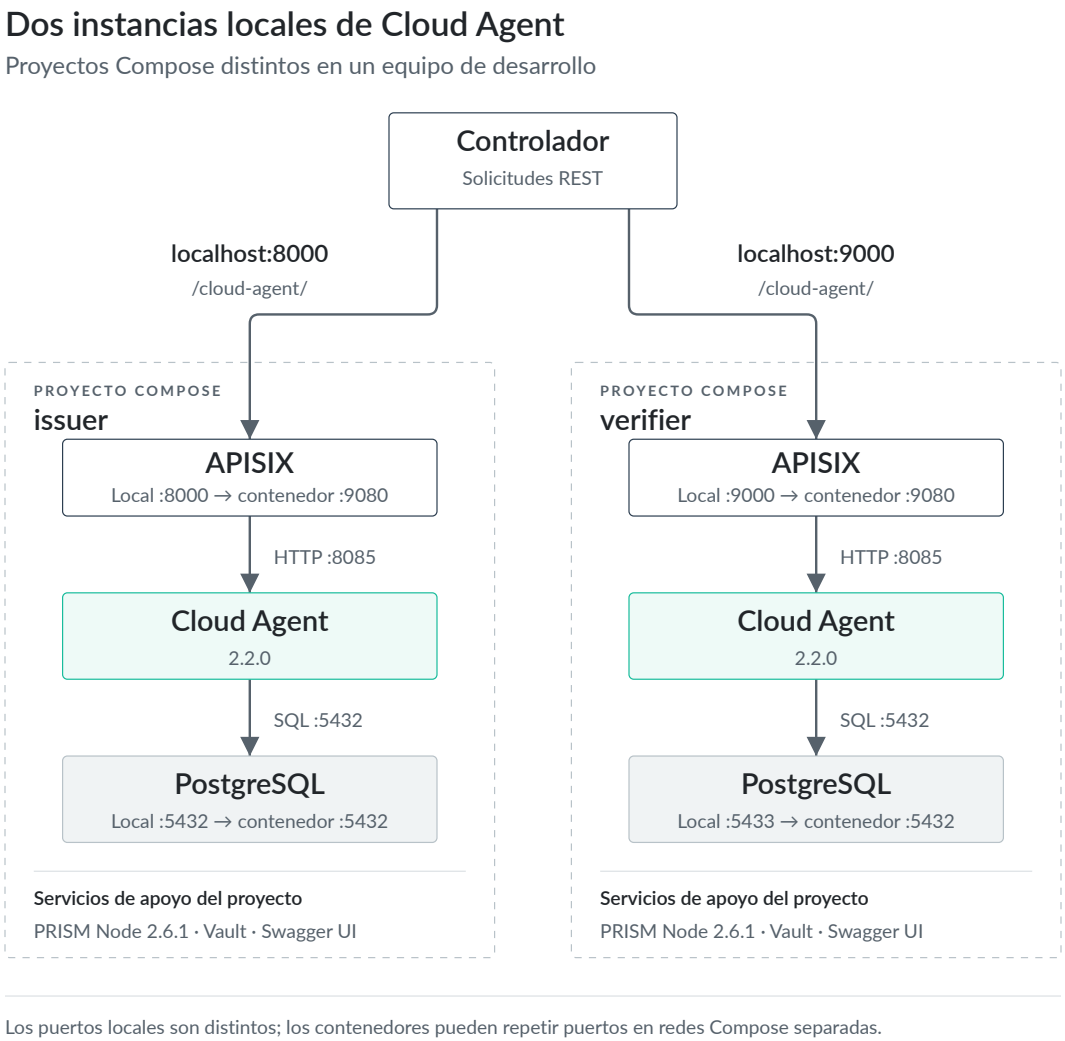


Figura 1: El controlador selecciona una instancia mediante su puerto de API en el anfitrión. Cada proyecto Compose inicia su propio agente, base de datos y servicios de apoyo. PostgreSQL expone su puerto del anfitrión en la interfaz de bucle local; el agente usa el puerto interno de la base de datos.

Las versiones actuales de Hyperledger Identus publican la imagen de Cloud Agent en Docker Hub como `docker.io/hyperledgeridentus/identus-cloud-agent`. El README local antiguo menciona `GITHUB_TOKEN` y `ghcr.io`; esa nota precede al cambio de registro Docker que documenta Identus Platform `v2.15`.

Iniciar el Cloud Agent del emisor

Ejecuta el emisor desde la raíz del repositorio. Usa el comando de tu sistema operativo.

macOS:

```
./infrastructure/local/run.sh -n issuer -b -w -e
↪  ./infrastructure/local/.env-issuer -p 8000 -d "$(ipconfig getifaddr
↪  $(route get default | grep interface | awk '{print $2}'))"
```

Linux:

```
./infrastructure/local/run.sh -n issuer -b -w -e
↪  ./infrastructure/local/.env-issuer -p 8000 -d "$(ip addr show $(ip
↪  route show default | awk '/default/ {print $5}') | grep 'inet ' | awk
↪  '{print $2}' | cut -d/ -f1)"
```

La primera ejecución descarga imágenes de contenedores y crea volúmenes Docker. Espera a que Docker indique que los contenedores funcionan correctamente. Después consulta el punto de acceso de estado del emisor:

```
curl http://localhost:8000/cloud-agent/_system/health
```

La respuesta debe incluir la versión de Cloud Agent que configuraste:

```
{"version":"2.2.0"}
```

Cloud Agent sirve la API REST del emisor en `http://localhost:8000/cloud-agent/`. Puedes acceder a Swagger UI en `http://localhost:8000/apidocs/` y al documento OpenAPI en `http://localhost:8000/docs/cloud-agent/api/docs.yaml`.

Iniciar el Cloud Agent del verificador

Inicia el verificador cuando necesites un segundo actor Cloud Agent para los flujos de solicitud de pruebas y verificación.

macOS:

```
./infrastructure/local/run.sh -n verifier -b -w -e
 ↪  ./infrastructure/local/.env-verifier -p 9000 -d "$(ipconfig getifaddr
 ↪  $(route get default | grep interface | awk '{print $2}'))"
```

Linux:

```
./infrastructure/local/run.sh -n verifier -b -w -e
 ↪  ./infrastructure/local/.env-verifier -p 9000 -d "$(ip addr show $(ip
 ↪  route show default | awk '/default/ {print $5}') | grep 'inet ' | awk
 ↪  '{print $2}' | cut -d/ -f1)"
```

Consulta el punto de acceso de estado del verificador:

```
curl http://localhost:9000/cloud-agent/_system/health
```

Respuesta esperada:

```
{"version":"2.2.0"}
```

Cloud Agent sirve la API REST del verificador en `http://localhost:9000/cloud-agent/`. Puedes acceder a Swagger UI en `http://localhost:9000/apidocs/` y al documento OpenAPI en `http://localhost:9000/docs/cloud-agent/api/docs.yaml`.

Detener las instancias locales

Detén cada instancia con el mismo valor de `-n` que usaste al iniciarla:

```
./infrastructure/local/stop.sh -n issuer
./infrastructure/local/stop.sh -n verifier
```

Para eliminar los volúmenes Docker locales de una instancia, añade `-d`:

```
./infrastructure/local/stop.sh -n issuer -d
./infrastructure/local/stop.sh -n verifier -d
```

Usa `-d` cuando quieras empezar con un estado local limpio. Omítelo cuando quieras conservar las billeteras, los DIDs, los esquemas, los registros de credenciales y otros datos de desarrollo entre reinicios.

Modelo SSI local

Cada instancia de Cloud Agent actúa como un actor SSI en este entorno local. El emisor controla los DIDs de emisión y firma credenciales. El verificador crea solicitudes de prueba y verifica presentaciones. La billetera del titular entra en el flujo más adelante mediante un SDK de Edge Agent o una aplicación de billetera.

El PRISM Node local ofrece a Cloud Agent un VDR de desarrollo para las operaciones `did:prism`. El libro mayor en memoria permite un entorno rápido que puedes desechar. Los despliegues de producción usan otra configuración para la persistencia, el aislamiento de inquilinos, la autenticación de API, los secretos y el acceso a la red Cardano.

API REST del agente

La API de Cloud Agent

Cloud Agent local expone dos interfaces públicas de protocolo mediante APISIX. Tu aplicación controladora usa la API REST para crear DIDs, registrar esquemas de credenciales, emitir credenciales verificables, administrar registros de conexión, solicitar presentaciones y consultar el estado del protocolo. Otros agentes y billeteras usan el punto de acceso DIDComm para entregar mensajes cifrados entre agentes.

Ambas interfaces comparten el mismo estado de billetera. Una solicitud REST modifica o lee registros de la billetera de Cloud Agent. Un mensaje DIDComm puede modificar esos mismos registros cuando Cloud Agent procesa un mensaje entrante de conexión, emisión o presentación. El README de Cloud Agent describe este patrón como un controlador que envía solicitudes HTTP al agente y procesa las notificaciones webhook que este le envía.

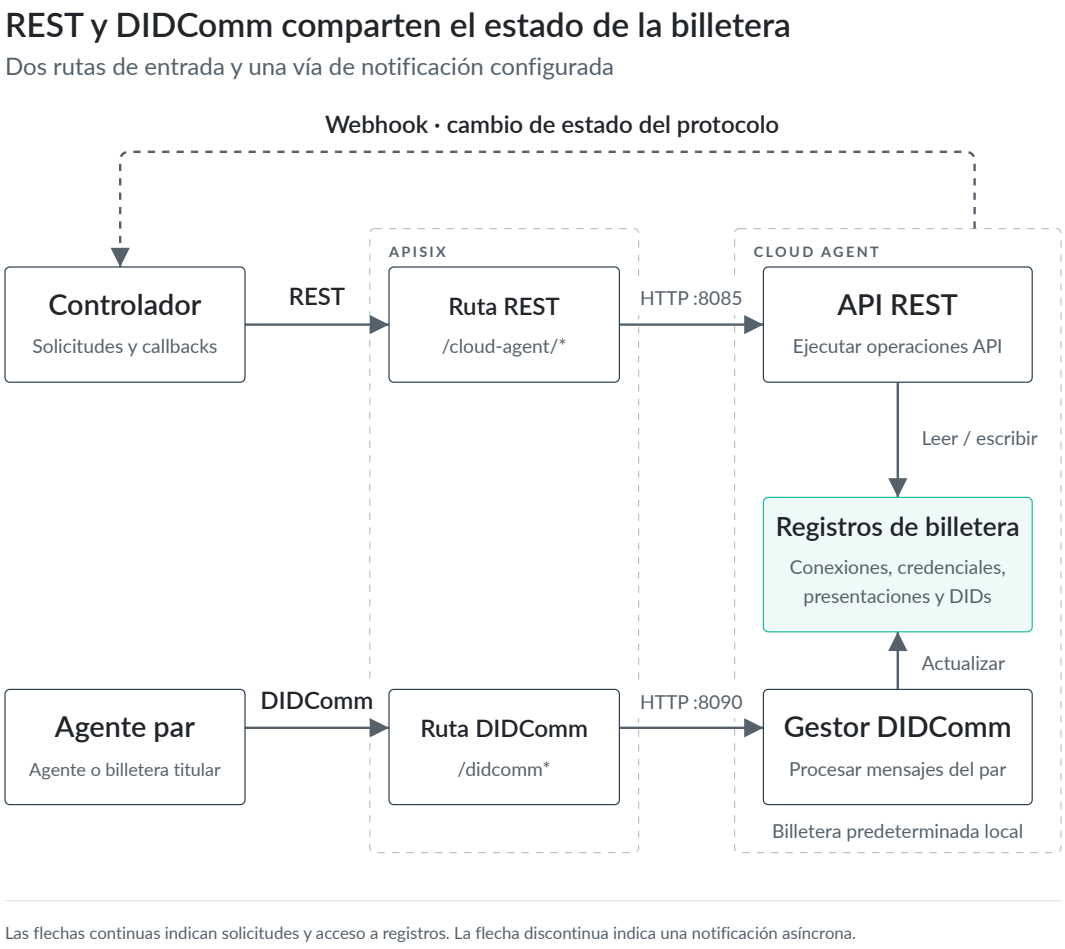


Figura 1: Una acción de API del controlador puede continuar mediante mensajes DIDComm posteriores que modifican el mismo registro de protocolo. Cloud Agent envía eventos de cambio de estado al punto de acceso webhook que configura el controlador.

El entorno local del capítulo anterior se ejecuta en modo de un solo inquilino. En ese modo, Cloud Agent usa la entidad predeterminada y la billetera predeterminada para las operaciones de la API REST y DIDComm. La documentación de autenticación de Cloud Agent define ambos identificadores como `00000000-0000-0000-0000-000000000000`.

Puedes confirmar la inicialización de la billetera predeterminada en los registros del contenedor:

```
docker logs local-cloud-agent-1 | grep "default"


... Initializing default wallet.
... Default wallet seed is not provided. New seed will be generated.
... Entity created:
↪  Entity(00000000-0000-0000-0000-000000000000,default,00000000-0000-0000
↪  -0000-000000000000,1970-01-01T00:00:00Z,1970-01-01T00:00:00Z)
```

Más adelante, la aplicación de ejemplo usará el modo de varios inquilinos. En ese modo, una instancia compartida de Cloud Agent atiende a varios inquilinos. La documentación de varios inquilinos define una entidad como la representación del inquilino y una billetera como el contenedor aislado de sus DIDs, conexiones, credenciales, claves, esquemas de credenciales y activos relacionados. Una billetera administrada por el servicio sirve para un emisor, un verificador o un servicio de billeteras con custodia en el servidor, donde el operador administra la ejecución, la base de datos, el almacenamiento de secretos, el acceso a la API y las copias de seguridad.

El punto de acceso `_system/health` del capítulo anterior informa de la versión del servicio en ejecución. La API del sistema expone otro punto de acceso para las métricas de ejecución:

```
curl http://localhost/cloud-agent/_system/metrics
# HELP jvm_memory_bytes_used
# TYPE jvm_memory_bytes_used gauge
jvm_memory_bytes_used{area="heap",} 1.07522616E8
jvm_memory_bytes_used{area="nonheap",} 1.74044936E8
...
```

La especificación OpenAPI de Cloud Agent describe el punto de acceso de métricas como una salida de texto Prometheus del registro interno de métricas. Úsalo cuando necesites datos de ejecución locales para investigar memoria, latencia o estado del servicio.

Especificación OpenAPI

La especificación OpenAPI (`OAS`) es un formato estándar para describir API HTTP. Ofrece a personas y herramientas un contrato común para rutas, cuerpos de solicitud, cuerpos de respuesta, esquemas de autenticación y esquemas de datos. El repositorio de código de Cloud Agent incluye un documento OpenAPI 3.1 con el título `Identus Cloud Agent API Reference`. El entorno Docker local expone ese documento mediante APISIX.

Para el agente local que iniciaste en el capítulo anterior, usa esta URL como contrato de API que corresponde a esa versión:

```
http://localhost/docs/cloud-agent/api/docs.yaml
```

Si iniciaste el agente con otro valor de `run.sh --port`, sustituye `localhost` por `localhost:<port>`. Swagger UI lee este mismo documento. Postman y los generadores de clientes pueden importarlo para crear una colección o un cliente auxiliar para la versión exacta de Cloud Agent que ejecutas.

La documentación alojada de Identus ofrece tutoriales actuales y conceptos en https://hyperledger-identus.github.io/docs/. Para trabajar con los puntos de acceso de este entorno local, usa de preferencia el documento OpenAPI que sirve tu agente en ejecución. Así evitas diferencias entre el libro, una página alojada y la versión de la imagen Docker.

Pasarela APISIX

APISIX actúa como proxy de los servicios de Cloud Agent dentro del entorno Docker local. El script `run.sh` vincula APISIX al puerto que seleccionaste; el puerto predeterminado es `80`. La configuración APISIX compartida de Cloud Agent define estas rutas:

Ruta local	Servicio de destino	Finalidad
http://localhost/cloud-agent/	`cloud-agent:8085`	API REST para aplicaciones controladoras.
http://localhost/docs/cloud-agent/api/docs.yaml	`cloud-agent:8085`	Documento OpenAPI del Cloud Agent en ejecución.
http://localhost/apidocs/	`swagger-ui:8080`	Swagger UI para llamadas interactivas a la API.
http://localhost/didcomm/	`cloud-agent:8090`	Punto de acceso público DIDComm para mensajes cifrados entre agentes.

El punto de acceso DIDComm es el punto de transporte que anuncian las invitaciones y los documentos DID. Un par lo usa para enviar mensajes DIDComm cifrados a este Cloud Agent. El capítulo sobre DIDComm explica el modelo de mensajes y el flujo del protocolo más adelante en el libro.

Las rutas de APISIX comparan las solicitudes entrantes con sus reglas, aplican los complementos de ruta y envían las solicitudes a los servicios de destino. La configuración local de Cloud Agent usa `proxy-rewrite` para eliminar el prefijo de ruta pública antes de enviar la solicitud, y habilita el complemento `cors` de APISIX en las rutas REST y DIDComm.

> **Advertencia**
>
> La configuración local de CORS de APISIX permite todos los orígenes en las rutas `/cloud-agent/*` y `/didcomm*`. Esa configuración facilita las comprobaciones locales desde el navegador. Un servicio desplegado debe limitar los orígenes permitidos a los dominios que alojan la aplicación controladora, las herramientas de administración o las interfaces de billetera que necesitan acceso desde un navegador.

Swagger UI

Swagger UI crea documentación de API y un formulario interactivo de solicitudes a partir de un documento OpenAPI. El entorno Docker local incluye un contenedor Swagger UI y APISIX lo expone en http://localhost/apidocs/.

Abre http://localhost/apidocs/ en tu navegador. En la lista de servidores, selecciona:

- `http://localhost/cloud-agent - The local instance of the Cloud Agent behind the APISIX proxy`.

Haz clic en `Authorize`. Swagger UI abre un cuadro de diálogo para la cabecera `apikey`. El entorno local desactiva la autenticación por clave de API de forma predeterminada, por lo que Cloud Agent acepta la solicitud aunque Swagger UI envíe cualquier valor. Por ahora, usa `test`. En capítulos posteriores, cuando el libro habilite la autenticación por clave de API, este campo debe contener la clave de API asignada a la entidad predeterminada o a la entidad del inquilino.

Después de autorizar, el cuadro de diálogo debe parecerse a este:

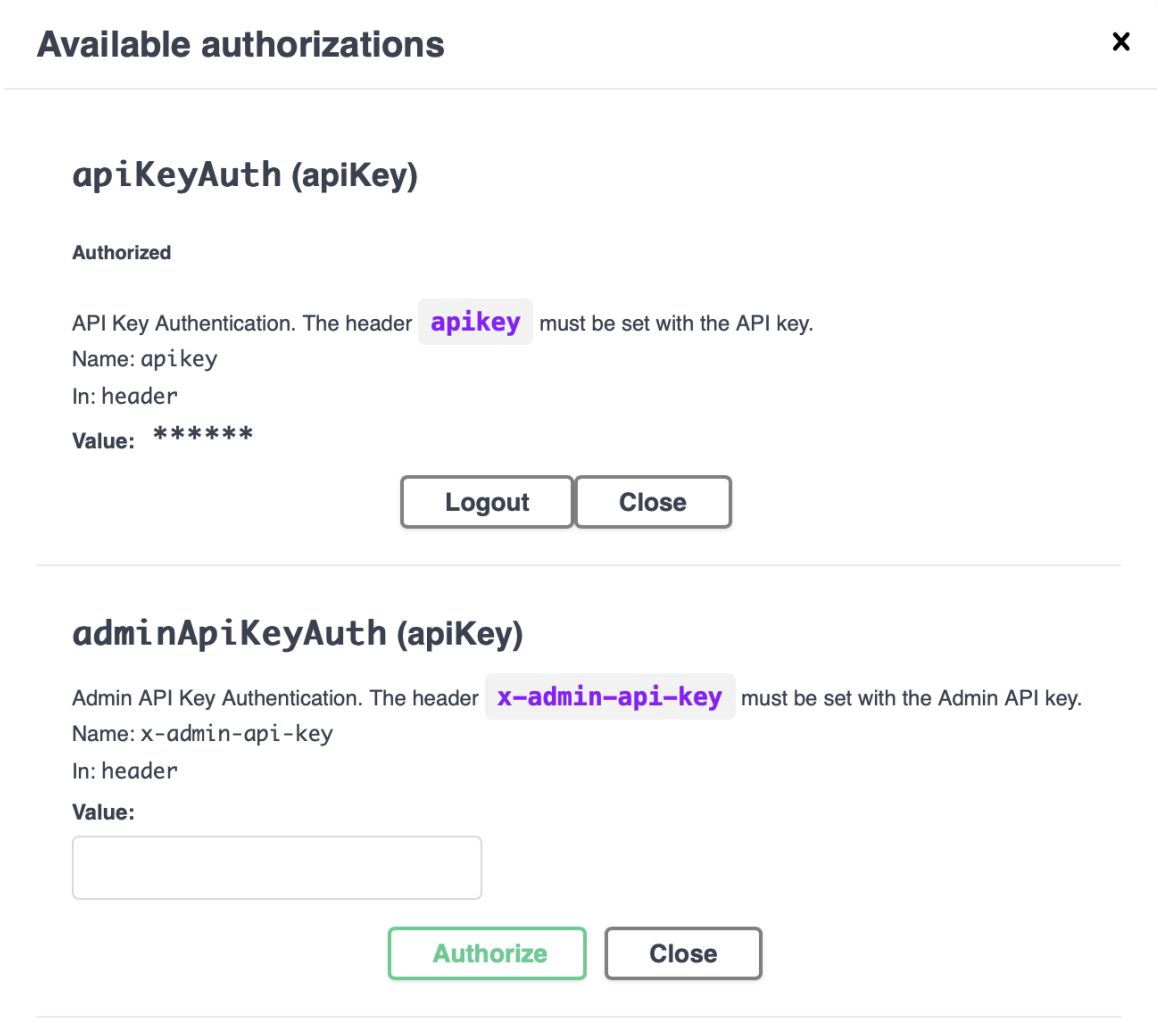


Figura 2: Cuadro de diálogo Apikey de Swagger UI

Cierra el cuadro de diálogo y prueba la primera solicitud. Expande `GET /connections` y haz clic en `Try it out`. Deja `offset`, `limit` y `thid` vacíos. Haz clic en `Execute` para enviar la solicitud.

La respuesta debe parecerse a esta:

```
{ "contents": [], "kind": "ConnectionsPage", "self": "", "pageOf": "" }
```

Un array `contents` vacío indica que la billetera predeterminada todavía no tiene registros de conexión. Este es el resultado esperado para un agente local nuevo.

> **Nota**
>
> Swagger UI puede generar un comando `curl` para la solicitud que envía. Puedes pegarlo en tu terminal para ejecutar la misma llamada de API fuera del navegador. Por ejemplo:
>
> ```
> curl -X 'GET' \
>   'http://localhost/cloud-agent/connections' \
>   -H 'accept: application/json' \
>   -H 'apikey: test'
> {"contents":[],"kind":"ConnectionsPage","self":"","pageOf":""}
> ```

Postman

Swagger UI sirve para comprobaciones rápidas con el agente en ejecución. Postman ofrece más opciones cuando necesitas entornos guardados, variables de solicitudes, aserciones mediante scripts, colecciones compartidas o comprobaciones manuales repetidas entre agentes emisores, titulares y verificadores.

Postman puede importar definiciones OpenAPI desde un archivo, una URL, texto YAML o JSON pegado, o un repositorio. Usa la URL OpenAPI del Cloud Agent local para que la colección corresponda a tu imagen Docker en ejecución.

1. Instala Postman si todavía no lo tienes.
2. Copia la URL OpenAPI local: `http://localhost/docs/cloud-agent/api/docs.yaml`. Si cambiaste el valor de `run.sh --port`, incluye ese puerto en la URL.
3. En Postman, ve a `File -> Import`.
4. Pega la URL OpenAPI en el cuadro de importación, o descarga el YAML desde esa URL e importa el archivo.
5. Importa la definición como una colección.

Después de importar, Postman debe mostrar una colección con el nombre `Identus Cloud Agent API Reference`. Si Postman no selecciona ese servidor, configura la URL base de la colección o la variable del servidor como `http://localhost/cloud-agent`.

Tutoriales

Los tutoriales oficiales de Identus cubren los flujos principales de la API: conexiones, administración de DIDs, esquemas de credenciales, emisión de credenciales, solicitudes de

presentación, webhooks, varios inquilinos e interacción con el VDR. Úsalos como referencia de los puntos de acceso para los próximos capítulos. Este libro conectará esas operaciones de API con la aplicación de ejemplo, el modelo de billetera, los flujos DIDComm y las comprobaciones de política del verificador.

Mediador

Descripción general

Hyperledger Identus Mediator es un servicio de enrutamiento DIDComm V2. Ofrece a las billeteras móviles, las billeteras de navegador y otros agentes con conectividad variable un punto de acceso estable donde otros agentes pueden enviar mensajes DIDComm cifrados.

En un flujo DIDComm con mediación, la billetera del titular establece primero la mediación con el mediador. El mediador devuelve información de enrutamiento. La billetera del titular anuncia después esa información en las entradas de servicio DIDComm y los registros de conexión posteriores. Un emisor o verificador que envía un mensaje DIDComm a ese titular cifra el mensaje para el titular, lo envuelve en un mensaje DIDComm `forward` para el mediador y envía el mensaje envuelto al punto de acceso del mediador. El mediador puede descifrar la envoltura externa de enrutamiento, pero no puede leer el cuerpo cifrado del mensaje del titular.

El mediador almacena los mensajes pendientes del titular en MongoDB. La billetera del titular usa después DIDComm Message Pickup para consultar el estado de la cola, solicitar la entrega y confirmar los mensajes recibidos. Este modelo permite que un emisor o verificador envíe mensajes cuando la billetera del titular está desconectada, sin revelar la dirección del dispositivo del titular a cada conexión.

El mediador no es un registro de datos verificables y no publica DIDs. Administra el transporte y el almacenamiento DIDComm. PRISM Node administra las operaciones `did:prism` respaldadas por el registro distribuido. Cloud Agent y los SDK de Edge Agent usan el mediador cuando su contraparte DIDComm necesita un punto de retransmisión estable.

Protocolos disponibles

El repositorio actual de Mediator documenta la compatibilidad con estos protocolos DIDComm:

- `https://didcomm.org/routing/2.0`
- `https://didcomm.org/coordinate-mediation/2.0`
- `https://didcomm.org/messagepickup/3.0`
- `https://didcomm.org/trust-ping/2.0`
- `https://didcomm.org/report-problem/2.0`
- `https://didcomm.org/basicmessage/2.0`

Para la configuración, los protocolos importantes son Coordinate Mediation, Routing y Message Pickup. Coordinate Mediation permite que una billetera receptora solicite mediación y registre claves de destinatario. Routing define el mensaje `forward` que envía tráfico a través del mediador. Message Pickup permite que la billetera receptora recupere los mensajes en cola.

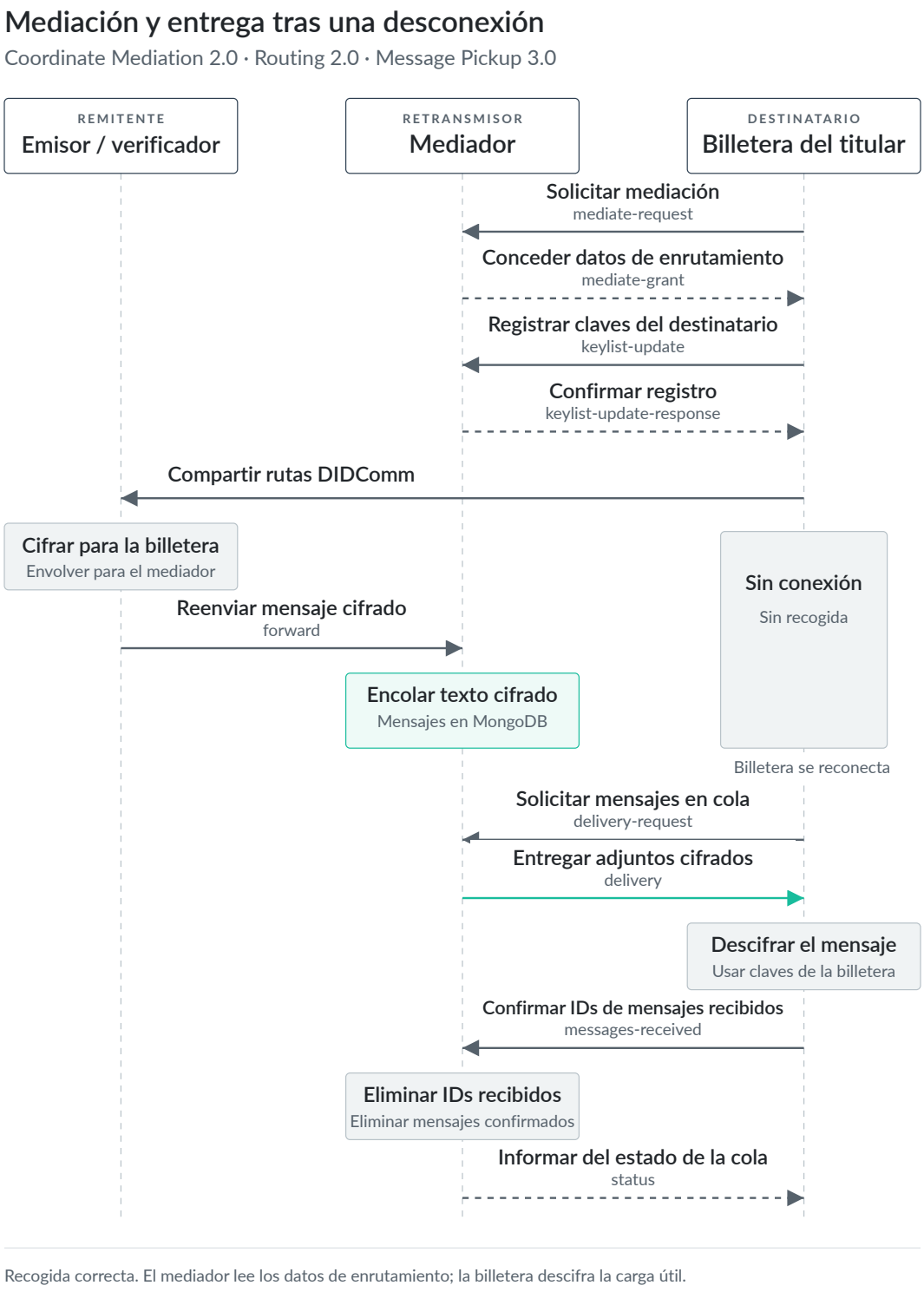


Figura 1: El mediador conserva los mensajes en cola hasta que la billetera confirma sus identificadores. Las solicitudes de recogida usan return_route: all para que el mediador pueda responder mediante la conexión de la billetera. El diagrama muestra la entrega después de un período sin conexión.

Selección de versión

Mediator tiene su propia línea de versiones, separada de las versiones de Identus Platform (consulta Capítulo). Consulta ambas antes de fijar una imagen de mediador. Identus Platform `v2.17` incluye Mediator `1.2.1`, que también es la versión estable más reciente del repositorio independiente de Mediator.

Este capítulo usa `MEDIATOR_VERSION=1.2.1`, que coincide con el conjunto de componentes de Identus Platform `v2.17`.

Requisitos previos

Instala Git, Docker y Docker Compose antes de ejecutar el entorno del mediador.

```
git --version
docker --version
docker compose version
```

Cada comando debe mostrar una versión. Si un comando falla, instala o repara esa herramienta antes de iniciar el mediador.

Clonar el repositorio del mediador

Clona el repositorio actual:

```
git clone https://github.com/hyperledger-identus/mediator identus-mediator
cd identus-mediator
```

El material antiguo de Identus puede usar `https://github.com/hyperledger/identus-mediator`. Los repositorios actuales de Identus pertenecen a la organización `hyperledger-identus` de GitHub.

Archivos de configuración local

El repositorio oficial ya incluye `docker-compose.yml` e `initdb.js`. No necesitas crear ninguno de los dos archivos cuando ejecutas el repositorio clonado. Lee ambos archivos antes de cambiar el despliegue del mediador. Esos archivos definen la imagen, la base de datos, las claves de identidad, los puntos de acceso públicos DIDComm y la comprobación de estado.

El archivo Compose local actual tiene esta estructura:

```yaml
services:
  mongo:
    image: mongo:6.0
    ports:
      - "27017:27017"
    command: ["--auth"]
    environment:
      - MONGO_INITDB_ROOT_USERNAME=admin
      - MONGO_INITDB_ROOT_PASSWORD=admin
      - MONGO_INITDB_DATABASE=mediator
    volumes:
      - ./initdb.js:/docker-entrypoint-initdb.d/initdb.js

  identus-mediator:
    image: docker.io/hyperledgeridentus/identus-mediator:${MEDIATOR_VERSIO
↪   N:-latest}
    ports:
      - "8080:8080"
    environment:
      - KEY_AGREEMENT_D=Z6D8LduZgZ6LnrOHPrMTS6uU2u5Btsrk1SGs4fn8M7c
      - KEY_AGREEMENT_X=Sr4SkIskjN_VdKTn0zkjYbhGTWArdUNE4j_DmUpnQGw
      - KEY_AUTHENTICATION_D=INXCnxFEl0atLIIQYruHzGd5sUivMRyQOzu87qVerug
      - KEY_AUTHENTICATION_X=MBjnXZxkMcoQVVL21hahWAw43RuAG-i64ipbeKKqwoA
      - SERVICE_ENDPOINTS=${SERVICE_ENDPOINTS:-http://localhost:8080;ws://
↪   localhost:8080/ws}
      - MONGODB_USER=admin
      - MONGODB_PASSWORD=admin
      - MONGODB_PROTOCOL=mongodb
      - MONGODB_HOST=mongo
      - MONGODB_PORT=27017
      - MONGODB_DB_NAME=mediator
    depends_on:
      - mongo
    healthcheck:
      test: curl --fail http://localhost:8080/health || exit 1
      interval: 30s
      timeout: 8s
      retries: 3
    extra_hosts:
      - "host.docker.internal:host-gateway"
```

El servicio `mongo` almacena cuentas de mediación y mensajes en cola. El archivo local expone MongoDB en el puerto `27017` del host para depuración. Las billeteras y los Cloud Agents no necesitan ese puerto del host. Los despliegues de producción deben mantener MongoDB dentro de la red privada del despliegue.

El servicio `identus-mediator` ejecuta la imagen del mediador de Docker Hub.

`MEDIATOR_VERSION` selecciona la etiqueta de la imagen. La asignación `ports` expone el servicio HTTP y WebSocket del mediador en el puerto `8080` del host. El mediador usa `SERVICE_ENDPOINTS` cuando construye su DID de pares y su invitación fuera de banda. Por tanto, ese valor debe indicar un punto de acceso al que pueda conectarse la billetera del titular.

Los valores de clave de demostración del archivo Compose local crean una identidad estable del mediador para el desarrollo local. No reutilices esas claves para un mediador compartido o de producción. Genera un nuevo par de claves X25519 para el acuerdo de claves y un nuevo par de claves Ed25519 para la autenticación. Proporciona después los cuatro campos JWK mediante la configuración del despliegue.

El archivo de inicialización de Mongo crea el usuario de base de datos, las colecciones, los índices y la regla TTL:

```
db.createUser({
    user: "admin",
    pwd: "admin",
    roles: [
        { role: "readWrite", db: "mediator" }
    ]
});

const database = 'mediator';
const collectionDidAccount = 'user.account';
const collectionMessages = 'messages';
const collectionMessagesSend = 'messages.outbound';

use(database);

db.createCollection(collectionDidAccount);
db.createCollection(collectionMessages);
db.createCollection(collectionMessagesSend);

db.getCollection(collectionDidAccount).createIndex({ 'did': 1 }, { unique:
 ↪  true });
db.getCollection(collectionDidAccount).createIndex({ 'alias': 1 }, {
 ↪  unique: true, partialFilterExpression: { "alias.0": { $exists: true } }
 ↪  });
db.getCollection(collectionDidAccount).createIndex({ "messagesRef.hash": 1,
 ↪  "messagesRef.recipient": 1 });

const expireAfterSeconds = 7 * 24 * 60 * 60;
db.getCollection(collectionMessages).createIndex(
    { ts: 1 },
    {
        name: "message-ttl-index",
```

```
        partialFilterExpression: { "message_type" : "Mediator" },
        expireAfterSeconds: expireAfterSeconds
    }
)
```

`user.account` almacena registros de cuentas de mediación con el DID como clave. `messages` almacena mensajes de protocolo del mediador y referencias a cargas útiles de usuarios en cola. `messages.outbound` almacena registros de mensajes salientes. El índice TTL selecciona registros cuyo `message_type` es `Mediator`; los mensajes de usuarios en cola permanecen hasta que el destinatario los recupera y confirma su recepción.

Un archivo `.env` local puede fijar la imagen del mediador y sus puntos de acceso públicos:

```
MEDIATOR_VERSION=1.2.1
SERVICE_ENDPOINTS=http://localhost:8080;ws://localhost:8080/ws
```

Usa una dirección IP de la LAN o un dominio público en `SERVICE_ENDPOINTS` cuando una billetera móvil deba conectarse desde fuera de la máquina anfitriona. Mantén el esquema de URL y el puerto en correspondencia con el punto de acceso que expones mediante Docker, un proxy inverso o un balanceador de carga.

Iniciar el mediador

El repositorio incluye el archivo local `docker-compose.yml`. Este inicia dos servicios:

- `mongo`, la base de datos MongoDB para cuentas y mensajes en cola.
- `identus-mediator`, el servicio de mediador JVM expuesto en el puerto `8080`.

El archivo Compose actual asigna el puerto `8080` del host al contenedor del mediador. Comprueba que ningún otro servicio local use ese puerto antes del inicio:

```
docker ps --format '{{.Names}} {{.Ports}}' | grep 8080
```

Si el comando muestra otro contenedor que usa `8080`, detén ese servicio o adapta la asignación de puertos de Compose antes de iniciar el mediador. Mantén el puerto expuesto del host en correspondencia con `SERVICE_ENDPOINTS`, porque las billeteras leen `SERVICE_ENDPOINTS` de la invitación DIDComm del mediador.

Ejecuta el comando para tu sistema operativo desde la raíz del repositorio.

macOS:

```
MEDIATOR_VERSION=1.2.1 SERVICE_ENDPOINTS="http://$(ipconfig getifaddr
 ↪  $(route get default | grep interface | awk '{print
 ↪  $2}')):8080;ws://$(ipconfig getifaddr $(route get default | grep
 ↪  interface | awk '{print $2}')):8080/ws" docker compose up -d
```

Linux:

```
MEDIATOR_VERSION=1.2.1 SERVICE_ENDPOINTS="http://$(ip addr show $(ip route
 ↪  show default | awk '/default/ {print $5}') | grep 'inet ' | awk '{print
 ↪  $2}' | cut -d/ -f1):8080;ws://$(ip addr show $(ip route show default |
 ↪  awk '/default/ {print $5}') | grep 'inet ' | awk '{print $2}' | cut -d/
 ↪  -f1):8080/ws" docker compose up -d
```

El valor de `SERVICE_ENDPOINTS` pasa a formar parte de las entradas de servicio del DID de pares y de la invitación fuera de banda del mediador. Usa una dirección a la que pueda conectarse la billetera. Una billetera de navegador que se ejecuta en el mismo host puede usar `localhost`; una billetera móvil en la misma red necesita la dirección IP de la LAN de la máquina anfitriona o un dominio público. Para despliegues de producción, usa puntos de acceso HTTPS y WSS.

Comprobar el mediador en ejecución

Comprueba los contenedores:

```
docker compose ps
```

El servicio del mediador define una comprobación de estado de Docker contra `/health`. Después del inicio, Docker debe indicar que `identus-mediator` está en buen estado.

Comprueba el punto de acceso de estado:

```
curl -i http://localhost:8080/health
```

Resultado esperado:

```
HTTP/1.1 200 OK
```

Comprueba la versión del mediador en ejecución:

```
curl http://localhost:8080/version
```

Resultado esperado:

```
1.2.1
```

Obtén el DID de pares del mediador:

```
curl http://localhost:8080/did
```

La respuesta debe comenzar con `did:peer:`. El software de la billetera del titular usa este DID como DID del mediador cuando inicia la mediación.

Obtén la invitación de mediación:

```
curl http://localhost:8080/invitation
```

La respuesta es una invitación DIDComm fuera de banda. Su campo `from` contiene el DID de pares del mediador. Su cuerpo debe incluir `goal_code` con el valor `request-mediate` y `accept` con `didcomm/v2`.

Para obtener la invitación codificada para URL que usan los flujos con códigos QR, llama a:

```
curl http://localhost:8080/invitationOOB
```

La respuesta debe contener un parámetro `_oob=`.

Notas de configuración

El archivo Compose local incluye claves de identidad de demostración. No las uses fuera del desarrollo local. Un mediador de producción necesita su propia clave de acuerdo de claves X25519 y su clave de autenticación Ed25519. El repositorio de Mediator incluye una guía para generar los valores `KEY_AGREEMENT_D`, `KEY_AGREEMENT_X`, `KEY_AUTHENTICATION_D` y `KEY_AUTHENTICATION_X` en un formato compatible con JWK.

Puedes configurar MongoDB con variables individuales:

```
MONGODB_PROTOCOL
MONGODB_HOST
MONGODB_PORT
MONGODB_USER
MONGODB_PASSWORD
MONGODB_DB_NAME
```

Mediator `1.2.0` añadió `MONGODB_CONNECTION_STRING`. Si está presente, esta cadena de conexión completa tiene prioridad sobre las variables individuales de MongoDB.

El mediador almacena dos categorías de mensajes. Los mensajes `Mediator` cubren la configuración de mediación, las operaciones de listas de claves, las solicitudes de recogida y otro tráfico de protocolo del mediador. Estos registros pueden caducar mediante el índice TTL que configura `initdb.js`. Los mensajes `User` son cargas útiles cifradas que esperan la recogida del titular. El mediador los conserva hasta que el destinatario los recupera y confirma su recepción mediante Message Pickup.

Detener el mediador

Detén el entorno sin eliminar el volumen de la base de datos:

```
docker compose down
```

Elimina los contenedores y los datos locales de MongoDB:

```
docker compose down -v
```

Usa `-v` cuando quieras un estado limpio del mediador local. Omítelo cuando quieras conservar las cuentas de mediación registradas y los mensajes de desarrollo en cola entre reinicios.

Sección III: construcción

Proyecto de ejemplo

Descripción general: billetera de billetes de avión

Una de las mejores formas de aprender y comprender una plataforma de software es construir algo con ella. Hemos escrito una aplicación Identus de ejemplo, de código abierto, en TypeScript, Swift y Kotlin para mostrar cómo crear soluciones para casos reales.

Flight Tix muestra cómo emitir credenciales verificables que representan a usuarios y objetos, y cómo presentar pruebas y verificar esas credenciales.

Flight Tix está disponible para cada SDK de Identus Edge. Consulta el repositorio de cada ejemplo para obtener más detalles:

- TypeScript: https://github.com/identusbook/example-ts
- Swift: https://github.com/identusbook/example-ios
- Kotlin: https://github.com/identusbook/example-android

Billeteras

Descripción general

Una billetera SSI es software que almacena y protege el material que necesita un titular, emisor o verificador para actuar en un protocolo de identidad. En el modelo de datos de credenciales verificables 2.0 de W3C, un repositorio de credenciales almacena las credenciales del titular y protege el acceso a ellas. En DID Core de W3C, un DID se resuelve en un documento DID que puede exponer métodos públicos de verificación y puntos de acceso de servicios. Las claves privadas, las credenciales, los registros DIDComm y el estado local de la billetera permanecen en el almacenamiento de la billetera, no en el documento DID.

Identus usa la palabra billetera en dos contextos habituales:

- Una billetera de Edge Agent se ejecuta dentro de una aplicación para el titular, como una aplicación de navegador, móvil o de escritorio. La aplicación incorpora un SDK de Identus Edge y proporciona almacenamiento mediante Pluto.
- Una billetera de Cloud Agent es un contenedor de billetera en el servidor al que se accede mediante la API REST de Cloud Agent. Un controlador o inquilino usa llamadas de API y webhooks en lugar de llamadas directas al SDK dentro del proceso.

No trates un registro distribuido, PRISM Node o un resolvedor DID como una copia de seguridad de la billetera. El estado público del DID puede ayudar a otras partes a resolver las claves del emisor y los puntos de acceso de servicios. No puede reconstruir las credenciales que conserva el titular, las claves privadas de DIDs de pares, el historial de mensajes DIDComm, los registros de conexión ni el estado de las presentaciones. Una billetera de producción necesita un diseño explícito de almacenamiento, cifrado, copias de seguridad y recuperación.

En el ejemplo de billetes de avión de esta sección, la aerolínea ejecuta Cloud Agent como emisor. El viajero usa una billetera de Edge Agent como titular. El personal de seguridad del aeropuerto puede usar Cloud Agent, un SDK de Edge u otra implementación de verificador, según el despliegue.

Responsabilidades de la billetera

Una billetera tiene más tareas que almacenar credenciales. Administra el estado local necesario para continuar los flujos de protocolo tras reinicios de la aplicación, cambios de dispositivo e interrupciones de red.

La billetera almacena o referencia:

- Registros DID y metadatos DID.
- Claves privadas, semillas de billetera o referencias a secretos que usan las operaciones DID.
- Credenciales verificables recibidas de emisores.
- Registros de presentaciones creadas para verificadores.
- Mensajes DIDComm, registros de conexión y estado del protocolo.
- Configuración del mediador para la entrega sin conexión.
- Referencias al estado de credenciales y a esquemas que necesitan los flujos de verificación posteriores.

La billetera usa ese estado para ejecutar el protocolo. La billetera del titular recibe ofertas de credenciales, solicita el consentimiento del titular, almacena las credenciales emitidas, crea presentaciones y envía respuestas DIDComm. La billetera del emisor firma credenciales y mantiene registros de emisión. La billetera o el agente del verificador recibe presentaciones, comprueba las pruebas criptográficas, resuelve el material del emisor, comprueba el estado de las credenciales y aplica después la política del verificador.

Mantén separadas las capas de verificación. Una comprobación criptográfica determina si la presentación, la prueba de la credencial, la clave del emisor, la vinculación del titular y los datos de estado superan la verificación. Una comprobación de política determina si el verificador acepta ese emisor, tipo de credencial, esquema, sujeto, antigüedad o nivel de garantía para la acción de negocio.

Billeteras de Edge Agent

Los SDK de Identus Edge Agent permiten que una aplicación ejecute las funciones de billetera y agente cerca del titular. Los repositorios actuales de Identus ofrecen:

- SDK de TypeScript para aplicaciones de navegador y Node.js. La documentación actual del SDK de TypeScript indica el paquete `@hyperledger/identus-sdk` y señala el cambio de nombre desde `@hyperledger/identus-edge-agent-sdk`.
- SDK de Swift para plataformas Apple.
- SDK de Kotlin Multiplatform para aplicaciones Android y JVM.

Los SDK usan el mismo vocabulario de arquitectura:

- Apollo proporciona operaciones criptográficas.
- Castor crea, administra y resuelve DIDs.
- Pollux administra operaciones de credenciales en los SDK de Swift y Kotlin. El SDK actual de TypeScript expone las funciones de credenciales mediante sus módulos y complementos.
- Mercury administra mensajes DIDComm v2 y las tareas relacionadas de mensajería segura.
- Pluto proporciona la interfaz de almacenamiento.
- Agent o EdgeAgent combina los componentes en una API de agente de nivel superior.

Empieza con Agent o EdgeAgent para el código de la aplicación. Las llamadas directas a Apollo, Castor, Mercury, Pollux o Pluto sirven cuando la aplicación necesita una operación de nivel inferior que la API del agente no expone. Por ejemplo, una billetera que implementa un protocolo DIDComm propio puede construir y empaquetar mensajes mediante Mercury y almacenar después los registros de protocolo resultantes mediante la misma capa de almacenamiento de la billetera.

Almacenamiento Pluto

Pluto es el límite principal de implementación en una billetera de Edge Agent. Identus define la interfaz de almacenamiento, y la aplicación proporciona la implementación que corresponde a su plataforma y modelo de amenazas.

En una billetera de navegador, la implementación puede usar IndexedDB u otra base de datos del navegador. En una billetera móvil, puede usar la base de datos de la plataforma y la protección de claves mediante Keychain o Keystore. En una billetera de prueba en el servidor, un almacén en memoria sirve para una ejecución desechable. El almacenamiento de producción debe cifrar los datos sensibles en reposo, vincular las claves de cifrado al modelo de seguridad de la plataforma cuando sea posible y admitir copias de seguridad y restauración comprobadas.

El diseño del almacenamiento cambia el comportamiento de la billetera. Si la aplicación almacena credenciales pero pierde las claves privadas, el titular puede perder la capacidad de demostrar el control del DID que usó durante la emisión o presentación. Si la aplicación almacena claves pero pierde las credenciales, el titular aún puede controlar el DID pero ya no tiene la carga útil de la credencial. Si la aplicación pierde los registros de protocolo DIDComm, puede no reanudar los flujos de emisión, conexión o prueba después de un reinicio.

Trata los almacenes Pluto de la comunidad como dependencias que debes evaluar, no como opciones predeterminadas de Identus. Comprueba su mantenimiento, comportamiento de cifrado, compatibilidad con migraciones y compatibilidad con la versión del SDK de la aplicación antes de elegir uno.

Billeteras de Cloud Agent

Cloud Agent se ejecuta en un servidor y expone operaciones de billetera mediante API REST y flujos basados en webhooks. El README de Cloud Agent lo describe como un agente en la nube basado en estándares de W3C y Aries que admite DIDComm v2, emisión, verificación y tenencia de credenciales, administración de secretos y varios inquilinos.

Una billetera de Cloud Agent sirve para servicios que necesitan operaciones de identidad en el servidor. Una aerolínea puede usar Cloud Agent para administrar DIDs del emisor, firmar credenciales de billetes, enviar ofertas y seguir el estado de emisión. Un verificador del aeropuerto puede usar Cloud Agent para solicitar presentaciones y recibir resultados de

verificación. Una empresa puede ejecutar Cloud Agent en modo de varios inquilinos cuando cada cliente o departamento necesita una billetera separada.

La custodia de Cloud Agent tiene un límite de confianza distinto al de una billetera de Edge Agent. El inquilino o controlador del negocio interactúa con la billetera mediante una API. El operador del servicio controla la ejecución, la base de datos, el almacenamiento de secretos, la red y el acceso operativo. Una billetera administrada por el servicio puede servir para organizaciones y billeteras de usuarios delegadas, pero el despliegue debe definir quién puede administrar billeteras, exportar datos, rotar secretos, recuperar semillas e inspeccionar registros.

Billeteras de varios inquilinos

En el modo de varios inquilinos de Cloud Agent, un administrador crea una billetera, crea una entidad para el inquilino y configura un método de autenticación para esa entidad. La documentación de incorporación de inquilinos de Identus describe la billetera como el contenedor de los activos del inquilino, incluidos DIDs, credenciales y conexiones. Después de la configuración, Cloud Agent limita las llamadas de API del inquilino a esa billetera.

Este modelo sirve para billeteras administradas por el servicio. No elimina el riesgo de custodia. La API de administración, la autenticación del inquilino, los identificadores de billetera, las filas de la base de datos, las rutas de Vault y el enrutamiento de webhooks deben preservar la separación entre inquilinos.

Material de claves de Cloud Agent

Identus Cloud Agent usa vías distintas de administración de claves para los DIDs PRISM y los DIDs de pares.

Para los DIDs PRISM administrados, Cloud Agent deriva el material de claves a partir de una semilla de billetera y una ruta de derivación. La documentación de administración de DIDs indica que Cloud Agent almacena la ruta de derivación en lugar del propio material de claves PRISM y reconstruye después el material de claves durante la ejecución a partir de la semilla y la ruta.

Para los DIDs de pares administrados, Cloud Agent genera el material de claves y lo almacena en el almacenamiento de secretos. Los DIDs de pares admiten actividades DIDComm, por lo que estas claves protegen las comunicaciones específicas de cada relación.

La documentación de almacenamiento de secretos de Cloud Agent indica HashiCorp Vault como servicio predeterminado de almacenamiento de secretos. Describe tipos de secretos como semillas, claves privadas, datos de definiciones de credenciales y secretos de vinculación de AnonCreds. En la configuración de varios inquilinos, las rutas de activos de Vault incluyen el ID de la billetera para que los secretos de cada billetera tengan su propia ruta.

Elegir la custodia Edge o Cloud

Usa una billetera de Edge Agent cuando el titular deba conservar las credenciales y claves en una aplicación bajo su control. Este modelo sirve para billeteras de viajeros, empleados o ciudadanos y para la presentación móvil de pruebas. El equipo de la aplicación debe administrar el almacenamiento, el cifrado, las copias de seguridad, la recuperación, el uso del mediador y el consentimiento local del usuario.

Usa una billetera de Cloud Agent cuando un servidor deba actuar por una organización o un inquilino delegado. Este modelo sirve para emisores, verificadores, billeteras de organizaciones administradas por un servicio y flujos de backend que necesitan API REST y webhooks. El equipo de despliegue debe administrar el almacenamiento de secretos, la autenticación, la separación entre inquilinos, la supervisión y los controles operativos.

Muchos sistemas Identus usan ambos modelos. El emisor y el verificador ejecutan Cloud Agent. El titular usa una billetera de Edge Agent. DIDComm, las credenciales, los DIDs y los protocolos de presentación conectan ambos modelos de custodia sin exigir la misma implementación de billetera en cada parte.

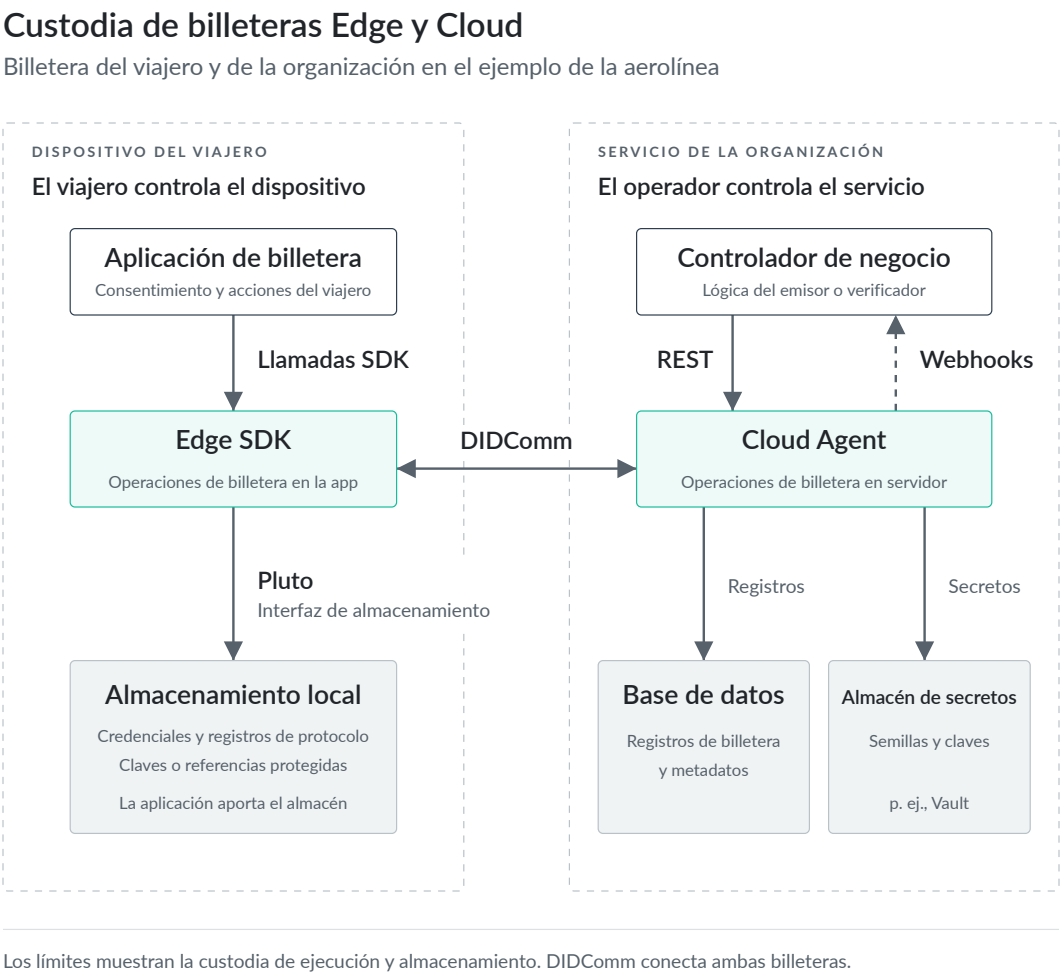


Figura 1: El equipo de la aplicación implementa el almacenamiento, la protección de claves y la recuperación para la billetera del viajero. El operador del servicio administra la ejecución de Cloud Agent, la base de datos, el almacenamiento de secretos y la recuperación. El enlace DIDComm representa el intercambio de protocolo; la entrega puede usar un mediador.

Flujo del billete de avión

El ejemplo de billetes de avión usa el límite de la billetera de forma concreta:

1. La aerolínea crea o selecciona una billetera de Cloud Agent para las operaciones de emisión.
2. La aerolínea crea un `did:prism` del emisor con un método de verificación `assertionMethod` y publica el DID cuando los verificadores necesitan una resolución respaldada por el registro distribuido.

3. El viajero abre una billetera de Edge Agent. La billetera crea o selecciona DIDs del titular y almacena el material secreto local mediante Pluto.
4. La aerolínea envía una oferta de credencial de billete a la billetera del viajero mediante DIDComm u otra vía de emisión compatible.
5. La billetera del viajero muestra la oferta, registra el consentimiento del titular, solicita la credencial, la recibe y la almacena.
6. El personal de seguridad del aeropuerto solicita pruebas del estado del billete, el vuelo, la vinculación del pasajero u otro conjunto de afirmaciones aceptado.
7. La billetera del viajero crea una presentación a partir de la credencial almacenada y la envía al verificador.
8. El software del verificador comprueba la prueba de la presentación, la resolución del DID del emisor, el estado de la credencial, la vinculación del titular, el esquema y las reglas específicas del formato.
9. La política de seguridad del aeropuerto decide si el resultado verificado satisface la regla del puesto de control.

Los pasos 8 y 9 son decisiones separadas. El software del verificador puede informar de que la prueba de la credencial es válida, el DID del emisor se resolvió y la credencial está activa. La política del aeropuerto aún puede rechazar la presentación si la fecha del vuelo, el aeropuerto, el emisor, el tipo de credencial o la vinculación del pasajero no corresponde a la regla del puesto de control.

Notas de implementación

Para billeteras de Edge Agent:

- Elige el SDK para la plataforma anfitriona antes de diseñar el almacenamiento.
- Trata Pluto como un componente obligatorio de la billetera, no como una caché opcional.
- Cifra el contenido de la billetera que incluya credenciales, registros DIDComm, semillas, claves privadas o referencias a secretos.
- Comprueba la restauración en un dispositivo o perfil de navegador limpio.
- Usa `did:peer` para la actividad DIDComm específica de cada relación y `did:prism` para la resolución pública.

Para billeteras de Cloud Agent:

- Decide si el despliegue usa una billetera predeterminada o varios inquilinos antes de incorporar usuarios.
- Mantén separadas las credenciales de administración y las de los inquilinos.
- Guarda las semillas de billetera y el material de claves privadas en el almacenamiento de secretos configurado.
- Crea copias de seguridad de la base de datos de Cloud Agent y del almacenamiento de secretos como una sola unidad de recuperación.
- Audita la entrega de webhooks, las claves de API de los inquilinos, los IDs de billetera y las rutas de Vault durante la incorporación de inquilinos.

Un equipo completa el trabajo de billetera solo después de comprobar la restauración y la reanudación del protocolo. Una billetera que puede emitir o recibir credenciales durante una demostración del flujo normal aún puede fallar en producción si el titular cambia de dispositivo, borra el almacenamiento del navegador, crece la cola del mediador o un inquilino de Cloud Agent necesita recuperarse después de restaurar la base de datos.

DIDComm

Descripción general

DIDComm (*Decentralized Identifier Communication*, comunicación mediante identificadores descentralizados) es un componente esencial del ecosistema de identidad autosoberana (SSI). Proporciona la capa de mensajería segura que permite interactuar a los identificadores descentralizados (DIDs). La especificación actual, **DIDComm v2**, evolucionó a partir de versiones anteriores para responder a las necesidades crecientes de privacidad, seguridad e interoperabilidad en las comunicaciones de identidad digital. Su propósito fundamental es permitir una comunicación segura, privada y autenticada entre entidades identificadas por DIDs, con independencia de su infraestructura o conjunto de tecnologías. Así transforma los DIDs de identificadores estáticos en puntos de comunicación dinámicos capaces de participar en interacciones complejas.

En el conjunto de tecnologías SSI, DIDComm se sitúa por encima de la capa básica de DID, que proporciona identificadores y material criptográfico, y por debajo de los protocolos específicos de aplicación que definen interacciones como el intercambio de credenciales. Esta posición convierte a DIDComm en el vínculo entre las interacciones SSI de Identus y permite todas las interacciones de identidad de nivel superior.

Características principales

El diseño de DIDComm incorpora varias características que lo hacen adecuado para las comunicaciones de identidad descentralizada:

La independencia del transporte permite que los mensajes DIDComm viajen por casi cualquier canal de comunicación: HTTP(S), WebSockets, Bluetooth, NFC, redes de malla o incluso códigos QR. Esta flexibilidad evita que las interacciones de identidad dependan de infraestructuras de red específicas y permite comunicaciones entre pares incluso sin conexión o con conectividad intermitente.

El cifrado de extremo a extremo garantiza que el contenido de los mensajes permanezca confidencial entre el remitente y los destinatarios previstos, incluso cuando atraviesa redes o intermediarios que no son de confianza. Mediante criptografía de clave pública derivada de los DIDs, DIDComm crea canales seguros que protegen la información sensible de identidad.

La autenticación y la confianza forman parte de DIDComm, porque cada mensaje permite verificar criptográficamente que procede de un DID específico. Esta autenticación no

necesita autoridades centrales y permite que las partes establezcan confianza directamente entre sí.

Las funciones asíncronas permiten comunicaciones que no requieren que ambas partes estén conectadas al mismo tiempo. Los mensajes pueden entrar en cola, almacenarse y reenviarse hasta llegar a su destino. Por ello, DIDComm sirve para numerosos casos reales donde la conectividad continua no está garantizada.

Los mecanismos de enrutamiento y reenvío de mensajes permiten que las comunicaciones recorran rutas complejas, incluso mediante mediadores y retransmisores, y mantengan la seguridad de extremo a extremo. Esta capacidad resulta esencial para entidades detrás de cortafuegos o NAT, o para aumentar la privacidad al ocultar metadatos de red.

Estructura del mensaje

Los mensajes de DIDComm Messaging usan envolturas cifradas JSON Web Message (JWM). Este formato estándar garantiza la interoperabilidad y proporciona las propiedades de seguridad necesarias.

Un mensaje DIDComm típico consta de:

- **Cabeceras**: metadatos del mensaje, incluidos identificadores, tipos, información temporal y detalles de enrutamiento.
- **Cuerpo**: el contenido o la carga útil del mensaje.
- **Adjuntos**: datos adicionales opcionales, que pueden incluir datos estructurados, documentos o medios.

Estos componentes forman un JWM que normalmente se cifra o firma según los requisitos de seguridad de la interacción. La estructura resultante proporciona un formato uniforme de envoltura que puede transportar cualquier tipo de contenido y garantizar su seguridad e integridad.

Cada mensaje contiene un identificador único y puede referenciar otros mensajes mediante mecanismos de hilos de conversación. Esto permite conversaciones complejas con varios mensajes. Los mensajes también declaran su tipo, que los vincula con protocolos específicos que definen la secuencia y el significado esperados de la interacción.

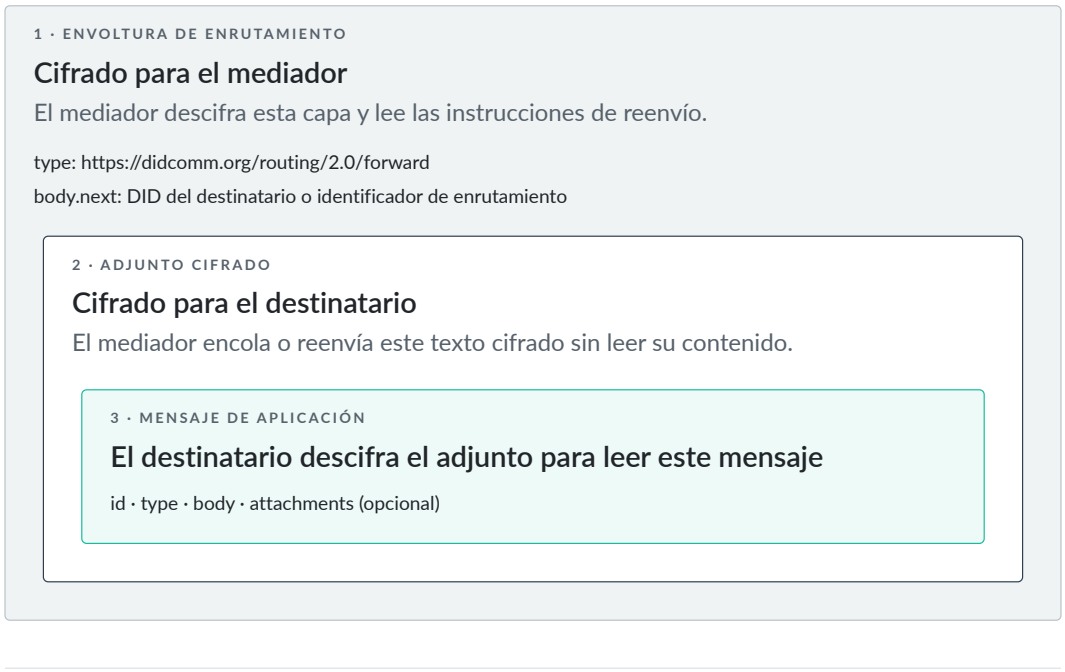


Figura 1: El remitente cifra el mensaje de aplicación para el destinatario, adjunta ese texto cifrado a un mensaje forward y cifra el mensaje externo para el mediador. Las capas expandidas muestran el contenido después de cada paso de descifrado.

Protocolos

DIDComm define formatos de mensaje y un marco para protocolos: secuencias estándar de mensajes que cumplen tareas específicas. Estos protocolos permiten la interoperabilidad al garantizar que distintas implementaciones comprendan las mismas secuencias de mensajes y sus significados.

El marco de protocolos incluye mecanismos para descubrir qué protocolos admite un agente y para gestionar sus versiones a medida que evolucionan. Esto permite actualizaciones graduales y compatibilidad con versiones anteriores en el ecosistema.

El ecosistema SSI ha desarrollado varios protocolos comunes:

- **Los protocolos de establecimiento de conexión** definen cómo dos partes pueden establecer un canal DIDComm seguro.

- **Los protocolos de emisión de credenciales** estandarizan cómo ofrecer y emitir credenciales verificables.
- **Los protocolos de presentación de credenciales** definen cómo solicitar y presentar credenciales.
- **Los protocolos de ping de confianza** ofrecen mecanismos simples para verificar que un canal de comunicación está activo.

Al estandarizar estos patrones de interacción, DIDComm permite que distintas implementaciones cooperen sin dificultades y contribuye a un ecosistema abierto en lugar de soluciones aisladas.

Puedes encontrar todos los protocolos publicados actualmente aquí. La comunidad los mantiene. También puedes construir y definir tus propios protocolos para tu caso de uso específico, aunque decidas no publicarlos.

Interacción de ejemplo

Para ilustrar cómo funciona DIDComm en la práctica, considera un caso típico de emisión de credenciales:

1. Un emisor y un titular establecen una conexión DIDComm e intercambian DIDs y puntos de acceso de servicios.
2. El emisor envía un mensaje de oferta que indica qué credenciales puede proporcionar.
3. El titular responde con un mensaje de solicitud de una credencial específica.
4. El emisor crea la credencial y la envía en un mensaje de emisión.
5. El titular confirma la recepción mediante un mensaje de confirmación.

Durante esta interacción, todos los mensajes usan cifrado de extremo a extremo. Aunque pasen por intermediarios, el contenido permanece privado entre el emisor y el titular. Los mensajes pueden viajar mediante distintos mecanismos de transporte: por ejemplo, comenzar con una conexión HTTPS y cambiar a Bluetooth para mensajes posteriores si las partes se acercan.

En los casos con mediación, el flujo adquiere más complejidad y capacidad. Un titular puede recibir mensajes mediante un servicio de mediador que proporciona una entrega constante incluso cuando el dispositivo del titular está desconectado. El mediador recibe mensajes cifrados, los almacena hasta que el titular se conecta y los reenvía sin poder leer nunca el contenido cifrado.

Ventajas

DIDComm ofrece varias ventajas esenciales que apoyan los principios básicos de la identidad autosoberana:

La protección de la privacidad ocupa un lugar central en el diseño de DIDComm. Al cifrar el contenido de los mensajes y permitir interacciones con seudónimos mediante DIDs

específicos de cada relación, DIDComm permite que las personas controlen qué información comparten y con quién. La independencia del transporte también ayuda a evitar la correlación mediante metadatos de red.

La comunicación descentralizada libera las interacciones de identidad de la dependencia de proveedores centrales de mensajería o centros de identidad. Las partes pueden comunicarse directamente o mediante los mediadores que elijan, y evitar la dependencia de un proveedor y los puntos únicos de fallo.

La interoperabilidad entre distintas implementaciones SSI mantiene el ecosistema abierto y competitivo. Un titular que usa una implementación de billetera puede interactuar sin dificultades con verificadores y emisores que usan conjuntos de software distintos, siempre que todos admitan los protocolos DIDComm.

El control del usuario abarca los datos de identidad y los propios canales de comunicación. Las personas pueden elegir cómo, cuándo y dónde reciben comunicaciones relacionadas con su identidad digital. Esto refuerza la naturaleza autosoberana del sistema.

DIDComm en Identus

Identus ha adoptado DIDComm v2 como protocolo estándar de comunicación y lo implementa en todo el marco para permitir mensajería segura y privada entre DIDs. Esta implementación estandariza numerosas interacciones importantes, incluida la comunicación esencial entre mediadores y sus pares.

La integración con mediadores tiene especial importancia en la arquitectura de Identus. Los mediadores pueden recibir, almacenar y reenviar mensajes DIDComm, lo que permite la comunicación asíncrona incluso cuando los destinatarios están desconectados temporalmente. Esta capacidad resulta esencial para soluciones prácticas de identidad que deben funcionar de forma fiable con distintas condiciones de conectividad.

En el momento de escribir este capítulo, Identus admite DIDComm mediante sondeo de puntos de acceso HTTP, que es con diferencia la implementación más probada y estable. También lo admite mediante WebSockets que una opción de función habilita en Cloud Agent.

Para obtener detalles sobre el envío y la recepción de mensajes en Identus mediante mediadores, consulta el capítulo .

Conexiones

Descripción general

Ahora que comprendemos mejor las billeteras y los DIDs, podemos iniciar nuestra primera interacción. En este capítulo exploraremos qué significa una conexión en SSI, estudiaremos los DIDs de pares, explicaremos cómo funcionan y por qué son necesarios para conexiones seguras, analizaremos las invitaciones fuera de banda y mostraremos código de ejemplo para conectar un cliente Edge con un agente.

Antes de continuar, recomendamos leer al menos el tutorial básico de conexiones de la documentación oficial de Identus.

Conexiones en la identidad autosoberana

Las conexiones son fundamentales para establecer *interacciones de confianza* entre pares. Permiten una comunicación segura y verificable para que las entidades intercambien credenciales y pruebas de forma descentralizada. Esta relación usa un estándar específico de identificadores descentralizados (**DID de pares**) y un protocolo (**DIDComm**) que garantiza la autenticidad, la integridad y la privacidad de las interacciones entre las partes conectadas.

Una conexión SSI tiene tres roles:

- **Invitador**: la entidad que inicia la conexión mediante una invitación.
- **Invitado**: la entidad que recibe la invitación y responde con una solicitud de conexión.
- **Mediador**: un intermediario que facilita la entrega de mensajes entre entidades, especialmente cuando una o ambas partes no están siempre conectadas.

> **Nota**
>
> El capítulo sobre mediadores los explica en detalle. Por ahora, debes comprender que prestan un servicio de retransmisión de mensajes entre pares. Almacenan mensajes y los entregan cuando un par vuelve a conectarse, se conecta al mediador y recupera sus mensajes.

DIDs de pares

Son un tipo especial de identificador descentralizado con propiedades que los hacen adecuados para establecer comunicaciones privadas y seguras entre pares.

Los documentos DID, como los de DIDs PRISM, deben estar disponibles públicamente y permitir que cualquier parte los resuelva. Por ello, almacenarlos en un VDR como la cadena de bloques Cardano permite cumplir este requisito de forma fiable.

Sin embargo, cuando Alice y Bob quieren interactuar, solo dos partes necesitan los detalles de esa conexión: Alice y Bob. Solo ellos necesitan resolver sus DIDs. Por tanto, los DIDs de pares describen un par de claves para cifrar y firmar datos que Alice y Bob envían y reciben mediante sus mediadores preferidos. Por ejemplo, cuando Alice acepta una invitación de Bob e inician el protocolo de conexión, Alice genera un DID de pares que le permite cifrar y firmar datos dirigidos mediante el mediador de Bob (mediador Y) que solo Bob puede descifrar. A su vez, Bob genera un DID de pares que le permite cifrar y firmar datos dirigidos mediante el mediador preferido de Alice (mediador X) que solo Alice puede descifrar.

Las principales ventajas de los DIDs de pares son:

1. Son descentralizados por naturaleza.
2. No tienen coste de transacción en la cadena de bloques.
3. Son privados: solo las partes implicadas los conocen.
4. Permiten su reutilización sin depender de internet y sin degradar la confianza. Siguen los principios de prioridad local y prioridad sin conexión.

Resolvamos un DID de pares. Llamamos resolución al desempaquetado y análisis de un DID para leer su contenido y usarlo en las interacciones necesarias. En este ejemplo resolveremos el DID de pares del mediador Identus local que iniciaste en el capítulo sobre mediadores, que escucha en `http://localhost:8080`.

```
curl http://localhost:8080/did

did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm7y.Vz6Mkhh1e5CEY
↪ Yq6JBUcTZ6Cp2ranCWRrv7Yax3Le4N59R6dd.SeyJ0IjoiZG0iLCJzIjp7InVyaSI6Imh0
↪ dHA6Ly9sb2NhbGhvc3Q6ODA4MCIsImEiOlsiZGlkY29tbS92MiJdfX0.SeyJ0IjoiZG0iL
↪ CJzIjp7InVyaSI6IndzOi8vbG9jYWxob3N0OjgwODAvd3MiLCJhIjpbImRpZGNvbW0vdjI
↪ iXX19
```

El mediador devuelve un DID de pares cuando enviamos una solicitud `GET` al punto de acceso `/did`. Un DID `did:peer:2` contiene toda la información necesaria: las claves y los puntos de acceso de servicios están codificados en la propia cadena DID. Por tanto, puede resolverse localmente sin contactar con una red o registro. Aquí usamos la biblioteca Python `did-peer-2`, una de las implementaciones de la especificación del método Peer DID.

```
pip install did-peer-2
python3 -c "import json, sys; from did_peer_2 import resolve;
 ↪  print(json.dumps(resolve(sys.argv[1]), indent=2))" \
  "$(curl -s http://localhost:8080/did)"
```


```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/multikey/v1"
  ],
  "id": "did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm7y.Vz6M
   ↪  khh1e5CEYYq6JBUcTZ6Cp2ranCWRrv7Yax3Le4N59R6dd.SeyJ0IjoiZG0iLCJzIjp7I
   ↪  nVyaSI6Imh0dHA6Ly9sb2NhbGhvc3Q6ODA4MCIsImEiOlsiZGlkY29tbS92MiJdfX0.S
   ↪  eyJ0IjoiZG0iLCJzIjp7InVyaSI6IndzOi8vbG9jYWxob3N0OjgwODAvd3MiLCJhIjpb
   ↪  ImRpZGNvbW0vdjIiXX19",
  "verificationMethod": [
    {
      "type": "Multikey",
      "id": "#key-1",
      "controller": "did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFv
       ↪  MjRAm7y.Vz6Mkhh1e5CEYYq6JBUcTZ6Cp2ranCWRrv7Yax3Le4N59R6dd.SeyJ0I
       ↪  joiZG0iLCJzIjp7InVyaSI6Imh0dHA6Ly9sb2NhbGhvc3Q6ODA4MCIsImEiOlsiZ
       ↪  GlkY29tbS92MiJdfX0.SeyJ0IjoiZG0iLCJzIjp7InVyaSI6IndzOi8vbG9jYWxo
       ↪  b3N0OjgwODAvd3MiLCJhIjpbImRpZGNvbW0vdjIiXX19",
      "publicKeyMultibase":
       ↪  "z6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm7y"
    },
    {
      "type": "Multikey",
      "id": "#key-2",
      "controller": "did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFv
       ↪  MjRAm7y.Vz6Mkhh1e5CEYYq6JBUcTZ6Cp2ranCWRrv7Yax3Le4N59R6dd.SeyJ0I
       ↪  joiZG0iLCJzIjp7InVyaSI6Imh0dHA6Ly9sb2NhbGhvc3Q6ODA4MCIsImEiOlsiZ
       ↪  GlkY29tbS92MiJdfX0.SeyJ0IjoiZG0iLCJzIjp7InVyaSI6IndzOi8vbG9jYWxo
       ↪  b3N0OjgwODAvd3MiLCJhIjpbImRpZGNvbW0vdjIiXX19",
      "publicKeyMultibase":
       ↪  "z6Mkhh1e5CEYYq6JBUcTZ6Cp2ranCWRrv7Yax3Le4N59R6dd"
    }
  ],
  "keyAgreement": [
    "#key-1"
  ],
  "authentication": [
    "#key-2"
  ],
  "service": [
```

```
    {
      "type": "DIDCommMessaging",
      "serviceEndpoint": {
        "uri": "http://localhost:8080",
        "accept": [
          "didcomm/v2"
        ]
      },
      "id": "#service"
    },
    {
      "type": "DIDCommMessaging",
      "serviceEndpoint": {
        "uri": "ws://localhost:8080/ws",
        "accept": [
          "didcomm/v2"
        ]
      },
      "id": "#service-1"
    }
  ],
  "alsoKnownAs": [
    "did:peer:3zQmdayqvm5GBi2QrjAz3y99vCEp7xMvcz8eDLi1KefSq38f"
  ]
}
```


Así se ve un DID de pares resuelto en su documento DID. La entrada `alsoKnownAs` contiene el identificador equivalente de forma corta `did:peer:3`, derivado del mismo DID.

En términos simples, un DID de pares es una carga útil JSON que contiene un conjunto de claves y puntos de acceso de servicios opcionales. Como este es el DID de pares de un mediador, contiene puntos de acceso para `DIDCommMessaging`. En este caso contiene dos: uno mediante `http` y otro mediante `websockets`. Estos valores proceden de la configuración `SERVICE_ENDPOINTS` del mediador.

Para profundizar en los DIDs de pares, consulta la especificación completa del método Peer DID. En el momento de escribir este texto, el ecosistema Hyperledger Identus solo admite DIDs de pares del método 2.

Invitaciones fuera de banda

Las invitaciones fuera de banda (OOB) son el punto de entrada de algunos protocolos. Normalmente se codifican en una carga útil JSON o una URL y se distribuyen fuera de banda, habitualmente mediante códigos QR, aunque pueden usar cualquier medio, como Bluetooth o NFC. Reúnen toda la información necesaria para que un par empiece a interactuar

con otro. Puedes considerarlas una forma de anunciar las «coordenadas» del invitador a cualquiera que quiera establecer una interacción con él.

En el mismo ejemplo anterior, el mediador local también proporciona una invitación OOB como URL mediante su punto de acceso `/invitationOOB`.

```
curl http://localhost:8080/invitationOOB

?_oob=eyJpZCI6IjA4NzQ3YWJiLTE4MGYtNGQxOC05ZTVjLWIzOGRiM2JiMTJhMiIsInR5cGUi
 ↪  OiJodHRwczovL2RpZGNvbW0ub3JnL291dC1vZi1iYW5kLzIuMC9pbnZpdGF0aW9uIiwiZn
 ↪  JvbSI6ImRpZDpwZWVyOjIuRXo2TFNnaHdTRTQzN3duREUxcHQzWDZoVkRVUXpTanNIemlu
 ↪  cFgzWEZ2TWpSQW03eS5WejZNa2hoMWU1Q0VZWXE2SkJVY1RaNkNwMnJhbkNXUnJ2N1lheD
 ↪  NMZTRONTlSNmRkLlNleUowSWpvaVpHMGlMQ0p6SWpwN0luVnlhU0k2SW1oMGRIQTZMeTlz
 ↪  YjJOaGJHaHZjM1E2T0RBNE1DSXNJbUVpT2xzaVpHbGtZMjl0YlM5Mk1pSmRmWDAuU2V5Sj
 ↪  BJam9pWkcwaUxDSnpJanA3SW5WeWFTSTZJbmR6T2k4dmJHOWpZV3hvYjNOME9qZ3dPREF2
 ↪  ZDNNaUxDSmhJanBiSW1ScFpHTnZiVzB2ZGpJaVhYMTkiLCJib2R5Ijp7ImdvYWxfY29kZS
 ↪  I6InJlcXVlc3QtbWVkaWF0ZSIsImdvYWwiOiJSZXF1ZXN0TWVkaWF0ZSIsImFjY2VwdCI6
 ↪  WyJkaWRjb21tL3YyIl19fQ
```

Al añadirla a la URL del mediador, se obtiene un enlace completo de invitación como `http://localhost:8080?_oob=eyJpZC...`.

Si decodificamos desde `base64` el valor de la variable de consulta `_oob`, obtenemos la carga útil `json`:

```
{
  "id": "08747abb-180f-4d18-9e5c-b38db3bb12a2",
  "type": "https://didcomm.org/out-of-band/2.0/invitation",
  "from": "did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm7y.Vz
   ↪  6Mkhh1e5CEYYq6JBUcTZ6Cp2ranCWRrv7Yax3Le4N59R6dd.SeyJ0IjoiZG0iLCJzIjp
   ↪  7InVyaSI6Imh0dHA6Ly9sb2NhbGhvc3Q6ODA4MCIsImEiOlsiZGlkY29tbS92MiJdfX0
   ↪  .SeyJ0IjoiZG0iLCJzIjp7InVyaSI6IndzOi8vbG9jYWxob3N0OjgwODAvd3MiLCJhIj
   ↪  pbImRpZGNvbW0vdjIiXX19",
  "body": {
    "goal_code": "request-mediate",
    "goal": "RequestMediate",
    "accept": [
      "didcomm/v2"
    ]
  }
}
```

Una invitación fuera de banda es una forma de empaquetar un DID de pares e indicar que puede usarse para una interacción concreta. En este caso, para un objetivo `RequestMediate` mediante `DIDComm`.

Como recordatorio, un DID de pares empaqueta un conjunto de claves y puntos de acceso de servicios opcionales. Por tanto, *como esta es una invitación OOB de un mediador*, contiene todo lo que necesitas —un DID de pares y un conjunto de puntos de acceso de servicios— para establecer este servicio como tu mediador.

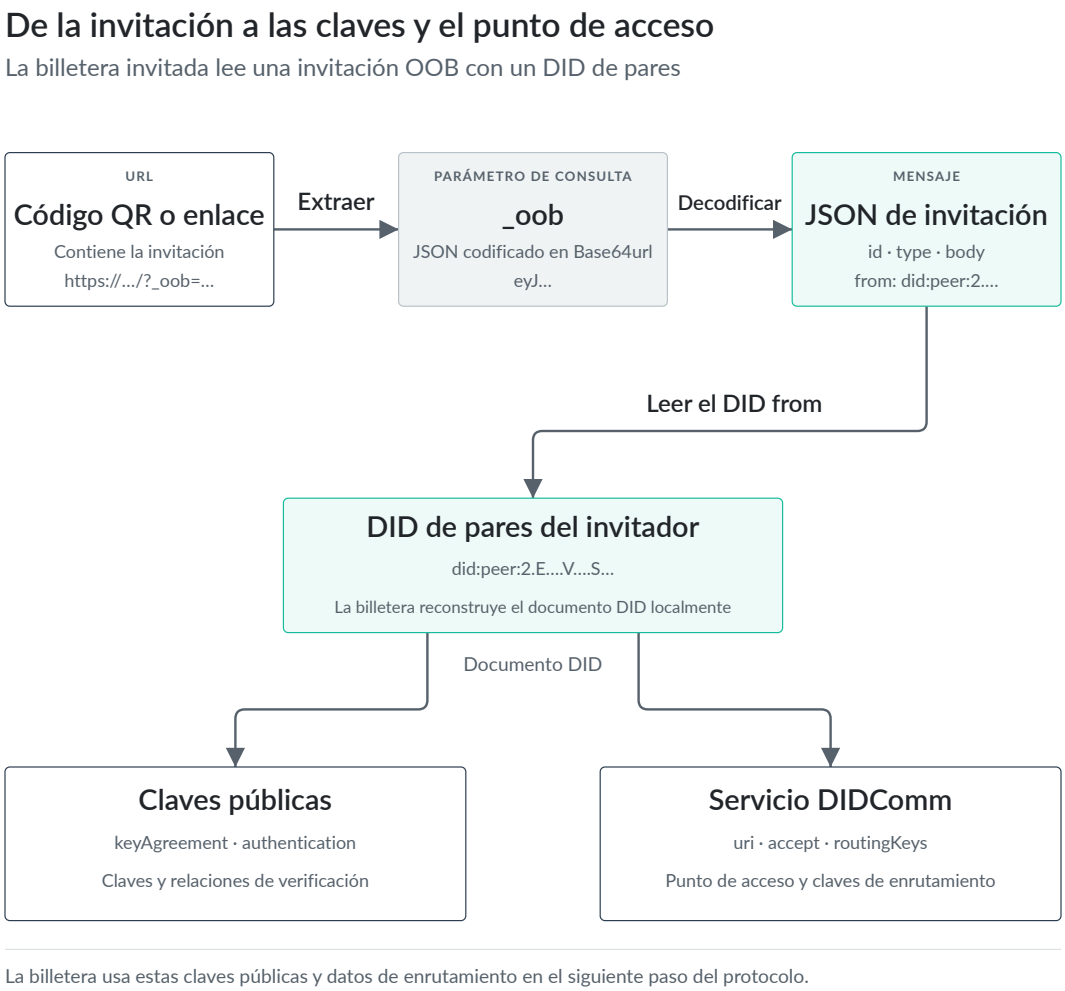


Figura 1: Este ejemplo usa did:peer:2 con información de servicio incorporada. La billetera decodifica la invitación, resuelve su DID from y lee las claves públicas y el servicio DIDComm. Los puntos suspensivos abrevian los valores codificados.

Conectar dos pares

Emitamos ahora otro tipo de invitación fuera de banda, desde Cloud Agent, para establecer una conexión.

Cloud Agent puede generar invitaciones fuera de banda. Otro par, como otro Cloud Agent o un cliente Edge, puede analizar la invitación y usarla para establecer una conexión. El

resultado debe ser un DID de pares en cada lado que les permita enviarse mensajes mediante `DIDComm`.

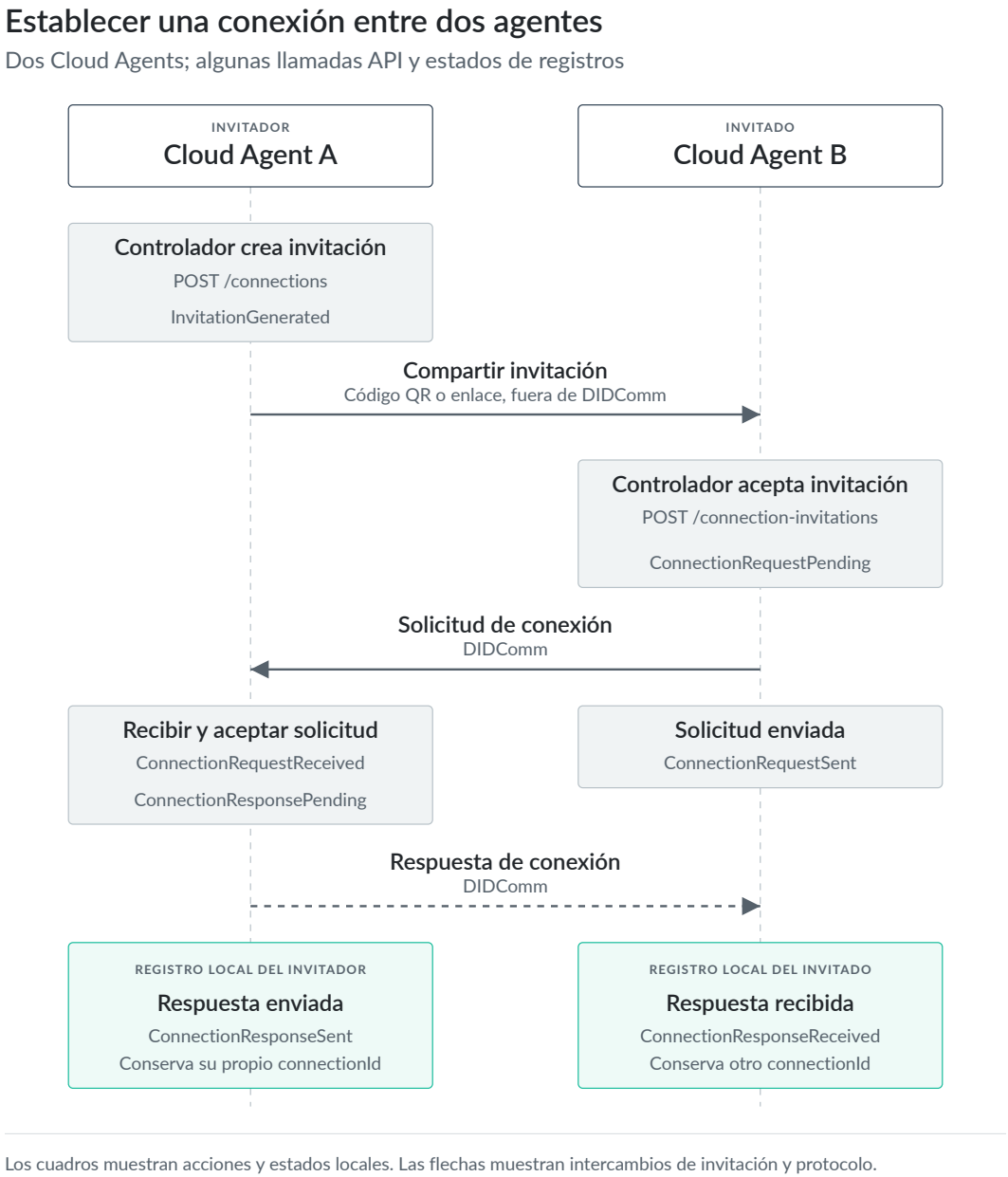


Figura 2: Cada controlador usa el connectionId de su propio Cloud Agent para leer o continuar la conexión. Los agentes actualizan sus registros cuando envían y reciben mensajes. El diagrama muestra algunos estados del flujo correcto.

Nuestro primer paso consiste en generar la invitación:

```
curl --location 'http://127.0.0.1:8080/cloud-agent/connections' \
--header 'Content-Type: application/json' \
--header 'Accept: application/json' \
--data '{"label": "test"}'
```

> **Nota**
>
> El único parámetro que podemos cambiar al generar una invitación es `label`. Es opcional y contiene una cadena simple que puedes usar para identificar la conexión después. Puedes usar un `uuid` que generas y administras en tus sistemas, un alias o el motivo de la conexión. La elección depende de la interacción y el caso de uso; aquí usaremos «test».

Cloud Agent responderá con una carga útil similar a esta:


```
{
    "connectionId": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "thid": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "label": "test",
    "role": "Inviter",
    "state": "InvitationGenerated",
    "invitation": {
        "id": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
        "type": "https://didcomm.org/out-of-band/2.0/invitation",
        "from": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7
        ↪  cpj.Vz6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV.SeyJ0Ijo
        ↪  iZG0iLCJzIjp7InVyaSI6Imh0dHA6Ly9ob3N0LmRvY2tlci5pbnRlcm5hbDo4M
        ↪  DgwL2RpZGNvbW0iLCJyIjpbXSwiYSI6WyJkaWRjb21tL3YyIl19fQ",
        "invitationUrl": "https://my.domain.com/path?_oob=eyJpZCI6ImZiMzZl
        ↪  ZGRkLWQ1MWUtNDJjZi1hNmZlLWU3NmQyZTYzOGI3MCIsInR5cGUiOiJodHRwcz
        ↪  ovL2RpZGNvbW0ub3JnL291dC1vZi1iYW5kLzIuMC9pbnZpdGF0aW9uIiwiZnJv
        ↪  bSI6ImRpZDpwZWVyOjIuRXo2TFNnWTZZNjdtSjc1WUNaZlpZeFlFUFFKWnMzdm
        ↪  FFZzJDYzkxdnBwb1RBN2Nwai5WejZNa3BYN0g3U05BNm9vRzVzbm4yTXpneW9S
        ↪  YWRFWnRzak5TTDF4N0hpaUxrcXlWLlNleUowSWpvaVpHMGlMQ0p6SWpwN0luVn
        ↪  lhU0k2SW1oMGRIQTZMeTlvYjNOMExtUnZZMnRsY2k1cGJuUmxjbTVoYkRvNE1E
        ↪  Z3dMMlJwWkdOdmJXMGlMQ0p5SWpwYlhTd2lZU0k2V3lKa2FXUmpiMjF0TDNZeU
        ↪  lsMTlmUSIsImJvZHkiOnsiYWNjZXB0IjpbXX19"
    },
    "createdAt": "2025-01-04T12:37:37.059649293Z",
    "metaRetries": 5,
    "self": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "kind": "Connection"
}
```


Después de crear la invitación de conexión, puedes obtener sus detalles mediante una solicitud `GET` con el `connectionId`:

```
curl --location 'http://127.0.0.1:8080/cloud-agent/connections/fb36eddd-d5
 ↪  1e-42cf-a6fe-e76d2e638b70' \
--header 'Accept: application/json' \
```

Debes recibir la misma carga útil que al crearla, salvo que haya cambiado algún dato, como `state`:


```
{
    "connectionId": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "thid": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "label": "test",
    "role": "Inviter",
    "state": "InvitationGenerated",
    "invitation": {
        "id": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
        "type": "https://didcomm.org/out-of-band/2.0/invitation",
        "from": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7
         ↪  cpj.Vz6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV.SeyJ0Ijo
         ↪  iZG0iLCJzIjp7InVyaSI6Imh0dHA6Ly9ob3N0LmRvY2tlci5pbnRlcm5hbDo4M
         ↪  DgwL2RpZGNvbW0iLCJyIjpbXSwiYSI6WyJkaWRjb21tL3YyIl19fQ",
        "invitationUrl": "https://my.domain.com/path?_oob=eyJpZCI6ImZiMzZl
         ↪  ZGRkLWQ1MWUtNDJjZi1hNmZlLWU3NmQyZTYzOGI3MCIsInR5cGUiOiJodHRwcz
         ↪  ovL2RpZGNvbW0ub3JnL291dC1vZi1iYW5kLzIuMC9pbnZpdGF0aW9uIiwiZnJv
         ↪  bSI6ImRpZDpwZWVyOjIuRXo2TFNnWTZZNjdtSjc1WUNaZlpZeFlFUFFKWnMzdm
         ↪  FFZzJDYzkxdnBwb1RBN2Nwai5WejZNa3BYN0g3U05BNm9vRzVzbm4yTXpneW9S
         ↪  YWRFWnRzak5TTDF4N0hpaUxrcXlWLlNleUowSWpvaVpHMGlMQ0p6SWpwN0luVn
         ↪  lhU0k2SW1oMGRIQTZMeTlvYjNOMExtUnZZMnRsY2k1cGJuUmxjbTVoYkRvNE1E
         ↪  Z3dMMlJwWkdOdmJXMGlMQ0p5SWpwYlhTd2lZU0k2V3lKa2FXUmpiMjF0TDNZeU
         ↪  lsMTlmUSIsImJvZHkiOnsiYWNjZXB0IjpbXX19"
    },
    "createdAt": "2025-01-04T12:37:37.059649Z",
    "metaRetries": 5,
    "self": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "kind": "Connection"
}
```


Veamos qué representa esta carga útil.

La carga útil de la invitación contiene metadatos importantes:

- **connectionId**: el identificador único del recurso de conexión, que permite obtener los detalles de la conexión.
- **thid**: el identificador único del *hilo* al que pertenece este registro de conexión. El valor será idéntico en ambos lados de la conexión, invitador e invitado.
- **label**: un alias legible de la conexión.
- **role**: el rol de Cloud Agent en esta conexión, `Inviter` o `Invitee`.

- **state**: el estado actual de la conexión. Depende del rol de Cloud Agent. Para `Inviter`, los estados pueden ser `InvitationGenerated`, `ConnectionRequestReceived`, `ConnectionResponsePending` y `ConnectionResponseSent`. Cloud Agent también puede analizar la invitación de otra parte. En ese caso, genera una conexión con el rol `Invitee`, cuyos estados posibles son `InvitationReceived`, `ConnectionRequestPending`, `ConnectionRequestSent` y `ConnectionResponseReceived`.
- **invitation**: los detalles de la invitación `DIDComm`.
- **createdAt**: fecha y hora de creación o recepción de esta conexión.
- **metaRetries**: el número máximo de intentos restantes de procesamiento en segundo plano para este registro.
- **self**: la referencia al recurso de conexión.
- **kind**: el tipo de objeto devuelto. En este caso, `Connection`.

Analicemos los detalles de `invitation`.

- **id**: el identificador único de la invitación. Debe usarse como identificador del hilo padre (`pthid`) del siguiente mensaje de solicitud de conexión.
- **type**: el URI de tipo de mensaje DIDComm (MTURI) al que corresponde el mensaje de invitación.
- **from**: el DID que representa al remitente y que los destinatarios usarán en interacciones posteriores.
- **invitationUrl**: el mensaje de invitación codificado como URL.

Resolvamos el DID `from`:

```
curl https://dev.uniresolver.io/1.0/identifiers/did:peer:2.Ez6LSgY6Y67mJ75
 ↪  YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj.Vz6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsj
 ↪  NSL1x7HiiLkqyV.SeyJ0IjoiZG0iLCJzIjp7InVyaSI6Imh0dHA6Ly9ob3N0LmRvY2tlci
 ↪  5pbnRlcm5hbDo4MDgwL2RpZGNvbW0iLCJyIjpbXSwiYSI6WyJkaWRjb21tL3YyIl19fQ
```


```
{
  "@context": "https://w3id.org/did-resolution/v1",
  "didDocument": {
    "@context": [
      "https://www.w3.org/ns/did/v1",
      "https://w3id.org/security/multikey/v1",
      {
        "@base": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA
         ↪  7cpj.Vz6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV.SeyJ0Ij
         ↪  oiZG0iLCJzIjp7InVyaSI6Imh0dHA6Ly9ob3N0LmRvY2tlci5pbnRlcm5hbDo4
         ↪  MDgwL2RpZGNvbW0iLCJyIjpbXSwiYSI6WyJkaWRjb21tL3YyIl19fQ"
      }
    ],
    "id": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj.Vz
     ↪  6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV.SeyJ0IjoiZG0iLCJzI
     ↪  jp7InVyaSI6Imh0dHA6Ly9ob3N0LmRvY2tlci5pbnRlcm5hbDo4MDgwL2RpZGNvbW0
     ↪  iLCJyIjpbXSwiYSI6WyJkaWRjb21tL3YyIl19fQ",
```

```
  "verificationMethod": [
    {
      "id": "#key-2",
      "type": "Multikey",
      "controller": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91v
      ↪  ppoTA7cpj.Vz6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV.Se
      ↪  yJ0IjoiZG0iLCJzIjp7InVyaSI6Imh0dHA6Ly9ob3N0LmRvY2tlci5pbnRlcm5
      ↪  hbDo4MDgwL2RpZGNvbW0iLCJyIjpbXSwiYSI6WyJkaWRjb21tL3YyIl19fQ",
      "publicKeyMultibase":
      ↪  "z6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV"
    },
    {
      "id": "#key-1",
      "type": "Multikey",
      "controller": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91v
      ↪  ppoTA7cpj.Vz6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV.Se
      ↪  yJ0IjoiZG0iLCJzIjp7InVyaSI6Imh0dHA6Ly9ob3N0LmRvY2tlci5pbnRlcm5
      ↪  hbDo4MDgwL2RpZGNvbW0iLCJyIjpbXSwiYSI6WyJkaWRjb21tL3YyIl19fQ",
      "publicKeyMultibase":
      ↪  "z6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj"
    }
  ],
  "keyAgreement": [
    "#key-1"
  ],
  "authentication": [
    "#key-2"
  ],
  "assertionMethod": [
    "#key-2"
  ],
  "service": [
    {
      "serviceEndpoint": {
        "uri": "http://host.docker.internal:8080/didcomm",
        "routingKeys": [],
        "accept": [
          "didcomm/v2"
        ]
      },
      "type": "DIDCommMessaging",
      "id": "#service"
    }
  ]
},
"didResolutionMetadata": {
```

```
    "contentType": "application/did+ld+json",
    "pattern": "^(did:peer:.+)$",
    "driverUrl":
     ↪  "http://uni-resolver-driver-did-uport:8081/1.0/identifiers/",
    "duration": 3,
    "driverDuration": 3,
    "did": {
      "didString": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppo
       ↪  TA7cpj.Vz6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV.SeyJ0Ij
       ↪  oiZG0iLCJzIjp7InVyaSI6Imh0dHA6Ly9ob3N0LmRvY2tlci5pbnRlcm5hbDo4MD
       ↪  gwL2RpZGNvbW0iLCJyIjpbXSwiYSI6WyJkaWRjb21tL3YyIl19fQ",
      "methodSpecificId": "2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA
       ↪  7cpj.Vz6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV.SeyJ0Ijoi
       ↪  ZG0iLCJzIjp7InVyaSI6Imh0dHA6Ly9ob3N0LmRvY2tlci5pbnRlcm5hbDo4MDgw
       ↪  L2RpZGNvbW0iLCJyIjpbXSwiYSI6WyJkaWRjb21tL3YyIl19fQ",
      "method": "peer"
    }
  },
  "didDocumentMetadata": {}
}
```

Esto ya resulta familiar: tenemos el conjunto habitual de claves y se parece al DID del mediador. En este caso vemos un `serviceEndpoint` que acepta `didcomm/v2` y sirve para `DIDCommMessaging`. El DID de Cloud Agent anuncia cómo recibirá mensajes `DIDComm`. Es decir: «Aquí tienes una invitación para conectarte conmigo. Incluye un DID con mis claves públicas y un punto de acceso donde recibo mensajes `DIDComm`».

La última parte que debemos analizar es `invitationUrl`, que al principio parece extraña:

```
"https://my.domain.com/path?_oob=eyJpZCI6ImZiMzZlZGRkLWQ1MWUtNDJjZi1hNmZlL
 ↪  WU3NmQyZTYzOGI3MCIsInR5cGUiOiJodHRwczovL2RpZGNvbW0ub3JnL291dC1vZi1iYW5
 ↪  kLzIuMC9pbnZpdGF0aW9uIiwiZnJvbSI6ImRpZDpwZWVyOjIuRXo2TFNnWTZZNjdtSjc1W
 ↪  UNaZlpZeFlFUFFKWnMzdmFFZzJDYzkxdnBwb1RBN2Nwai5WejZNa3BYN0g3U05BNm9vRzV
 ↪  zbm4yTXpneW9SYWRFWnRzak5TTDF4N0hpaUxrcXlWLlNleUowSWpvaVpHMGlMQ0p6SWpwN
 ↪  0luVnlhU0k2SW1oMGRIQTZMeTlvYjNOMExtUnZZMnRsY2k1cGJuUmxjbTVoYkRvNE1EZ3d
 ↪  MMlJwWkdOdmJXMGlMQ0p5SWpwYlhTd2lZU0k2V3lKa2FXUmpiMjF0TDNZeUlsMTlmUSIsI
 ↪  mJvZHkiOnsiYWNjZXB0IjpbXX19"
```

El primer dato que parece incorrecto es el dominio: ¿de dónde viene `my.domain.com`? Cloud Agent lo incluye como valor fijo y no permite personalizarlo. Pero lo importante es la carga útil del campo `_oob`. La URL no importa porque lo que necesitamos está dentro del campo codificado en `base64`. Decodifiquémoslo.

```
echo 'eyJpZCI6ImZiMzZlZGRkLWQ1MWUtNDJjZi1hNmZlLWU3NmQyZTYzOGI3MCIsInR5cGUi
 ↪  OiJodHRwczovL2RpZGNvbW0ub3JnL291dC1vZi1iYW5kLzIuMC9pbnZpdGF0aW9uIiwiZn
 ↪  JvbSI6ImRpZDpwZWVyOjIuRXo2TFNnWTZZNjdtSjc1WUNaZlpZeFlFUFFKWnMzdmFFZzJD
 ↪  YzkxdnBwb1RBN2Nwai5WejZNa3BYN0g3U05BNm9vRzVzbm4yTXpneW9SYWRFWnRzak5TTD
 ↪  F4N0hpaUxrcXlWLlNleUowSWpvaVpHMGlMQ0p6SWpwN0luVnlhU0k2SW1oMGRIQTZMeTlv
 ↪  YjNOMExtUnZZMnRsY2k1cGJuUmxjbTVoYkRvNE1EZ3dMMlJwWkdOdmJXMGlMQ0p5SWpwYl
 ↪  hTd2lZU0k2V3lKa2FXUmpiMjF0TDNZeUlsMTlmUSIsImJvZHkiOnsiYWNjZXB0IjpbXX19'
 ↪  | base64 -d
```

```
{
  "id": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
  "type": "https://didcomm.org/out-of-band/2.0/invitation",
  "from": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj.Vz
   ↪ 6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV.SeyJ0IjoiZG0iLCJzIjp
   ↪ 7InVyaSI6Imh0dHA6Ly9ob3N0LmRvY2tlci5pbnRlcm5hbDo4MDgwL2RpZGNvbW0iLCJ
   ↪ yIjpbXSwiYSI6WyJkaWRjb21tL3YyIl19fQ",
  "body": {
    "accept": []
  }
}
```


La carga útil codificada de `_oob` contiene la información mínima para indicar que es una invitación `DIDComm` de un DID de pares.

Presentación y emisión de credenciales sin conexión previa

En identidad digital, establecer confianza e intercambiar credenciales verificables normalmente requiere una serie de interacciones entre un emisor o verificador y un titular. Aunque muchos protocolos necesitan una conexión o relación existente, un flujo sin conexión previa ofrece una vía más directa para ciertos casos. Este método resulta útil cuando aún no existe una relación entre el emisor o verificador y el titular, o cuando la interacción concreta no la necesita.

La diferencia fundamental de la emisión y presentación sin conexión previa está en el inicio. Los flujos tradicionales pueden requerir una conexión formal, como intercambiar DIDs y establecer un canal DIDComm, *antes* de ofrecer una credencial. El flujo sin conexión previa usa una invitación fuera de banda (OOB) para iniciar directamente el proceso.

Cuando un emisor o verificador quiere ofrecer una credencial o solicitar una prueba de esta forma, genera una invitación OOB. Esta invitación es un mensaje autónomo o una referencia, como una URL o un código QR, que el titular puede recibir mediante un código QR, correo electrónico, un sitio web u otro medio. Al analizar la invitación OOB, el agente del titular puede comprender de inmediato la intención de ofrecer una credencial o solicitar su prueba.

La invitación OOB contiene suficiente información, o referencias a ella, para que el agente del titular solicite la oferta de credencial al emisor o presente una prueba al verificador. Esto evita un intercambio preliminar separado para establecer una conexión. Cuando el emisor o verificador recibe una solicitud derivada de esta invitación OOB, puede enviar la oferta de credencial o la solicitud de prueba.

Desde la aceptación de la oferta por el titular, los pasos posteriores se parecen a los de los protocolos estándar de emisión y solicitud de prueba. La eficiencia y la diferencia principales

del enfoque sin conexión previa se concentran al inicio de la interacción: permite una vía más rápida y directa cuando no se necesita una conexión persistente. Por ello sirve para atestaciones anónimas, verificaciones únicas u ofertas públicas de credenciales donde establecer y administrar conexiones para cada titular resulta poco práctico.

Credenciales verificables

Descripción general

Considera una credencial verificable (VC) como una declaración firmada que puede acompañar a la persona u organización que describe. Una aerolínea puede emitir una tarjeta de embarque a un viajero. Una universidad puede emitir una credencial de título a un graduado. Un organismo público puede emitir un permiso a una empresa. En cada caso, el titular conserva la credencial y después la presenta a un verificador que necesita una prueba de una afirmación.

El modelo de datos de credenciales verificables 2.0 de W3C define los roles principales: emisor, titular, verificador y sujeto. El emisor crea la credencial. El titular la almacena y presenta. El verificador comprueba la presentación. El sujeto es la persona, organización, cosa o cuenta que describen las afirmaciones. A veces el titular y el sujeto coinciden. En otros casos, un progenitor puede conservar una credencial de su hijo o un administrador de una empresa puede conservar credenciales sobre la empresa.

Una VC contiene cuatro tipos de información:

- Afirmaciones, como nombre, número de billete, nivel de afiliación, rango de edad o categoría de licencia.
- Información del emisor, para que el verificador determine quién hizo la declaración.
- Material de prueba, para que el verificador compruebe la integridad y la autoría.
- Metadatos, como fecha de emisión, fecha de caducidad, esquema, tipo de credencial y estado.

La prueba permite que el verificador detecte alteraciones. No decide si debe aceptar la credencial para un propósito de negocio determinado. Esa decisión depende de la política. Una puerta de embarque puede aceptar solo credenciales que la aerolínea emitió para el vuelo actual. Un verificador puede rechazar una credencial válida si no confía en el emisor, la credencial ha caducado, no acepta el esquema o la credencial está revocada.

El README de Identus Cloud Agent describe Cloud Agent como un agente basado en W3C y Aries que puede emitir, conservar y verificar credenciales mediante DIDComm v2. Identus admite varios formatos de credenciales mediante una API REST y flujos de agente basados en webhooks.

Formatos

Las VC comparten un propósito, pero no todas usan la misma envoltura, formato de prueba, modelo de privacidad o procedimiento de verificación. Identus Cloud Agent documenta tres

formatos de emisión mediante sus API: JWT-VC, SD-JWT-VC y AnonCreds. En las cargas útiles de Cloud Agent aparecen como valores de `credentialFormat`, por ejemplo `JWT`, `SDJWT` y `AnonCreds`.

Credenciales verificables JWT

JWT-VC empaqueta las afirmaciones de la credencial en un JSON Web Token. Los equipos suelen elegirlo cuando sus verificadores ya usan herramientas JOSE y verificación del emisor basada en DIDs. El verificador decodifica el JWT, valida la firma con la clave pública del emisor resuelta desde su DID y lee las afirmaciones.

JWT-VC tiene una limitación de privacidad. Normalmente el titular presenta la credencial completa, por lo que el verificador ve todas sus afirmaciones. Esto sirve para una puerta de embarque que necesita todos los detalles del billete. Resulta poco adecuado cuando el titular debe revelar solo uno o dos campos.

La guía de emisión DIDComm de Identus usa `jwtVcPropertiesV1` para ofertas de credenciales JWT e indica que esta estructura corresponde al perfil del modelo de datos VC 1.1 de W3C que usa el agente.

Credenciales verificables SD-JWT

SD-JWT-VC se basa en Selective Disclosure JWT. Permite que el titular revele afirmaciones seleccionadas de una credencial en lugar del conjunto completo. Un viajero que demuestra el acceso a una sala de espera puede necesitar revelar el nivel de afiliación y la fecha de caducidad, pero no la fecha de nacimiento ni el perfil completo de cliente.

El borrador SD-JWT VC de IETF define formatos de datos y reglas de procesamiento para credenciales JSON con divulgación selectiva. En términos generales, el emisor firma una estructura que establece compromisos sobre los valores de las afirmaciones. El titular presenta después solo las divulgaciones que necesita la solicitud del verificador, y este las comprueba contra los compromisos firmados.

Identus usa `sdJwtVcPropertiesV1` para este formato y requiere claves Ed25519 del emisor y del titular para la emisión SD-JWT. La documentación Present Proof de Identus señala un detalle: si la credencial SD-JWT no tiene una clave `cnf`, el titular no puede crear y firmar una presentación vinculada al desafío y dominio del verificador. Con `cnf`, el titular puede demostrar el control de la clave vinculada durante la presentación.

AnonCreds

AnonCreds es un formato de credencial con pruebas de conocimiento cero. Procede del ecosistema Hyperledger Indy y ahora lo mantiene Hyperledger AnonCreds. Los titulares lo usan para pruebas que preservan la privacidad, como demostrar predicados sobre atributos sin divulgar sus valores originales.

AnonCreds requiere una definición de credencial. Ese artefacto referencia un esquema y contiene material criptográfico del emisor que se usa para generar pruebas. En Identus, las afirmaciones AnonCreds usan una estructura plana de cadenas a cadenas, y el emisor referencia la definición de credencial mediante `credentialDefinitionId` dentro de `anoncredsVcPropertiesV1`.

OID4VCI

OpenID for Verifiable Credential Issuance (OpenID4VCI) es un protocolo de emisión, no un formato de credencial. Amplía los flujos de tipo OAuth 2.0 para que una billetera pueda obtener credenciales de un emisor mediante un punto de acceso de credenciales. La OpenID Foundation aprobó OpenID4VCI 1.0 como especificación final el 16 de septiembre de 2025. La especificación define los metadatos del emisor, las configuraciones de credenciales, el flujo de autorización y el flujo de solicitud de credenciales.

La guía OID4VCI de Identus describe Cloud Agent como servidor emisor de credenciales, integrado con un servidor de autorización que sigue el contrato esperado. Identus proporciona un flujo de ejemplo con Keycloak y emisión mediante código de autorización.

Contexto de W3C VC 2.0

Los primeros despliegues VC 1.1 ya no describen todo el panorama de estándares. W3C anunció la familia de credenciales verificables 2.0 como recomendaciones el 15 de mayo de 2025. La recomendación VC JOSE y COSE define formas de proteger las credenciales y presentaciones del modelo de datos VC 2.0 mediante JOSE, SD-JWT y COSE.

La documentación de Identus sigue indicando los formatos y cargas útiles concretos que admite Cloud Agent. Por ello, el trabajo de implementación debe seguir primero la documentación de Cloud Agent y considerar los documentos W3C VC 2.0 como la dirección de la familia más amplia de estándares.

Esquemas

Un esquema de credencial describe la estructura de las afirmaciones. Responde a preguntas como: ¿qué campos se esperan?, ¿cuáles son obligatorios?, ¿qué tipo de datos debe usar cada campo? Un esquema no decide si una afirmación es verdadera. Proporciona una estructura compartida para que emisores, titulares y verificadores lean la credencial.

Especificaciones de esquemas

Identus usa dos familias de esquemas entre sus formatos de credenciales.

Las credenciales JWT y SD-JWT usan JSON Schema. La guía oficial de esquemas de Identus requiere que el cuerpo del esquema establezca `$schema` como `https://json-schema.org/draft/2020-12/schema`, que corresponde al borrador actual descrito por el proyecto JSON Schema.

Las credenciales AnonCreds usan esquemas AnonCreds junto con definiciones de credenciales. La definición de credencial referencia un esquema mediante su ID y debe existir antes de que el emisor cree una oferta de credencial AnonCreds.

Esquema de ejemplo

Una credencial de billete de avión puede usar un JSON Schema como este:

```
{
  "$id": "https://example.com/schemas/boarding-pass-1.0",
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Boarding pass",
  "type": "object",
  "properties": {
    "passengerName": { "type": "string" },
    "flightNumber": { "type": "string" },
    "departureAirport": { "type": "string" },
    "arrivalAirport": { "type": "string" },
    "departureDateTime": { "type": "string", "format": "date-time" },
    "seatNumber": { "type": "string" },
    "boardingGroup": { "type": "string" },
    "ticketClass": { "type": "string" }
  },
  "required": [
    "passengerName",
    "flightNumber",
    "departureAirport",
    "arrivalAirport",
    "departureDateTime",
    "seatNumber"
  ],
  "additionalProperties": false
}
```

Crear esquemas en Identus

Identus Cloud Agent tiene un registro de esquemas para crear y resolver esquemas de credenciales. La guía oficial documenta dos puntos de acceso de creación:

```
POST /cloud-agent/schema-registry/schemas
POST /cloud-agent/schema-registry/schemas/did-url
```

Usa el punto de acceso HTTP cuando los verificadores deban resolver el esquema mediante una URL HTTP. Usa el punto de acceso de URL DID cuando deban resolverlo mediante una URL DID. Son modos de resolución separados; considera la vía que usará el verificador antes de emitir credenciales basadas en el esquema.

Cada registro de esquema incluye metadatos sobre su definición. `name` proporciona una etiqueta legible, `version` identifica la versión, `author` indica el DID que lo creó y `tags` ayuda a filtrar los registros de esquemas. El campo `schema` contiene la definición JSON Schema. Cloud Agent trata como única la combinación de `author`, `id` del esquema y `version`.

El campo `author` debe contener el DID PRISM de forma corta que creó el mismo Cloud Agent. No necesita publicarse antes de crear el esquema. Durante la configuración, un emisor puede crear un DID, crear un esquema y publicar el DID después, cuando el flujo de credenciales necesite resolución pública.

Versiones

Actualizar un esquema significa crear un nuevo registro con el JSON Schema modificado y una versión superior. No cambies el significado de una versión de esquema después de emitir credenciales basadas en ella. Las credenciales antiguas siguen siendo más fáciles de verificar cuando su esquema original aún se resuelve en la misma estructura.

Para cambios incompatibles, como eliminar campos obligatorios o cambiar tipos de campos, usa una nueva versión principal. Para cambios aditivos, como añadir un campo opcional, usa una versión secundaria. Así las políticas del emisor y verificador siguen siendo comprensibles cuando coexisten varias versiones de credenciales.

Emisión

La emisión transforma los datos de afirmaciones del emisor en una credencial que conserva una billetera o agente. En Identus, puede realizarse mediante una conexión DIDComm, una invitación sin conexión previa u OpenID4VCI.

Las guías de DIDs de Identus son importantes antes de iniciar la emisión. El registrador de Cloud Agent puede crear un DID PRISM sin conexión. La plantilla de documento DID admite métodos de verificación para fines como `authentication` y `assertionMethod`. Un DID PRISM de forma larga incorpora la operación de creación en el propio DID. Un DID PRISM de forma corta necesita publicarse antes de que otras partes lo resuelvan desde el registro distribuido. La guía de publicación de DIDs de Cloud Agent documenta el flujo de publicación y los cambios de estado de `CREATED` a `PUBLICATION_PENDING` y después a `PUBLISHED`.

Requisitos previos

Para la emisión DIDComm, la guía Issue Credential Protocol 3.0 de Identus describe los requisitos específicos de cada formato.

La emisión JWT requiere una conexión entre los agentes del emisor y del titular, un DID PRISM publicado del emisor con `assertionMethod`, un esquema de credencial y un DID PRISM del titular con `authentication`.

La emisión SD-JWT tiene la misma estructura, pero los DIDs PRISM del emisor y titular deben usar claves Ed25519 para los métodos de verificación correspondientes.

La emisión AnonCreds requiere una conexión y una definición de credencial AnonCreds.

Estados de la credencial

Los estados del emisor son:

```
OfferPending -> OfferSent -> RequestReceived -> CredentialPending ->
↪  CredentialGenerated -> CredentialSent
```

Los estados del titular son:

```
OfferReceived -> RequestPending -> RequestSent -> CredentialReceived
```

Estos estados sirven cuando una billetera o controlador necesita reanudar un flujo, mostrar el progreso al usuario o decidir si aún requiere una acción manual del emisor.

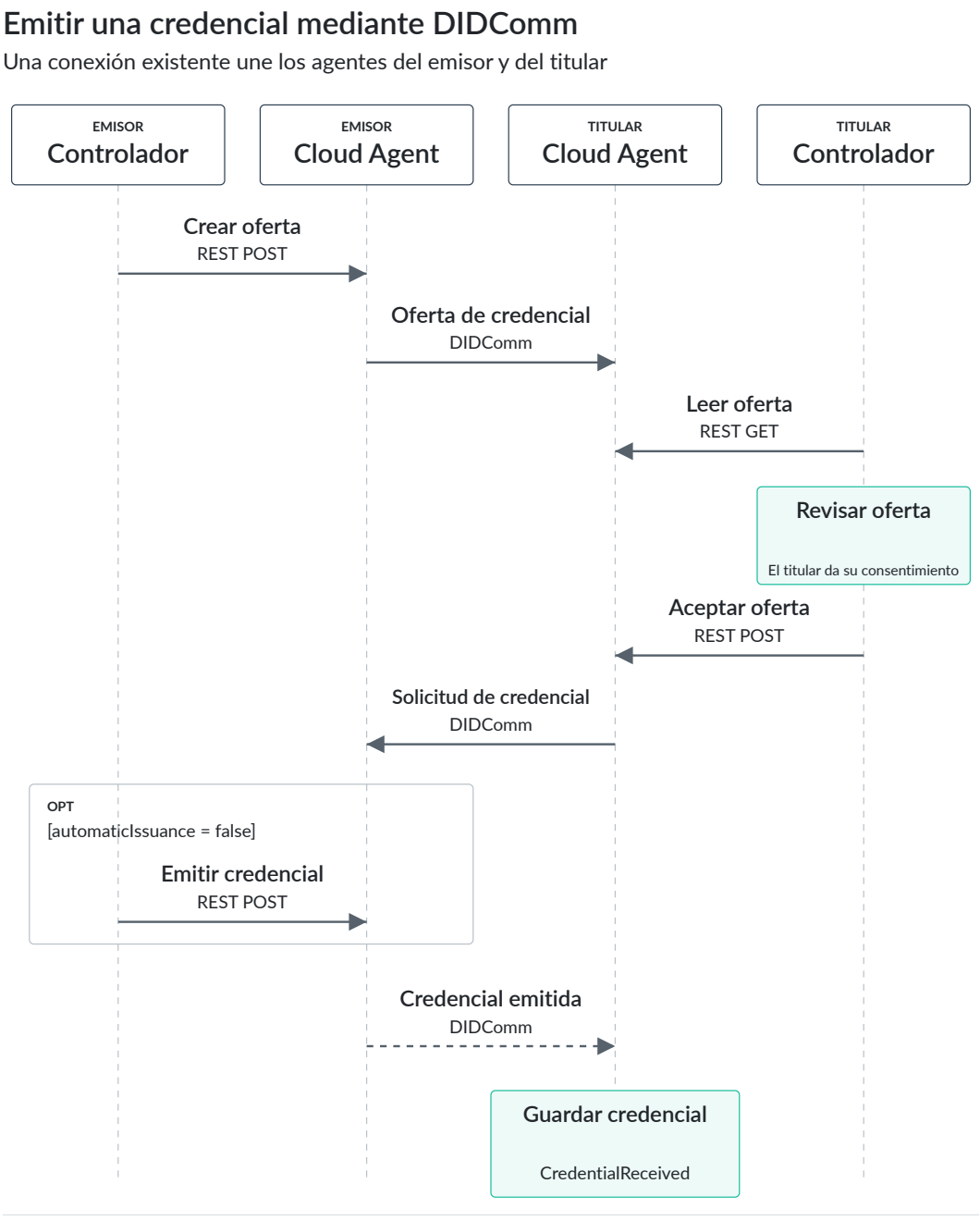


Figura 1: El controlador del titular lee la oferta y registra su consentimiento antes de llamar a accept-offer. Con la emisión automática habilitada, el agente del emisor procesa la solicitud sin la llamada opcional del controlador. La emisión manual usa issue-credential en el registro del emisor. El diagrama omite respuestas REST y estados intermedios; cada controlador usa los IDs de registros de su propio agente.

Flujo del emisor

El emisor empieza por preparar cuatro datos: el DID del emisor, el esquema o la definición de credencial, las afirmaciones y el formato de credencial.

Para una tarjeta de embarque, la aerolínea selecciona su DID de emisor, elige el esquema de tarjeta de embarque y prepara afirmaciones como el nombre del pasajero, el número de vuelo, la fecha de salida, el asiento y el grupo de embarque. La elección de formato depende de las necesidades del verificador. Un lector de puerta puede necesitar solo una JWT-VC. Usa SD-JWT-VC cuando el viajero deba revelar solo parte de la credencial. Usa AnonCreds cuando el verificador necesite pruebas de conocimiento cero o pruebas de predicados.

El emisor crea una oferta de credencial mediante Cloud Agent:

```
POST /cloud-agent/issue-credentials/credential-offers
```

Las ofertas JWT usan `jwtVcPropertiesV1`:

```
{
  "connectionId": "holder-connection-id",
  "credentialFormat": "JWT",
  "jwtVcPropertiesV1": {
    "claims": {
      "passengerName": "Alice Wonderland",
      "flightNumber": "ID123",
      "departureAirport": "SCL",
      "arrivalAirport": "EZE",
      "seatNumber": "12A"
    },
    "issuingDID": "did:prism:issuer",
    "credentialSchema": {
      "id":
       ↪  "http://localhost:8080/cloud-agent/schema-registry/schemas/...",
      "type": "JsonSchemaValidator2018"
    }
  }
}
```

Las ofertas SD-JWT usan `sdJwtVcPropertiesV1`:

```
{
  "connectionId": "holder-connection-id",
  "credentialFormat": "SDJWT",
  "sdJwtVcPropertiesV1": {
    "claims": {
      "passengerName": "Alice Wonderland",
```

```
      "flightNumber": "ID123",
      "boardingGroup": "A",
      "exp": 1883000000
    },
    "issuingDID": "did:prism:issuer",
    "credentialSchema": {
      "id":
       ↪  "http://localhost:8080/cloud-agent/schema-registry/schemas/...",
      "type": "JsonSchemaValidator2018"
    }
  }
}
```

Las ofertas AnonCreds usan `anoncredsVcPropertiesV1` y un `credentialDefinitionId`:

```
{
  "connectionId": "holder-connection-id",
  "credentialFormat": "AnonCreds",
  "anoncredsVcPropertiesV1": {
    "claims": {
      "passengerName": "Alice Wonderland",
      "flightNumber": "ID123",
      "seatNumber": "12A"
    },
    "issuingDID": "did:prism:issuer",
    "credentialDefinitionId": "credential-definition-id"
  }
}
```

Después de crear la oferta, el emisor observa el estado del registro de emisión. Si el titular acepta la oferta y el emisor usa emisión automática, Cloud Agent emite la credencial cuando llega la solicitud del titular. Si la emisión automática está deshabilitada, el emisor envía la credencial desde el registro correspondiente:

```
POST /cloud-agent/issue-credentials/records/{recordId}/issue-credential
```

Flujo del titular

El titular recibe una oferta de credencial, la revisa y la acepta. En una billetera para usuarios, esa revisión debe mostrar el emisor, el tipo de credencial, el formato solicitado, una vista previa de las afirmaciones y los detalles relevantes para la privacidad. El titular debe saber qué acepta antes de que la billetera lo almacene.

El titular puede recuperar los registros de emisión de credenciales:

```
GET /cloud-agent/issue-credentials/records
```

Para aceptar una oferta JWT o SD-JWT, el titular proporciona un DID PRISM como `subjectId`:

```
{
  "subjectId": "did:prism:holder"
}
```

Para credenciales SD-JWT con vinculación de clave, el titular puede incluir `keyId`:

```
{
  "subjectId": "did:prism:holder",
  "keyId": "key-1"
}
```

Las ofertas AnonCreds no usan `subjectId` de la misma forma. El titular acepta la oferta con un cuerpo vacío:

```
{}
```

Después de aceptar, el titular recibe la credencial emitida mediante DIDComm. Identus documenta estados del titular como `OfferReceived`, `RequestPending`, `RequestSent` y `CredentialReceived`. La billetera almacena la credencial y después la usa en un flujo de presentación.

Emisión sin conexión previa

La emisión sin conexión previa sirve cuando el emisor y el titular aún no tienen una conexión DIDComm. La guía de emisión sin conexión previa de Identus documenta un punto de acceso de invitación fuera de banda:

```
POST /cloud-agent/issue-credentials/credential-offers/invitation
```

El emisor crea una invitación y comparte el código devuelto, por ejemplo mediante un código QR o enlace. El titular acepta la invitación mediante una billetera o SDK. El emisor responde con una oferta de credencial y continúan los pasos normales de aceptación del titular y emisión del emisor.

Un puesto de registro de una conferencia podría mostrar un código QR que inicia la emisión de una credencial de asistente. El asistente lo escanea con una billetera, acepta la oferta y recibe la credencial sin crear primero una relación permanente con el emisor.

Emisión OpenID4VCI

OID4VCI sirve para emisores que ya usan recorridos de usuario basados en OAuth. La emisión DIDComm sirve para billeteras y emisores que ya participan en protocolos entre agentes. Un despliegue de producción puede admitir ambas vías, y la política decide cuál corresponde a cada tipo de credencial.

En el modelo OID4VCI de Identus, Cloud Agent actúa como servidor emisor de credenciales. La billetera sigue el flujo de autorización, obtiene un token de acceso del servidor de autorización y usa ese token en el punto de acceso de credenciales. La política concreta reside en la configuración del emisor y del servidor de autorización, no en la billetera.

Presentación y verificación

La emisión proporciona una credencial al titular. La presentación permite que el titular demuestre algo con ella.

En Identus, la guía Present Proof Protocol 3.0 describe los flujos del verificador, titular y demostrador mediante DIDComm. Un verificador crea una solicitud de prueba:

```
POST /cloud-agent/present-proof/presentations
```

La solicitud puede incluir `domain` y `challenge`. Estos valores vinculan la presentación al verificador y a la sesión, lo que reduce el riesgo de repetición. El titular revisa la solicitud, selecciona credenciales y la acepta:

```
PATCH /cloud-agent/present-proof/presentations/{id}
```

La carga útil concreta depende del formato de credencial. Las presentaciones JWT usan `proofId`. Las presentaciones SD-JWT divulgan afirmaciones seleccionadas y usan `credentialFormat: SDJWT`. Las presentaciones AnonCreds usan una solicitud de presentación AnonCreds.

Después de verificar, Identus puede llegar a `PresentationVerified`. El verificador puede aceptar después la presentación, lo que cambia el registro a `PresentationAccepted`. Esta separación importa. La verificación comprueba la validez criptográfica y las reglas del protocolo. La aceptación registra una decisión de negocio del verificador.

En el ejemplo de la aerolínea, una credencial de billete válida aún puede incumplir la política de negocio. Un viajero puede tener un billete legítimo y aun así carecer del visado obligatorio, llegar después del cierre de la puerta o no superar una comprobación de seguridad. La verificación VC informa de si la credencial es auténtica, íntegra y aceptable según las reglas del protocolo. El verificador aún decide si la acepta para el viaje.

Revocación

La revocación permite que el emisor marque una credencial como no válida antes de su caducidad natural. Una aerolínea puede revocar una tarjeta de embarque después de cancelar una reserva. Una universidad puede revocar una credencial emitida por error. Una empresa puede revocar una credencial de empleado cuando termina su relación laboral.

La guía de revocación de Identus documenta la revocación de credenciales JWT mediante Verifiable Credentials Status List v2021. Una credencial revocable contiene un objeto `credentialStatus` con campos como `type`, `statusPurpose`, `statusListIndex` y `statusListCredential`.

Mecanismo de revocación

Una credencial revocable contiene datos de estado como estos:

```
{
  "credentialStatus": {
    "id": "http://localhost:8080/cloud-agent/credential-status/[UUID]#[IND
     ↪  EX]",
    "type": "StatusList2021Entry",
    "statusPurpose": "revocation",
    "statusListIndex": "94567",
    "statusListCredential":
     ↪  "http://localhost:8080/cloud-agent/credential-status/[UUID]"
  }
}
```

La URL `statusListCredential` apunta a una credencial de lista de estado. Esa credencial contiene una cadena de bits codificada donde cada posición corresponde a una credencial. `statusListIndex` identifica la posición de la credencial en esa cadena. Cuando el emisor revoca una credencial, cambia el bit correspondiente de 0 a 1.

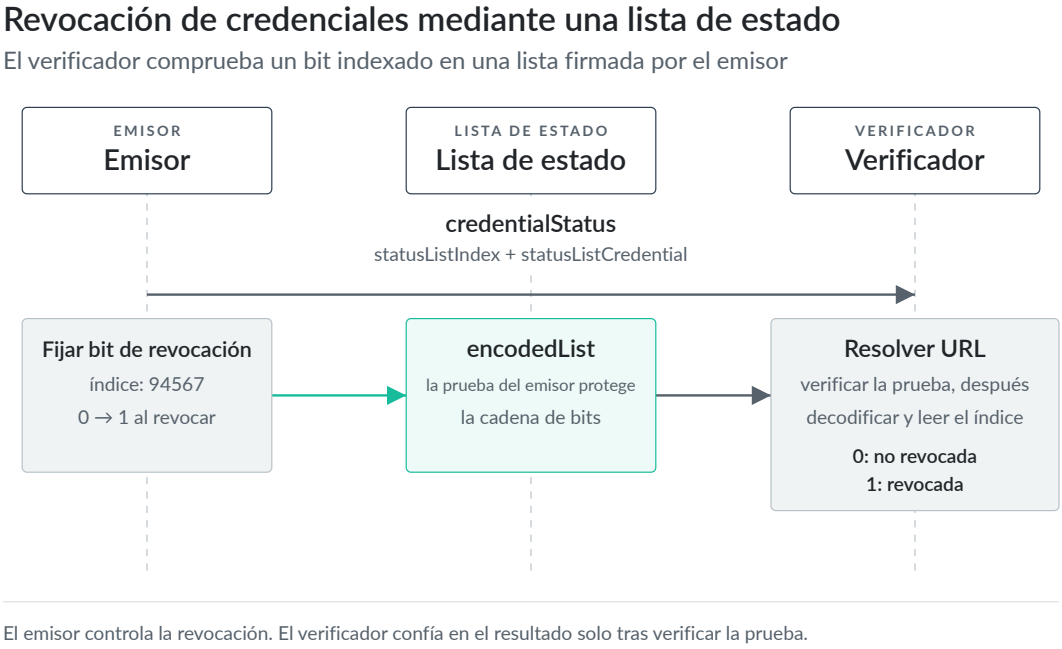


Figura 2: La credencial apunta a credentialStatus.statusListCredential y proporciona statusListIndex. El emisor cambia el bit indexado, y el verificador resuelve la lista, verifica su prueba, decodifica encodedList y lee ese bit.

Revocar una credencial

Solo el emisor de una credencial puede revocarla. Identus expone la operación de revocación como:

```
PATCH /cloud-agent/revoke-credential/{credential_id}
```

El emisor puede encontrar el ID de la credencial al listar los registros de credenciales emitidas y llamar después al punto de acceso de revocación para la credencial de destino. Tras la revocación, una verificación Present Proof posterior falla si el titular presenta esa credencial. Así la revocación permanece en la vía de verificación y no depende de que el titular elimine credenciales antiguas de su almacenamiento.

Comprobar el estado de revocación

Cuando un verificador recibe una presentación con una credencial que contiene `credentialStatus`, comprueba la lista de estado antes de aceptar la credencial.

Primero recupera la credencial de lista de estado desde `credentialStatus.statusListCredential`. Después verifica la prueba incorporada en esa credencial de estado. Identus documenta dos

tipos de prueba compatibles para credenciales de lista de estado: `DataIntegrityProof` con `eddsa-jcs-2022` para claves Ed25519 y `EcdsaSecp256k1Signature2019` para claves secp256k1.

Después de verificar la prueba, el verificador decodifica `credentialSubject.encodedList` y comprueba el bit en `statusListIndex`. Si el bit vale 1, la credencial está revocada. Si vale 0, la lista de estado no ha revocado esa credencial.

Este modelo de lista de estado tiene una ventaja de privacidad: el verificador recupera toda la lista en lugar de preguntar al emisor por una credencial concreta. El emisor no sabe qué credencial comprobó el verificador.

W3C publicó Bitstring Status List v1.0 como parte de la familia de recomendaciones VC 2.0 el 15 de mayo de 2025. La documentación de Identus citada arriba aún indica Status List v2021 para la revocación de credenciales JWT. Por ello, los implementadores deben seguir la guía actual de Identus para el comportamiento de Cloud Agent y consultar versiones futuras de Identus por posibles migraciones de listas de estado.

Verificación

Presentar una prueba

La verificación en Identus comienza cuando un verificador pide a un titular que presente pruebas de una o más credenciales. Para flujos DIDComm, Identus Cloud Agent usa el protocolo Present Proof. El Cloud Agent del verificador envía una solicitud de presentación mediante una conexión DIDComm existente. El Cloud Agent del titular devuelve una presentación y el del verificador comprueba el material de prueba recibido.

La guía Present Proof de Identus describe el flujo de API DIDComm:

1. El controlador del verificador llama a `POST /present-proof/presentations` en su Cloud Agent. La solicitud identifica la conexión DIDComm y describe la prueba que quiere el verificador.
2. El controlador del titular lee los registros de presentaciones pendientes desde `GET /present-proof/presentations`.
3. El controlador del titular acepta una solicitud mediante `PATCH /present-proof/presentations/{id}` con `action: "request-accept"`. Para credenciales JWT y SD-JWT, proporciona IDs de registros de credenciales mediante `proofId`. Para AnonCreds, proporciona la correspondencia de pruebas de credenciales en `anoncredPresentationRequest`.
4. El Cloud Agent del titular genera la presentación y la envía al Cloud Agent del verificador.
5. El Cloud Agent del verificador verifica la presentación. Un registro correcto del lado del verificador pasa por `PresentationReceived` y `PresentationVerified`.
6. El controlador del verificador acepta la presentación verificada mediante `PATCH /present-proof/presentations/{id}` con `action: "presentation-accept"`, o la rechaza en la lógica de la aplicación.

La estructura exacta de la solicitud depende del formato de credencial. Las solicitudes de presentación JWT y SD-JWT pueden incluir `options.challenge` y `options.domain` para que el titular firme material vinculado a la solicitud del verificador. Las solicitudes AnonCreds usan un objeto `anoncredPresentationRequest` con un nonce, atributos solicitados, predicados solicitados, restricciones e intervalos opcionales de no revocación.

El protocolo DIDComm Present Proof 3.0 es independiente del formato. Transporta una solicitud de presentación y una presentación, pero el formato de credencial determina las comprobaciones criptográficas que se ejecutan.

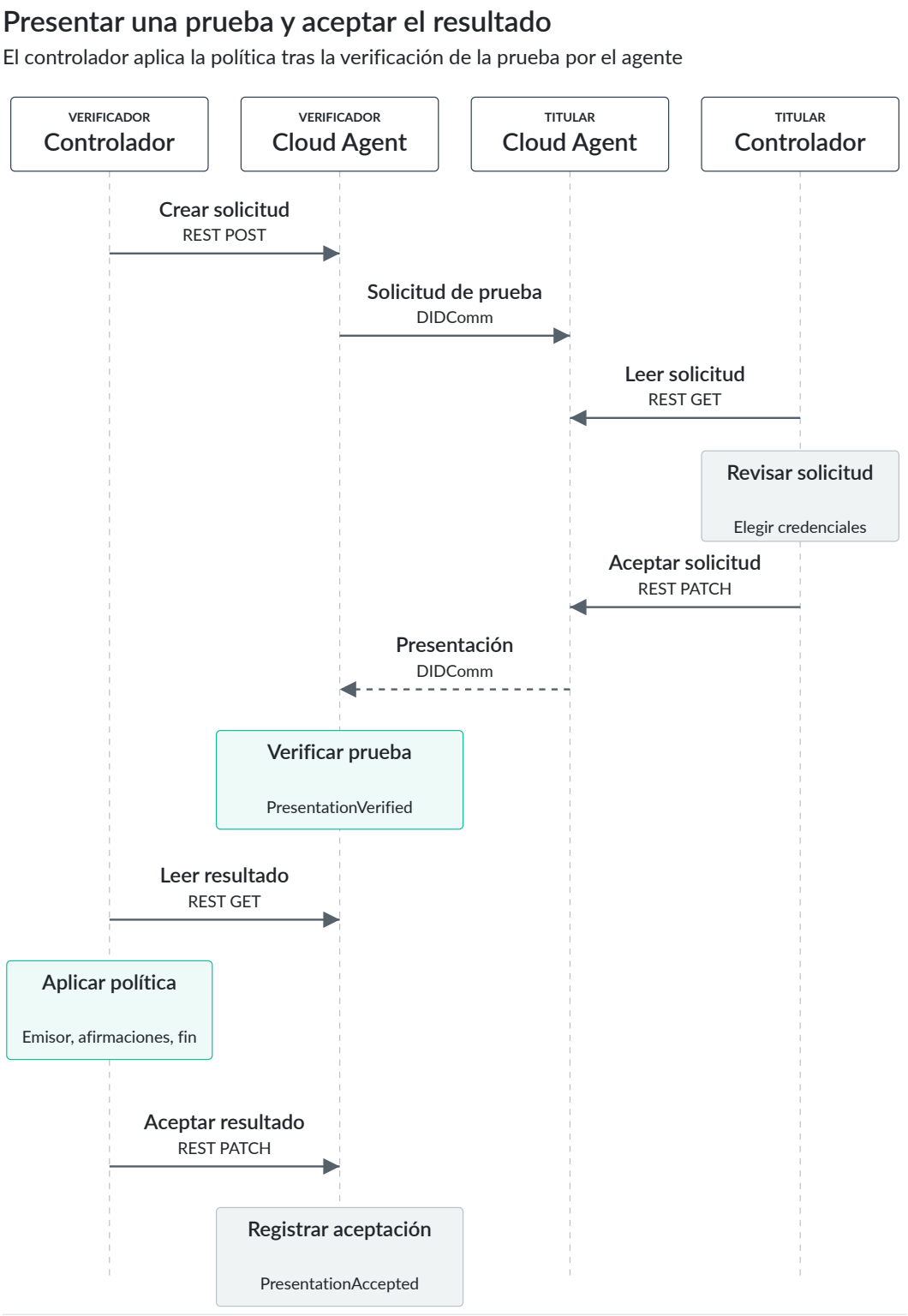


Figura 1: El verificador proporciona la vinculación de la solicitud para el formato elegido: challenge y domain para JWT o SD-JWT con vinculación de clave, o un nonce de AnonCreds. El controlador del titular selecciona credenciales y llama a request-accept. Después de PresentationVerified, el controlador del verificador comprueba su política y llama a presentation-accept solo si supera las comprobaciones. Un rechazo por política detiene la acción de la aplicación. Este diagrama sigue el flujo correcto y omite respuestas REST.

Verificación y aceptación

La verificación criptográfica y la aceptación del verificador son decisiones separadas.

El Cloud Agent del verificador puede comprobar si la presentación tiene una estructura correcta para su formato, está vinculada a la solicitud del verificador y cuenta con el material criptográfico esperado. Para JWT-VC, comprueba las firmas JWT y las claves resueltas mediante DID. Para SD-JWT-VC, comprueba la firma del emisor, los resúmenes de las afirmaciones divulgadas y la vinculación de clave del titular cuando la credencial contiene una clave `cnf`. Para AnonCreds, comprueba pruebas de igualdad, de predicados, agregadas y las pruebas de no revocación que solicitó el verificador.

El controlador del verificador aún decide si debe aceptar la prueba para la acción de negocio. El modelo de datos de credenciales verificables 2.0 de W3C separa verificación y validación: la verificación comprueba el mecanismo de protección y la validación aplica reglas de negocio a un uso específico de la credencial.

Después de que Cloud Agent informe de una presentación verificada, el controlador del verificador debe comprobar:

1. El verificador confía en el DID del emisor para el tipo y esquema de credencial.
2. La credencial usa el esquema o definición de credencial que indica la solicitud del verificador.
3. Las afirmaciones divulgadas satisfacen la política del verificador.
4. El período de validez y los datos de estado o revocación corresponden al momento que interesa al verificador.
5. El verificador acepta que el titular o presentador tiene autorización para este caso de uso.

Una prueba válida aún puede no superar la aceptación. El verificador de un local puede recibir una credencial de edad criptográficamente válida y rechazarla si el emisor no está en su lista de emisores de confianza o si la afirmación revelada no cumple la regla de edad del local.

Verificación directa de credenciales

Cloud Agent expone un punto de acceso de verificación directa en `POST /verification/credential`. Úsalo cuando el software del verificador tiene una credencial JWT codificada y necesita resultados explícitos de comprobación fuera de un intercambio DIDComm Present Proof.

El punto de acceso acepta una lista de credenciales codificadas y comprobaciones de verificación. La implementación actual del servicio incluye comprobaciones como verificación de firma y algoritmo, identificación del emisor, caducidad, fecha mínima de validez, audiencia, validación de esquema, validación del esquema del sujeto y análisis semántico de afirmaciones.

El punto de acceso de verificación directa no sustituye una solicitud de presentación. Verifica los datos de credencial que recibe. No demuestra que el titular actual controla la clave del

sujeto de la credencial para un desafío del verificador, ni decide si la aplicación debe aceptar un emisor o una afirmación.

Políticas de verificación

Identus usa el término `verification policy` para reglas almacenadas del verificador. El código de Cloud Agent expone puntos de acceso `/verification/policies` para crear, actualizar, obtener, listar y eliminar esas políticas. El modelo actual contiene una `CredentialSchemaAndTrustedIssuersConstraint` con:

1. `schemaId`, el esquema que espera el verificador.
2. `trustedIssuers`, los DIDs de emisores que acepta para ese esquema.

La descripción del punto de acceso en el código de Cloud Agent indica que estas políticas se aplican a credenciales verificables W3C en formato JWT y actualmente usan las restricciones `schemaId` y `trustedIssuers`.

El controlador del verificador puede usar una política antes de enviar una solicitud de presentación, después de recibir una presentación verificada o en ambos momentos. Antes de la solicitud, la política puede ayudarlo a pedir el esquema que acepta. Después de la verificación, puede ayudarlo a rechazar una credencial de un emisor que no es de confianza. El resultado de verificación criptográfica de Cloud Agent debe contribuir a esta decisión de política, no representar toda la decisión.

Si el despliegue usa un registro de confianza externo, mantén la consulta del registro separada de la verificación de pruebas. El verificador o su controlador consulta el registro, resuelve el conjunto de emisores de confianza para el esquema solicitado e incorpora el resultado a su decisión de política.

Divulgación selectiva

La divulgación selectiva permite que el titular revele solo las afirmaciones necesarias para la solicitud del verificador. Identus admite presentaciones con divulgación selectiva mediante SD-JWT y AnonCreds, pero ambos formatos usan mecanismos distintos.

La presentación JWT-VC simple expone la carga útil de la credencial necesaria para las comprobaciones del verificador. El formato sirve cuando este necesita la credencial completa, como una puerta de embarque que comprueba toda la tarjeta de embarque. Resulta poco adecuado cuando necesita un atributo o un predicado.

SD-JWT usa hashes con sal. El emisor firma una carga útil JWT que contiene resúmenes de las afirmaciones que permiten divulgación selectiva. Durante la presentación, el titular envía el SD-JWT firmado por el emisor y las divulgaciones seleccionadas para ese verificador. El verificador calcula el hash de cada valor divulgado con su sal y comprueba que el resumen aparece en la carga útil firmada. SD-JWT proporciona divulgación selectiva para afirmaciones JSON; no es un sistema de pruebas de predicados de conocimiento cero.

Identus añade un detalle de vinculación de clave para las presentaciones SD-JWT. Si el titular acepta una oferta SD-JWT con `keyId`, la credencial emitida puede incluir una vinculación de clave `cnf`. Una presentación posterior del titular puede firmar el `challenge` y el `domain` del verificador. Si la credencial SD-JWT no tiene clave `cnf`, la guía Present Proof de Identus indica que el titular no puede crear una presentación que firme el `challenge` y el `domain` del verificador.

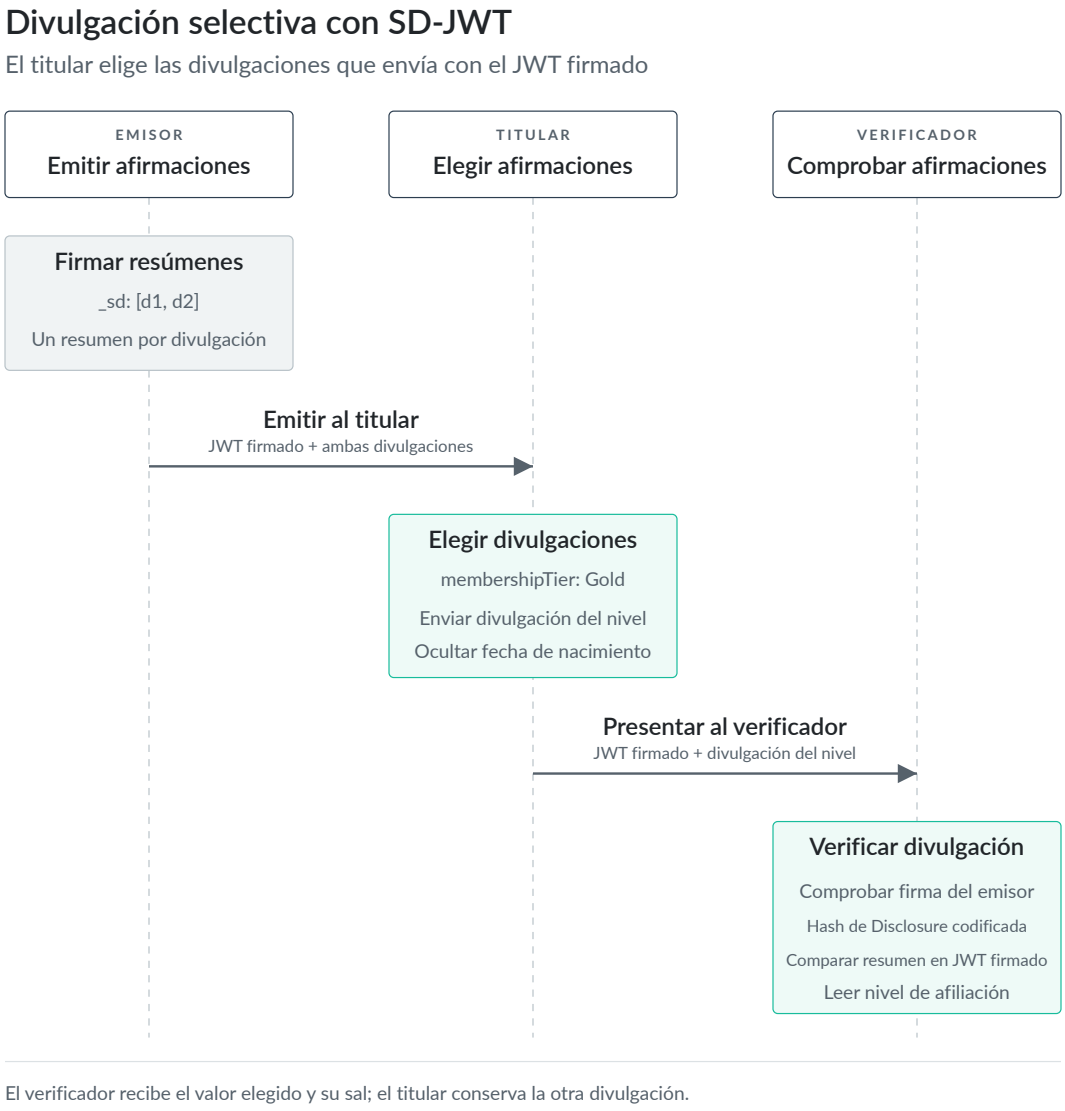


Figura 2: Este ejemplo usa dos propiedades de objeto que permiten divulgación selectiva. Cada Disclosure codifica [salt, name, value] como base64url. El verificador calcula el hash de esa cadena codificada y compara el resumen base64url con la carga útil firmada por el emisor, según RFC 9901, sección 4.2.3. Las etiquetas d1 y d2 representan resúmenes completos. Una credencial con cnf puede admitir vinculación de clave del titular a la solicitud del verificador.

AnonCreds usa pruebas de conocimiento cero. La solicitud de presentación del verificador puede pedir atributos revelados, predicados como `age >= 18`, restricciones como `schema_id` o `cred_def_id` e intervalos de no revocación. El titular selecciona credenciales que satisfacen la solicitud y crea una prueba. El verificador resuelve los esquemas, definiciones de credenciales y datos de revocación necesarios, y verifica la presentación.

AnonCreds permite que el titular demuestre un predicado sin revelar el atributo original. Para verificar la edad, un emisor público puede emitir una credencial que contiene un atributo de edad derivado de una fecha. Un local con restricción de edad puede solicitar un predicado AnonCreds para `age >= 21`. El titular presenta una prueba que satisface el predicado y el verificador comprueba la prueba, la restricción de emisor, la definición de credencial y el intervalo de revocación sin recibir la fecha completa de nacimiento del titular.

La divulgación selectiva no elimina la política del verificador. Este aún comprueba la confianza en el emisor, la correspondencia del esquema, el estado de la credencial, la vinculación de la solicitud y las reglas locales de aceptación de las afirmaciones o predicados divulgados.

Sección IV: despliegue

Instalación: entorno de producción

Alcance

La sección 2 inicia dos instancias locales de Cloud Agent con un PRISM Node de desarrollo. Este capítulo mantiene el mismo recorrido de aprendizaje y lo lleva hacia un despliegue `preprod` con características de producción. El objetivo es mostrar los límites entre servicios que un desarrollador debe comprender antes de que un servicio real de emisor o verificador dependa de Identus.

Los ejemplos de este capítulo son una base de preproducción para el aprendizaje de operadores. Muestran cómo copiar la configuración Docker local, fijar versiones actuales de componentes, exponer URLs públicas de Cloud Agent, proteger puertos de servicios, conectar PRISM Node con servicios Cardano y separar las simplificaciones del tutorial de los requisitos de producción.

Ejecuta primero este capítulo en Cardano `preprod`. Pasa a `mainnet` después de que el despliegue supere las comprobaciones de estado, billetera, DB Sync, publicación de DIDs y copias de seguridad al final del capítulo.

Selección de versiones

Usa las notas de versiones de Identus Platform como primera fuente de compatibilidad. Este libro es compatible con Identus Platform `v2.17`, que incluye Cloud Agent `2.2.0`, Mediator `1.2.1`, NeoPRISM `0.14.1` y SDK-TS `8.0.0`. La Tabla 1 muestra el conjunto completo de componentes. Cloud Agent `2.2.0` añade la integración del driver VDR y NeoPRISM como backend de nodo.

Los repositorios de los componentes siguen publicando versiones entre una versión de la plataforma y la siguiente. Trata cualquier versión de un componente más reciente que la fijada por la plataforma como una versión que requiere pruebas de compatibilidad. El README de Cloud Agent advierte que los pares de imágenes de Cloud Agent y PRISM Node sin comprobar pueden fallar en las comprobaciones de compatibilidad.

Este capítulo fija estas versiones:

```
AGENT_VERSION=2.2.0
PRISM_NODE_VERSION=2.6.1
CARDANO_WALLET_TAG=v2026-05-11
CARDANO_DB_SYNC_VERSION=13.7.1.0
```

`AGENT_VERSION` sigue la última versión de la plataforma. `PRISM_NODE_VERSION` sigue la última versión de PRISM Node comprobada para este capítulo, `v2.6.1`. Si cambias cualquiera de las imágenes Identus, vuelve a ejecutar la lista completa de comprobaciones al final del capítulo.

Las notas de la versión `v2026-05-11` de Cardano Wallet la emparejan con `cardano-node 11.0.1`. La última versión de Cardano DB Sync comprobada para este capítulo era `13.7.1.0`.

Modelo de despliegue

Un despliegue Identus de producción tiene varios actores y servicios:

- La aplicación controladora llama a la API REST de Cloud Agent y recibe webhooks.
- Cloud Agent administra el estado de los inquilinos, billeteras, DIDs, mensajes DIDComm V2, emisión, verificación y almacenamiento de credenciales, y publicación de listas de estado.
- PostgreSQL almacena el estado de Cloud Agent. PRISM Node y Cardano DB Sync necesitan sus propias bases de datos PostgreSQL.
- Vault almacena semillas de billetera y material secreto cuando `SECRET_STORAGE_BACKEND=vault`.
- El backend del nodo DID publica y resuelve operaciones `did:prism`. La configuración actual de Cloud Agent puede usar PRISM Node, NeoPRISM u otros drivers VDR según las opciones de driver seleccionadas.
- PRISM Node actúa como nodo de nivel 2 sobre Cardano. Con `NODE_LEDGER=cardano`, usa Cardano Wallet para enviar transacciones y Cardano DB Sync para leer bloques.
- Cardano Wallet se conecta a un nodo Cardano mediante su socket y expone una API HTTP a PRISM Node.
- Cardano DB Sync sigue el mismo nodo Cardano e indexa bloques en PostgreSQL para PRISM Node.

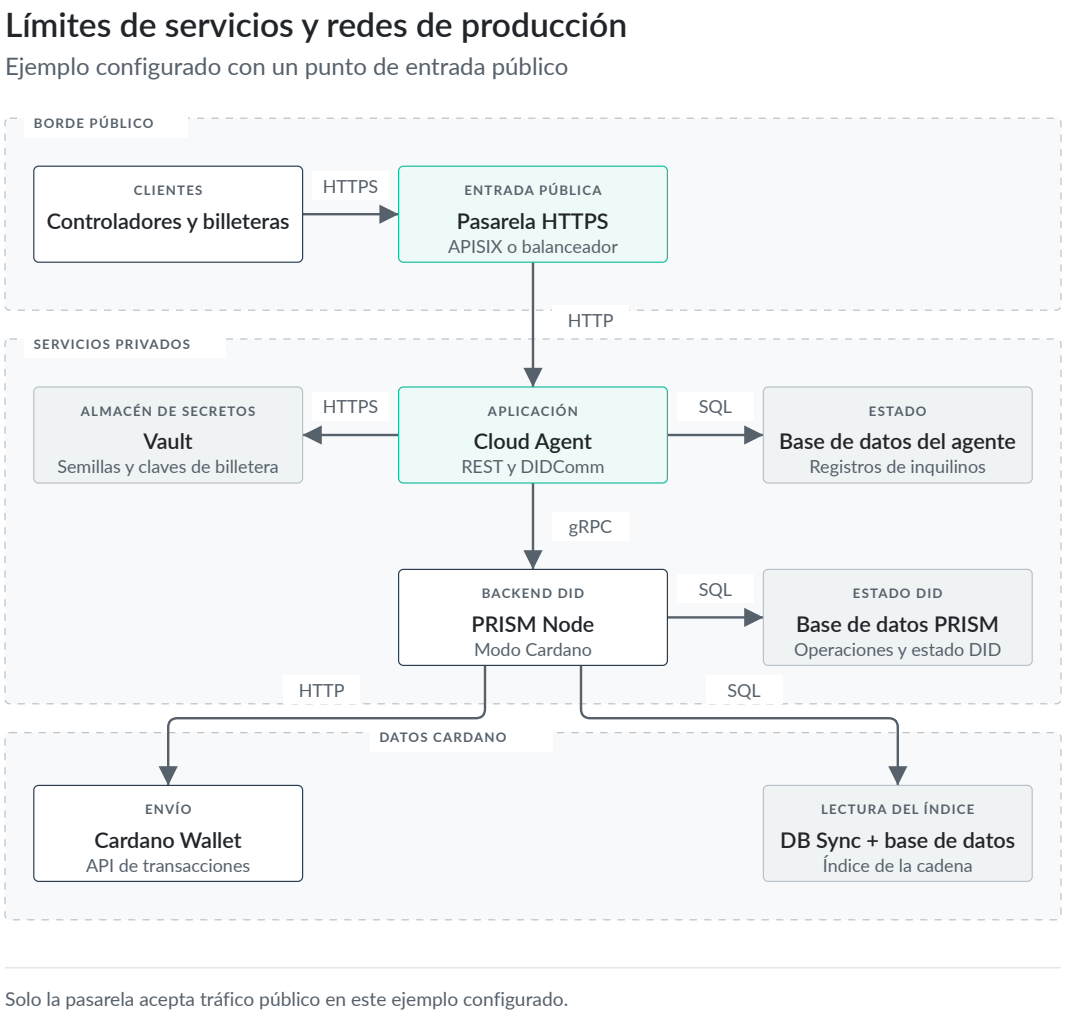


Figura 1: Este ejemplo configurado expone solo la pasarela HTTPS. Cloud Agent y cada servicio backend de datos, secretos y DIDs permanecen en redes privadas.

Para una vía de publicación `did:prism`:

1. El controlador del emisor solicita a Cloud Agent crear, actualizar o desactivar un DID `did:prism`.
2. Cloud Agent usa la semilla de billetera y la ruta de derivación almacenada para reconstruir el material de claves de la operación DID.
3. Cloud Agent firma la operación y la envía al backend de nodo DID configurado.
4. PRISM Node recibe la operación firmada, la programa, usa Cardano Wallet para enviar una transacción Cardano y lee bloques indexados mediante Cardano DB Sync.
5. Cloud Agent o el resolvedor lee el estado del DID cuando la operación alcanza la profundidad de confirmación configurada.

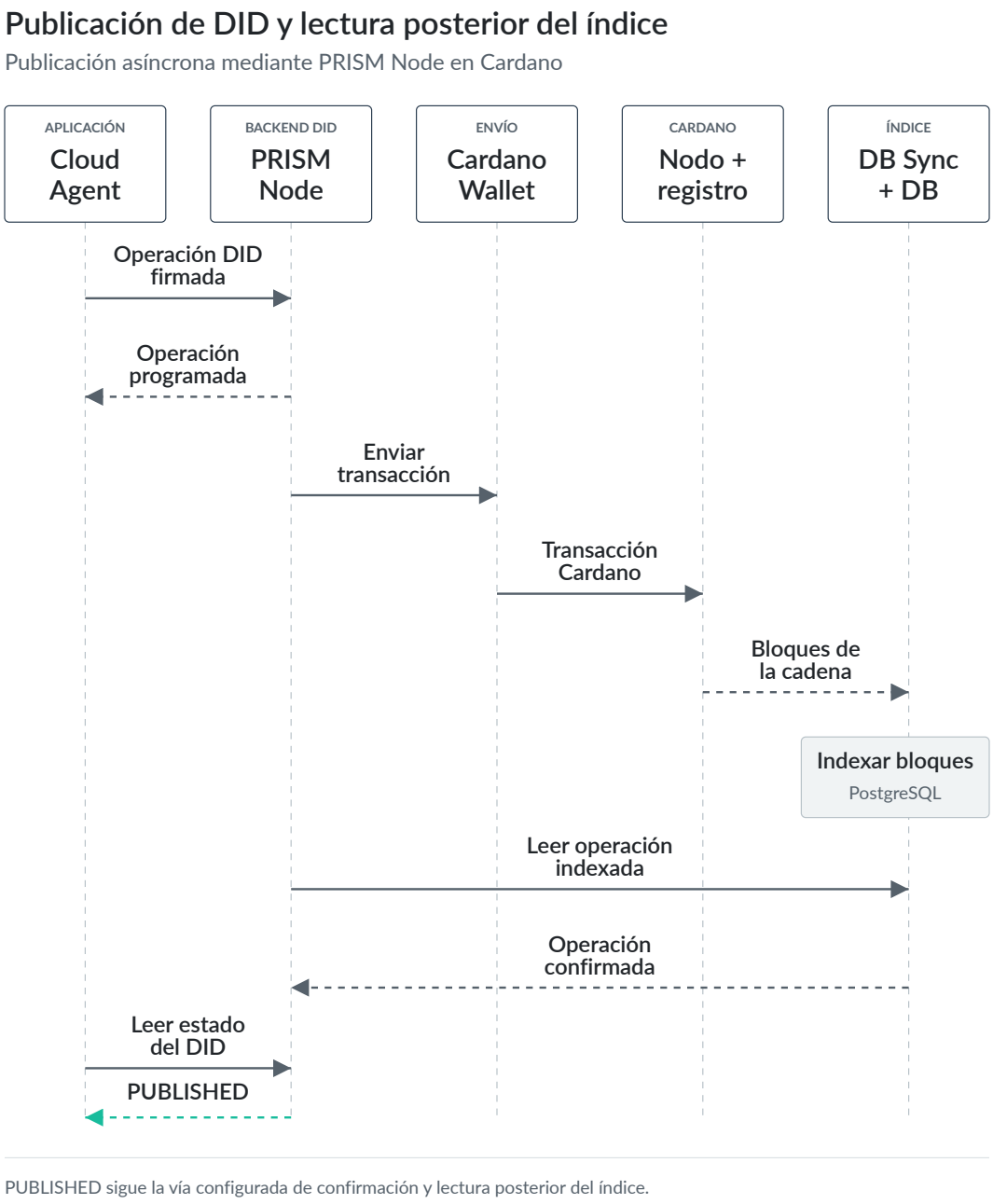


Figura 2: La publicación es asíncrona. PRISM Node envía mediante Cardano Wallet y lee la operación confirmada desde el índice DB Sync antes de que Cloud Agent indique que el DID está publicado.

La verificación de credenciales sigue una vía de decisión distinta. El software del verificador realiza comprobaciones técnicas como la verificación de pruebas, resolución de DIDs, uso de claves, estado de credenciales y vinculación de presentaciones. La parte que confía en el resultado aplica después comprobaciones de política, como DIDs de emisores aceptados,

esquemas, tipos de credenciales, entradas de registros de confianza o reglas de negocio.

Requisitos de hardware y datos

Usa la carga real y los requisitos de servicios Cardano para dimensionar la producción. Cloud Agent consume pocos recursos en comparación con los servicios Cardano. El dimensionamiento depende de la tasa de solicitudes, número de inquilinos, volumen de webhooks, emisión y verificación de credenciales, latencia de base de datos y latencia del almacenamiento de secretos.

Cardano DB Sync consume más recursos que el resto de este entorno. La página oficial de requisitos del sistema DB Sync indicaba estos requisitos de mainnet cuando se comprobó para este capítulo:

- Host Linux.
- Al menos 64 GB de RAM.
- Al menos 4 núcleos de CPU.
- Almacenamiento SSD con 60 000 IOPS o más.
- Al menos 700 GB de disco.

La misma página indicaba ejemplos actuales de almacenamiento mainnet de unos 203 GB para la base de datos del nodo Cardano, unos 10 GB para el estado del registro distribuido de DB Sync y unos 438 GB para la base de datos PostgreSQL de DB Sync. Indicaba ejemplos de memoria mainnet de unos 24 GB para `cardano-node` y unos 21 GB de RSS para `cardano-db-sync`.

`preprod` es mucho menor. La página DB Sync indicaba unos 12 GB para la base de datos del nodo Cardano, unos 2 GB para el estado del registro distribuido de DB Sync, unos 16 GB para su base de datos PostgreSQL, unos 5,5 GB de RAM para `cardano-node` y unos 3,5 GB de RSS para `cardano-db-sync`.

Ejecuta DB Sync y su base de datos PostgreSQL cerca uno del otro. La documentación recomienda ubicarlos juntos para reducir la latencia del tráfico de base de datos durante la sincronización. Si separas servicios entre máquinas, usa una red privada de baja latencia y supervisa las IOPS de la base de datos.

Preparar la configuración básica de Identus

Clona el repositorio actual de Cloud Agent y copia el directorio de infraestructura local en un directorio `preprod`:

```
git clone https://github.com/hyperledger-identus/cloud-agent
 ↪  identus-cloud-agent
cd identus-cloud-agent
```

```
cp -R infrastructure/local infrastructure/preprod
cp infrastructure/shared/docker-compose.yml
 ↪  infrastructure/shared/docker-compose-preprod.yml
```

Edita `infrastructure/preprod/run.sh` para que el script de preproducción use el archivo Compose copiado:

```
 PORT=${PORT} NETWORK=${NETWORK} DOCKERHOST=${DOCKERHOST} docker compose \
    -p ${NAME} \
-   -f ${SCRIPT_DIR}/../shared/docker-compose.yml \
+   -f ${SCRIPT_DIR}/../shared/docker-compose-preprod.yml \
    --env-file ${ENV_FILE} ${DEBUG} up ${BACKGROUND} ${WAIT}
```

Si tu copia aún referencia el registro antiguo de Cloud Agent, actualiza el nombre de imagen en `infrastructure/shared/docker-compose-preprod.yml`:

```
 cloud-agent:
-   image: ghcr.io/hyperledger/identus-cloud-agent:${AGENT_VERSION}
+   image: docker.io/hyperledgeridentus/identus-cloud-agent:${AGENT_VERSION
 ↪   :-latest}
```

Mantén PostgreSQL fuera de la red del host. Añade una vía privada de administración cuando los operadores necesiten acceso directo a la base de datos:

```
 db:
-   ports:
-     - "127.0.0.1:${PG_PORT:-5432}:5432"
+   # Keep PostgreSQL inside the Docker network for this tutorial.
+   # Add a host binding for a private administration path.
+   # ports:
+   #   - "127.0.0.1:${PG_PORT:-5432}:5432"
```

Define explícitamente el entorno de Cloud Agent. Su configuración actual admite `NODE_BACKEND`, ajustes de conexión de PRISM Node y NeoPRISM, y opciones de drivers VDR:

```
 cloud-agent:
   environment:
     POLLUX_STATUS_LIST_REGISTRY_PUBLIC_URL:
 ↪  ${POLLUX_STATUS_LIST_REGISTRY_PUBLIC_URL}
     DIDCOMM_SERVICE_URL: ${DIDCOMM_SERVICE_URL}
     REST_SERVICE_URL: ${REST_SERVICE_URL}
     PRISM_NODE_HOST: ${PRISM_NODE_HOST:-prism-node}
```

```
      PRISM_NODE_PORT: ${PRISM_NODE_PORT:-50053}
+     PRISM_NODE_USE_PLAIN_TEXT: ${PRISM_NODE_USE_PLAIN_TEXT:-true}
+     NODE_BACKEND: ${NODE_BACKEND:-prism-node}
+     NEOPRISM_BASE_URL: ${NEOPRISM_BASE_URL}
      VAULT_ADDR: ${VAULT_ADDR:-http://vault-server:8200}
      VAULT_TOKEN: ${VAULT_TOKEN:-root}
-     SECRET_STORAGE_BACKEND: postgres
-     DEV_MODE: true
+     SECRET_STORAGE_BACKEND: ${SECRET_STORAGE_BACKEND:-postgres}
+     DEV_MODE: ${DEV_MODE:-true}
+     VDR_PRISM_NODE_DRIVER_ENABLED: ${VDR_PRISM_NODE_DRIVER_ENABLED:-false}
+     VDR_NEOPRISM_DRIVER_ENABLED: ${VDR_NEOPRISM_DRIVER_ENABLED:-false}
      ADMIN_TOKEN:
      API_KEY_SALT:
      API_KEY_ENABLED:
      API_KEY_AUTHENTICATE_AS_DEFAULT_USER:
      API_KEY_AUTO_PROVISIONING:
```

Usa PRISM Node para este capítulo:

```
NODE_BACKEND=prism-node
PRISM_NODE_HOST=prism-node
PRISM_NODE_PORT=50053
PRISM_NODE_USE_PLAIN_TEXT=true
```

Los nuevos despliegues deben evaluar NeoPRISM antes de mainnet. La documentación VDR actual de Cloud Agent recomienda el driver NeoPRISM para producción e indica el de PRISM Node como una opción de producción heredada. Este capítulo conserva PRISM Node para mostrar en detalle los límites de Cardano Wallet y DB Sync.

Crear el archivo de entorno de preproducción

Crea `infrastructure/preprod/.env-preprod`:

```
cat > ./infrastructure/preprod/.env-preprod <<EOF
### Identus image pins
AGENT_VERSION=2.2.0
PRISM_NODE_VERSION=2.6.1

### Docker project and public URLs
PORT=8000
NETWORK=identus-preprod
DOCKERHOST=agent.example.test
```

```
REST_SERVICE_URL=https://agent.example.test/cloud-agent
DIDCOMM_SERVICE_URL=https://agent.example.test/didcomm
POLLUX_STATUS_LIST_REGISTRY_PUBLIC_URL=https://agent.example.test/cloud-ag
↪  ent

### Cloud Agent access control
API_KEY_ENABLED=true
API_KEY_AUTO_PROVISIONING=false
API_KEY_AUTHENTICATE_AS_DEFAULT_USER=false
ADMIN_TOKEN=replace-with-admin-token
API_KEY_SALT=replace-with-long-random-salt
DEFAULT_WALLET_ENABLED=false

### Secret storage
SECRET_STORAGE_BACKEND=postgres
VAULT_ADDR=http://vault-server:8200
VAULT_USE_SEMANTIC_PATH=true
VAULT_DEV_ROOT_TOKEN_ID=replace-for-lab
VAULT_TOKEN=replace-for-lab

### Databases
AGENT_DB_USER=postgres
AGENT_DB_PASSWORD=replace-with-agent-db-password
PGADMIN_DEFAULT_PASSWORD=replace-with-pgadmin-password

### DID node backend
NODE_BACKEND=prism-node
PRISM_NODE_HOST=prism-node
PRISM_NODE_PORT=50053
PRISM_NODE_USE_PLAIN_TEXT=true
VDR_PRISM_NODE_DRIVER_ENABLED=false
VDR_NEOPRISM_DRIVER_ENABLED=false

### PRISM Node Cardano mode
NODE_LEDGER=cardano
NODE_CARDANO_NETWORK=testnet
NODE_CARDANO_WALLET_ID=replace-after-wallet-create
NODE_CARDANO_WALLET_PASSPHRASE=replace-after-wallet-create
NODE_CARDANO_PAYMENT_ADDRESS=replace-after-wallet-create
NODE_CARDANO_WALLET_API_HOST=cardano-wallet
NODE_CARDANO_WALLET_API_PORT=8090
NODE_CARDANO_DB_SYNC_HOST=cardano-db-sync-postgres:5432
NODE_CARDANO_DB_SYNC_DATABASE=cexplorer
NODE_CARDANO_DB_SYNC_USERNAME=postgres
NODE_CARDANO_DB_SYNC_PASSWORD=replace-with-db-sync-password
```

```
### Cardano services
CARDANO_NETWORK=preprod
CARDANO_WALLET_TAG=v2026-05-11
CARDANO_DB_SYNC_VERSION=13.7.1.0
NODE_DB=$PWD/cardano/node-db
WALLET_DB=$PWD/cardano/wallet-db
NODE_CONFIGS=$PWD/cardano/configs
NODE_SOCKET_DIR=$PWD/cardano/ipc
NODE_SOCKET_NAME=node.socket
WALLET_PORT=127.0.0.1:8090

### Keycloak, disabled until the Keycloak section is completed
KEYCLOAK_ENABLED=false
KEYCLOAK_URL=https://iam.example.test
KEYCLOAK_REALM=identus
KEYCLOAK_CLIENT_ID=cloud-agent
KEYCLOAK_CLIENT_SECRET=replace-when-keycloak-enabled
KEYCLOAK_UMA_AUTO_UPGRADE_RPT=false
EOF
```

`SECRET_STORAGE_BACKEND=postgres` mantiene pequeño el laboratorio. La documentación de almacenamiento de secretos de Cloud Agent indica que PostgreSQL almacena los secretos en texto sin cifrar y no sirve para producción. Antes de que llegue tráfico del emisor a este entorno, establece `SECRET_STORAGE_BACKEND=vault` y configura Vault con un servidor de producción, una política de tokens o AppRole.

Ejemplo Compose con características de producción

El ejemplo `infrastructure/shared/docker-compose-preprod.yml` reúne los cambios anteriores en un archivo. Conserva la vista completa de configuración del tutorial original y mantiene visibles los límites de producción.

Este archivo aún es una base de laboratorio de preproducción. Sustituye el servidor Vault de desarrollo, las credenciales predeterminadas de base de datos, el listener HTTP de APISIX y la definición del servicio DB Sync antes de mainnet. Mantén el archivo bajo control de versiones para que los desarrolladores comparen el entorno local con el entorno de características de producción.

```
---
version: "3.8"

services:
  db:
    image: postgres:13
```

```
    environment:
      POSTGRES_MULTIPLE_DATABASES: "pollux,connect,agent,node_db"
      POSTGRES_USER: ${AGENT_DB_USER:-postgres}
      POSTGRES_PASSWORD: ${AGENT_DB_PASSWORD}
    volumes:
      - pg_data_db:/var/lib/postgresql/data
      -
↪   ./postgres/init-script.sh:/docker-entrypoint-initdb.d/init-script.sh
      - ./postgres/max_conns.sql:/docker-entrypoint-initdb.d/max_conns.sql
    # Keep the database inside the Docker network.
    # ports:
    #   - "127.0.0.1:${PG_PORT:-5432}:5432"
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "${AGENT_DB_USER:-postgres}",
↪   "-d", "agent"]
      interval: 10s
      timeout: 5s
      retries: 5

  pgadmin:
    image: dpage/pgadmin4
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL:-pgadmin4@pgadmin.org}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
      PGADMIN_CONFIG_SERVER_MODE: "False"
    volumes:
      - pgadmin:/var/lib/pgadmin
    ports:
      - "127.0.0.1:${PGADMIN_PORT:-5050}:80"
    depends_on:
      db:
        condition: service_healthy
    profiles:
      - debug

  vault-server:
    image: hashicorp/vault:1.15.6
    environment:
      VAULT_ADDR: "http://0.0.0.0:8200"
      VAULT_DEV_ROOT_TOKEN_ID: ${VAULT_DEV_ROOT_TOKEN_ID}
    command: server -dev -dev-root-token-id=${VAULT_DEV_ROOT_TOKEN_ID}
    cap_add:
      - IPC_LOCK
    healthcheck:
      test: ["CMD", "vault", "status"]
      interval: 10s
```

```
    timeout: 5s
    retries: 5

prism-node:
  image: ghcr.io/input-output-hk/prism-node:${PRISM_NODE_VERSION}
  environment:
    NODE_PSQL_HOST: db:5432
    NODE_PSQL_DATABASE: node_db
    NODE_PSQL_USERNAME: ${AGENT_DB_USER:-postgres}
    NODE_PSQL_PASSWORD: ${AGENT_DB_PASSWORD}
    NODE_LEDGER: ${NODE_LEDGER:-cardano}
    NODE_CARDANO_NETWORK: ${NODE_CARDANO_NETWORK:-testnet}
    NODE_CARDANO_WALLET_ID: ${NODE_CARDANO_WALLET_ID}
    NODE_CARDANO_WALLET_PASSPHRASE: ${NODE_CARDANO_WALLET_PASSPHRASE}
    NODE_CARDANO_PAYMENT_ADDRESS: ${NODE_CARDANO_PAYMENT_ADDRESS}
    NODE_CARDANO_WALLET_API_HOST:
↪   ${NODE_CARDANO_WALLET_API_HOST:-cardano-wallet}
    NODE_CARDANO_WALLET_API_PORT: ${NODE_CARDANO_WALLET_API_PORT:-8090}
    NODE_CARDANO_DB_SYNC_HOST: ${NODE_CARDANO_DB_SYNC_HOST}
    NODE_CARDANO_DB_SYNC_DATABASE:
↪   ${NODE_CARDANO_DB_SYNC_DATABASE:-cexplorer}
    NODE_CARDANO_DB_SYNC_USERNAME: ${NODE_CARDANO_DB_SYNC_USERNAME}
    NODE_CARDANO_DB_SYNC_PASSWORD: ${NODE_CARDANO_DB_SYNC_PASSWORD}
  depends_on:
    db:
      condition: service_healthy
    cardano-wallet:
      condition: service_started
    cardano-db-sync-postgres:
      condition: service_started

cloud-agent:
  image: docker.io/hyperledgeridentus/identus-cloud-agent:${AGENT_VERSIO
↪ N:-latest}
  environment:
    POLLUX_DB_HOST: db
    POLLUX_DB_PORT: 5432
    POLLUX_DB_NAME: pollux
    POLLUX_DB_USER: ${AGENT_DB_USER:-postgres}
    POLLUX_DB_PASSWORD: ${AGENT_DB_PASSWORD}
    CONNECT_DB_HOST: db
    CONNECT_DB_PORT: 5432
    CONNECT_DB_NAME: connect
    CONNECT_DB_USER: ${AGENT_DB_USER:-postgres}
    CONNECT_DB_PASSWORD: ${AGENT_DB_PASSWORD}
    AGENT_DB_HOST: db
```

```
      AGENT_DB_PORT: 5432
      AGENT_DB_NAME: agent
      AGENT_DB_USER: ${AGENT_DB_USER:-postgres}
      AGENT_DB_PASSWORD: ${AGENT_DB_PASSWORD}
      POLLUX_STATUS_LIST_REGISTRY_PUBLIC_URL:
↪   ${POLLUX_STATUS_LIST_REGISTRY_PUBLIC_URL}
      DIDCOMM_SERVICE_URL: ${DIDCOMM_SERVICE_URL}
      REST_SERVICE_URL: ${REST_SERVICE_URL}
      PRISM_NODE_HOST: ${PRISM_NODE_HOST:-prism-node}
      PRISM_NODE_PORT: ${PRISM_NODE_PORT:-50053}
      PRISM_NODE_USE_PLAIN_TEXT: ${PRISM_NODE_USE_PLAIN_TEXT:-true}
      NODE_BACKEND: ${NODE_BACKEND:-prism-node}
      NEOPRISM_BASE_URL: ${NEOPRISM_BASE_URL}
      VAULT_ADDR: ${VAULT_ADDR:-http://vault-server:8200}
      VAULT_TOKEN: ${VAULT_TOKEN}
      VAULT_APPROLE_ROLE_ID: ${VAULT_APPROLE_ROLE_ID}
      VAULT_APPROLE_SECRET_ID: ${VAULT_APPROLE_SECRET_ID}
      VAULT_USE_SEMANTIC_PATH: ${VAULT_USE_SEMANTIC_PATH:-true}
      SECRET_STORAGE_BACKEND: ${SECRET_STORAGE_BACKEND:-postgres}
      DEV_MODE: ${DEV_MODE:-false}
      DEFAULT_WALLET_ENABLED: ${DEFAULT_WALLET_ENABLED:-false}
      DEFAULT_WALLET_SEED: ${DEFAULT_WALLET_SEED}
      DEFAULT_WALLET_WEBHOOK_URL: ${DEFAULT_WALLET_WEBHOOK_URL}
      DEFAULT_WALLET_WEBHOOK_API_KEY: ${DEFAULT_WALLET_WEBHOOK_API_KEY}
      DEFAULT_WALLET_AUTH_API_KEY: ${DEFAULT_WALLET_AUTH_API_KEY}
      GLOBAL_WEBHOOK_URL: ${GLOBAL_WEBHOOK_URL}
      GLOBAL_WEBHOOK_API_KEY: ${GLOBAL_WEBHOOK_API_KEY}
      WEBHOOK_PARALLELISM: ${WEBHOOK_PARALLELISM}
      ADMIN_TOKEN: ${ADMIN_TOKEN}
      API_KEY_SALT: ${API_KEY_SALT}
      API_KEY_ENABLED: ${API_KEY_ENABLED:-true}
      API_KEY_AUTHENTICATE_AS_DEFAULT_USER:
↪   ${API_KEY_AUTHENTICATE_AS_DEFAULT_USER:-false}
      API_KEY_AUTO_PROVISIONING: ${API_KEY_AUTO_PROVISIONING:-false}
      KEYCLOAK_ENABLED: ${KEYCLOAK_ENABLED:-false}
      KEYCLOAK_URL: ${KEYCLOAK_URL}
      KEYCLOAK_REALM: ${KEYCLOAK_REALM}
      KEYCLOAK_CLIENT_ID: ${KEYCLOAK_CLIENT_ID}
      KEYCLOAK_CLIENT_SECRET: ${KEYCLOAK_CLIENT_SECRET}
      KEYCLOAK_UMA_AUTO_UPGRADE_RPT:
↪   ${KEYCLOAK_UMA_AUTO_UPGRADE_RPT:-false}
      VDR_PRISM_NODE_DRIVER_ENABLED:
↪   ${VDR_PRISM_NODE_DRIVER_ENABLED:-false}
      VDR_NEOPRISM_DRIVER_ENABLED: ${VDR_NEOPRISM_DRIVER_ENABLED:-false}
    depends_on:
      db:
```

```
      condition: service_healthy
    prism-node:
      condition: service_started
    vault-server:
      condition: service_healthy
  healthcheck:
    test: ["CMD", "curl", "-f", "http://cloud-agent:8085/_system/health"]
    interval: 30s
    timeout: 10s
    retries: 5
  extra_hosts:
    - "host.docker.internal:host-gateway"

swagger-ui:
  image: swaggerapi/swagger-ui:v5.1.0
  environment:
    - 'URLS=[
      { name: "Cloud Agent", url: "/docs/cloud-agent/api/docs.yaml" }
      ]'

apisix:
  image: apache/apisix:2.15.0-alpine
  volumes:
    - ./apisix/conf/apisix.yaml:/usr/local/apisix/conf/apisix.yaml:ro
    - ./apisix/conf/config.yaml:/usr/local/apisix/conf/config.yaml:ro
  ports:
    - "${PORT}:9080/tcp"
  depends_on:
    - cloud-agent
    - swagger-ui

cardano-node:
  image: cardanofoundation/cardano-wallet:${CARDANO_WALLET_TAG}
  environment:
    CARDANO_NODE_SOCKET_PATH: /ipc/${NODE_SOCKET_NAME}
  volumes:
    - ${NODE_DB}:/data
    - ${NODE_SOCKET_DIR}:/ipc
    - ${NODE_CONFIGS}:/configs
  entrypoint: []
  command: >
    cardano-node run
      --topology /configs/cardano/${CARDANO_NETWORK}/topology.json
      --database-path /data
      --socket-path /ipc/${NODE_SOCKET_NAME}
      --config /configs/cardano/${CARDANO_NETWORK}/config.json
```

```
      +RTS -N -A16m -qg -qb -RTS
  restart: on-failure

cardano-wallet:
  image: cardanofoundation/cardano-wallet:${CARDANO_WALLET_TAG}
  volumes:
    - ${WALLET_DB}:/wallet-db
    - ${NODE_SOCKET_DIR}:/ipc
    - ${NODE_CONFIGS}:/configs
  ports:
    - "${WALLET_PORT}:8090"
  entrypoint: []
  command: >
    cardano-wallet serve
      --node-socket /ipc/${NODE_SOCKET_NAME}
      --database /wallet-db
      --listen-address 0.0.0.0
      --testnet /configs/cardano/${CARDANO_NETWORK}/byron-genesis.json
  depends_on:
    - cardano-node
  restart: on-failure

cardano-db-sync-postgres:
  image: postgres:14
  environment:
    POSTGRES_USER: ${NODE_CARDANO_DB_SYNC_USERNAME:-postgres}
    POSTGRES_PASSWORD: ${NODE_CARDANO_DB_SYNC_PASSWORD}
    POSTGRES_DB: ${NODE_CARDANO_DB_SYNC_DATABASE:-cexplorer}
  volumes:
    - cardano_db_sync_postgres:/var/lib/postgresql/data

cardano-db-sync:
  image: ghcr.io/intersectmbo/cardano-db-sync:${CARDANO_DB_SYNC_VERSION}
  depends_on:
    - cardano-node
    - cardano-db-sync-postgres
  volumes:
    - ${NODE_SOCKET_DIR}:/ipc
    - ${NODE_CONFIGS}:/configs
  environment:
    CARDANO_NODE_SOCKET_PATH: /ipc/${NODE_SOCKET_NAME}
    NETWORK: ${CARDANO_NETWORK}
    POSTGRES_HOST: cardano-db-sync-postgres
    POSTGRES_PORT: 5432
    POSTGRES_DB: ${NODE_CARDANO_DB_SYNC_DATABASE:-cexplorer}
    POSTGRES_USER: ${NODE_CARDANO_DB_SYNC_USERNAME:-postgres}
```

```
      POSTGRES_PASSWORD: ${NODE_CARDANO_DB_SYNC_PASSWORD}

volumes:
  pg_data_db:
  pgadmin:
  cardano_db_sync_postgres:
```

Configuración y seguridad de producción

Trata el archivo de preproducción como una referencia práctica de marcadores de posición y límites de servicios. Guarda el ejemplo en el repositorio para que los desarrolladores vean cada componente. Carga los valores reales desde el sistema de despliegue, el almacén de secretos de CI, Vault o un mecanismo de secretos sellados.

Una configuración de producción de Cloud Agent debe definir estas decisiones explícitamente:

- Versiones fijas de imágenes de Cloud Agent, PRISM Node o NeoPRISM, Cardano Wallet, nodo Cardano y DB Sync.
- URLs públicas de `REST_SERVICE_URL`, `DIDCOMM_SERVICE_URL` y `POLLUX_STATUS_LIST_REGISTRY_PUBLIC`
- Modelo de inquilinos: billetera predeterminada deshabilitada para varios inquilinos, claves de API o Keycloak habilitados y un `ADMIN_TOKEN` largo y aleatorio.
- Backend de secretos: `SECRET_STORAGE_BACKEND=vault` para despliegues de emisores y verificadores que conservan semillas de billetera.
- Backend DID: PRISM Node para este tutorial, o NeoPRISM después de validar el driver de producción más reciente en la versión de destino.
- Conexiones de red Cardano: `NODE_CARDANO_NETWORK=testnet` o `mainnet` para PRISM Node, y `CARDANO_NETWORK=preprod` o `mainnet` para la configuración del nodo Cardano, Cardano Wallet y DB Sync.

Los valores de URL pública forman parte del comportamiento del protocolo. Las billeteras de titulares y los servicios de mediador usan el punto de acceso DIDComm para entregar mensajes DIDComm V2. El software del verificador o las aplicaciones controladoras usan la URL REST para llamadas de API y acceso a listas de estado. Si un balanceador de carga cambia la ruta o el host, actualiza las URLs de Cloud Agent para que correspondan a la dirección accesible desde el exterior.

Establece `DEV_MODE=false` para despliegues que superen el tutorial local. Rota `ADMIN_TOKEN`, claves de API, contraseñas de base de datos, credenciales de Vault, secretos de cliente Keycloak y contraseñas de billetera mediante el mismo proceso que otras credenciales de producción. Almacena las semillas reales de billetera, frases mnemónicas Cardano y credenciales de administración Keycloak en el sistema de secretos de producción.

Refuerzo de Docker y PostgreSQL

El archivo Compose conserva la estructura original del tutorial para que el desarrollador inspeccione todos los servicios. Los operadores de producción deben reducir la superficie de exposición de Docker.

Mantén los puertos PostgreSQL fuera de interfaces públicas. Las bases de datos de Cloud Agent, PRISM Node y DB Sync pueden compartir un clúster PostgreSQL administrado. Cada servicio debe usar una base de datos separada, credenciales separadas y una política de copias de seguridad que corresponda a sus necesidades de recuperación. DB Sync almacena datos de índice de la cadena que pueden reconstruirse o restaurarse desde instantáneas. Las bases de datos de Cloud Agent y PRISM Node contienen estado de aplicación que debe tener copias de seguridad y pruebas de restauración.

Sustituye las contraseñas predeterminadas `postgres` antes de la primera ejecución no local. El tutorial usa `POSTGRES_MULTIPLE_DATABASES` por comodidad; un despliegue de producción debe crear bases de datos y usuarios mediante migraciones, código de infraestructura o aprovisionamiento de bases de datos administradas.

Los secretos de Docker Compose montan archivos bajo `/run/secrets/<name>` dentro del contenedor. Imágenes como PostgreSQL admiten la convención `_FILE` para algunos valores secretos. Usa ese patrón cuando la imagen lo admita:

```
secrets:
  agent_db_password:
    file: ./config/secrets/agent_db_password

services:
  db:
    image: postgres:13
    secrets:
      - agent_db_password
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/agent_db_password
```

Cloud Agent lee sus valores de base de datos, API, Vault y Keycloak desde variables de entorno. Inyéctalos mediante el orquestador o la plataforma de despliegue. Los secretos Compose para variables de Cloud Agent requieren un wrapper de punto de entrada que lea archivos de secretos y exporte los nombres de variable esperados.

Establece la rotación de registros para contenedores de larga ejecución. El nodo Cardano, Cardano Wallet y DB Sync producen registros continuos durante la sincronización. El ejemplo posterior de este capítulo establece límites de tamaño `json-file` para los servicios Cardano. Aplica la misma política a Cloud Agent, PRISM Node, APISIX, Vault y PostgreSQL cuando uses Compose fuera del laboratorio.

SSL y proxy inverso

Expón Cloud Agent mediante HTTPS. El servicio APISIX del ejemplo escucha en el puerto HTTP `9080` para mantener simple el laboratorio. En producción, termina TLS en APISIX, un balanceador de carga u otro proxy inverso y reenvía a las rutas internas de Cloud Agent y DIDComm.

Define un plan de rutas públicas antes de crear DIDs o emitir credenciales:

```
https://agent.example.test/cloud-agent  -> Cloud Agent REST API
https://agent.example.test/didcomm      -> Cloud Agent DIDComm endpoint
https://agent.example.test/apidocs      -> Swagger UI, private or disabled
https://iam.example.test/realms/...     -> Keycloak OIDC and UMA endpoints
```

Si APISIX termina TLS, configura TLS 1.2 y TLS 1.3 a nivel de pasarela o en cada recurso SSL específico de SNI:

```
apisix:
  ssl:
    ssl_protocols: TLSv1.2 TLSv1.3
```

Vincula la API de administración APISIX a una red privada. Si el despliegue expone Swagger UI, protégelo con controles de red o autenticación. La ruta OpenAPI sirve para incorporar desarrolladores y proporciona a los atacantes un mapa completo de la API.

Keycloak necesita una configuración de proxy separada. Su documentación de producción requiere HTTPS para el tráfico sensible de autenticación y recomienda un proxy inverso o balanceador de carga.

Dirige el proxy al puerto de aplicación de Keycloak, normalmente `8443` o `8080` cuando habilitas HTTP detrás del proxy. Mantén interno el puerto de administración `9000`.

Expón las rutas públicas OIDC y estáticas que necesitan los clientes, como `/realms/`, `/resources/` y `/.well-known/`. Mantén `/admin/`, `/metrics` y `/health` en una vía interna de administración. Configura `proxy-headers` y las direcciones de proxies de confianza para que Keycloak acepte valores reenviados de host y esquema desde esos proxies.

Administración de claves con HashiCorp Vault

Cloud Agent crea, almacena y usa material de semillas de billetera para operaciones DID. La semilla de billetera de un inquilino permite que el agente vuelva a derivar claves DID después de reiniciar o desplegar de nuevo. Perderla puede dejar inutilizables los DIDs existentes para futuras actualizaciones, desactivación o flujos de emisión que dependen de las mismas claves.

Establece Vault como backend de producción:

```
SECRET_STORAGE_BACKEND=vault
VAULT_ADDR=https://vault.example.internal:8200
VAULT_USE_SEMANTIC_PATH=true
```

Reserva `VAULT_TOKEN` para el laboratorio o un procedimiento de emergencia. Para despliegue automático, prefiere AppRole:

```
VAULT_APPROLE_ROLE_ID=replace-with-role-id
VAULT_APPROLE_SECRET_ID=replace-with-secret-id
```

Cloud Agent espera permisos bajo `/secret/*` para el montaje y diseño de política del tutorial. Una política mínima para esa ruta es:

```
path "secret/*" {
  capabilities = ["create", "read", "update", "patch", "delete", "list"]
}
```

Vault almacena secretos de Cloud Agent bajo rutas específicas de billetera, como `/secret/<wallet-id>/seed`, rutas de claves de DIDs de pares y rutas genéricas de secretos. Crea copias de seguridad del almacenamiento Vault, comprueba la restauración y documenta el procedimiento de apertura o claves de recuperación. Un servidor Vault de desarrollo iniciado con `server -dev` es almacenamiento de laboratorio. La guía de refuerzo de producción de HashiCorp señala TLS, usuarios de ejecución sin privilegios, decisiones de bloqueo de memoria o intercambio cifrado, separación de almacenamiento y rutas de red restringidas.

Operación con varios inquilinos

El modelo de inquilinos de Cloud Agent separa las acciones de administración de las del inquilino. El administrador gestiona billeteras, entidades, claves de API y permisos opcionales de Keycloak. El inquilino usa Cloud Agent para acciones SSI dentro de su billetera asignada.

Usa estas variables de Cloud Agent para el modo básico de varios inquilinos:

```
ADMIN_TOKEN=replace-with-admin-token
API_KEY_ENABLED=true
API_KEY_AUTO_PROVISIONING=false
API_KEY_AUTHENTICATE_AS_DEFAULT_USER=false
DEFAULT_WALLET_ENABLED=false
```

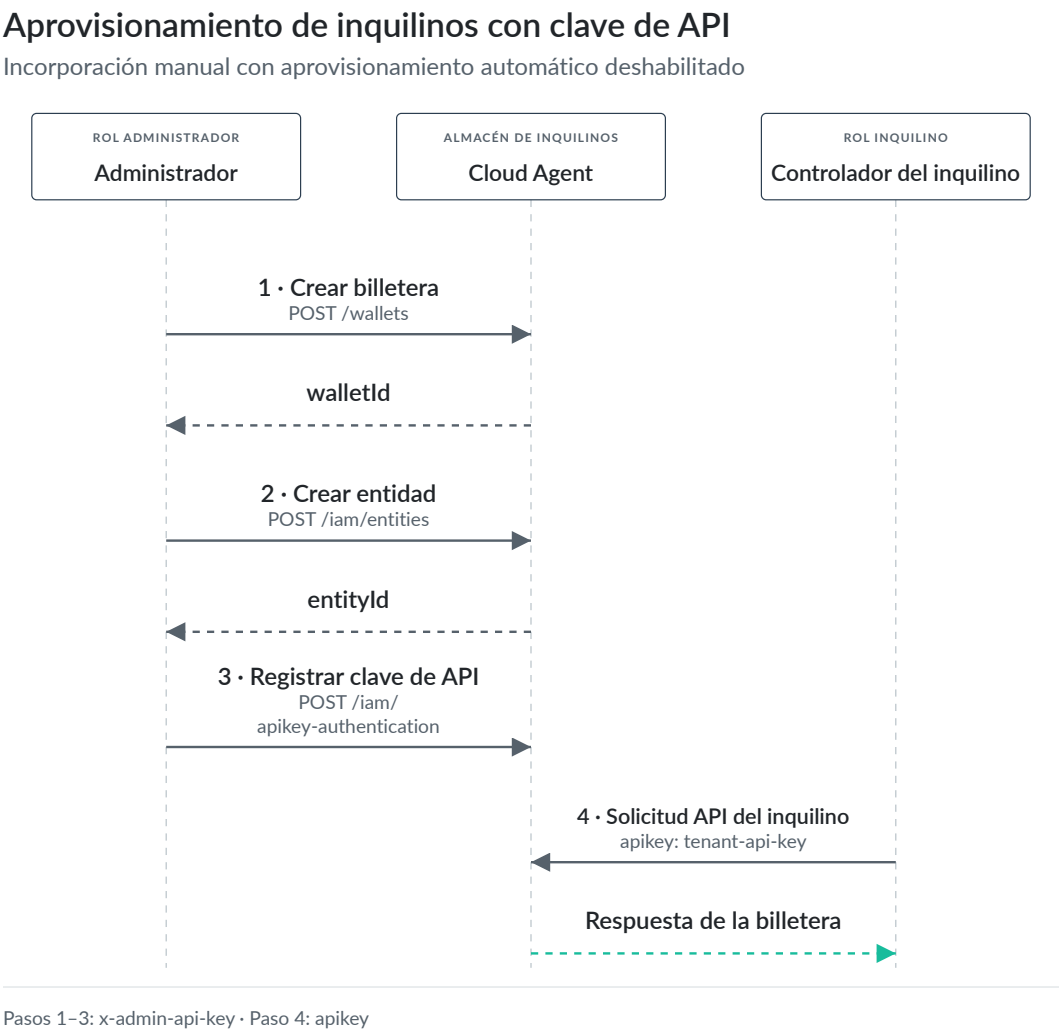


Figura 3: Con API_KEY_AUTO_PROVISIONING=false, el administrador crea la billetera, la entidad y la clave de API. La clave de API del inquilino selecciona la billetera asignada a la entidad.

Con estos ajustes, Cloud Agent comienza con un conjunto vacío de billeteras de inquilinos. El administrador crea una billetera, crea una entidad vinculada a ella y registra una clave de API para esa entidad:

```
curl -X POST "https://agent.example.test/cloud-agent/wallets" \
  -H "x-admin-api-key: $ADMIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "issuer-tenant-a"
  }'
```

```
curl -X POST "https://agent.example.test/cloud-agent/iam/entities" \
  -H "x-admin-api-key: $ADMIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "tenant-a",
    "walletId": "replace-with-wallet-id"
  }'
```

```
curl -X POST
 ↪  "https://agent.example.test/cloud-agent/iam/apikey-authentication" \
  -H "x-admin-api-key: $ADMIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "entityId": "replace-with-entity-id",
    "apiKey": "replace-with-tenant-api-key"
  }'
```

El inquilino usa la clave de API asignada en la cabecera `apikey`:

```
curl "https://agent.example.test/cloud-agent/did-registrar/dids" \
  -H "apikey: replace-with-tenant-api-key" \
  -H "Accept: application/json"
```

La respuesta se limita a la billetera del inquilino. Un inquilino que emite credenciales desde una billetera obtiene acceso a los DIDs, credenciales, conexiones y registros de verificación de ese inquilino.

IAM externo con Keycloak

Usa Keycloak cuando los inquilinos deban autenticarse mediante OIDC y recibir permisos de billetera mediante UMA. Esto sustituye las claves estáticas de API de Cloud Agent por tokens de Keycloak para el acceso de inquilinos. En este modelo, Cloud Agent aún administra los recursos de billetera. Keycloak administra la autenticación de usuarios y los tokens de autorización.

Configura Keycloak con un cliente dedicado para Cloud Agent y habilita los servicios de autorización necesarios para UMA. Después cambia Cloud Agent a Keycloak:

```
KEYCLOAK_ENABLED=true
KEYCLOAK_URL=https://iam.example.test
KEYCLOAK_REALM=identus
KEYCLOAK_CLIENT_ID=cloud-agent
KEYCLOAK_CLIENT_SECRET=replace-with-client-secret
KEYCLOAK_UMA_AUTO_UPGRADE_RPT=false
```

En el flujo Keycloak, el administrador crea una billetera en Cloud Agent, registra un usuario en Keycloak y concede a ese sujeto de Keycloak permisos sobre la billetera:

```
curl -X POST "https://agent.example.test/cloud-agent/wallets/replace-with-
↪  wallet-id/uma-permissions" \
  -H "x-admin-api-key: $ADMIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "subject": "replace-with-keycloak-user-id"
  }'
```

El inquilino obtiene un token de acceso OIDC de Keycloak, solicita un token de parte solicitante UMA con `grant_type=urn:ietf:params:oauth:grant-type:uma-ticket` y llama a Cloud Agent con:

```
Authorization: Bearer replace-with-rpt
```

Si `KEYCLOAK_UMA_AUTO_UPGRADE_RPT=true`, Cloud Agent puede aceptar el token de acceso del inquilino y ejecutar la conversión a RPT que describe su guía de IAM externo. Mantén `false` hasta que la aplicación controladora administre el intercambio de tokens y el tratamiento de errores.

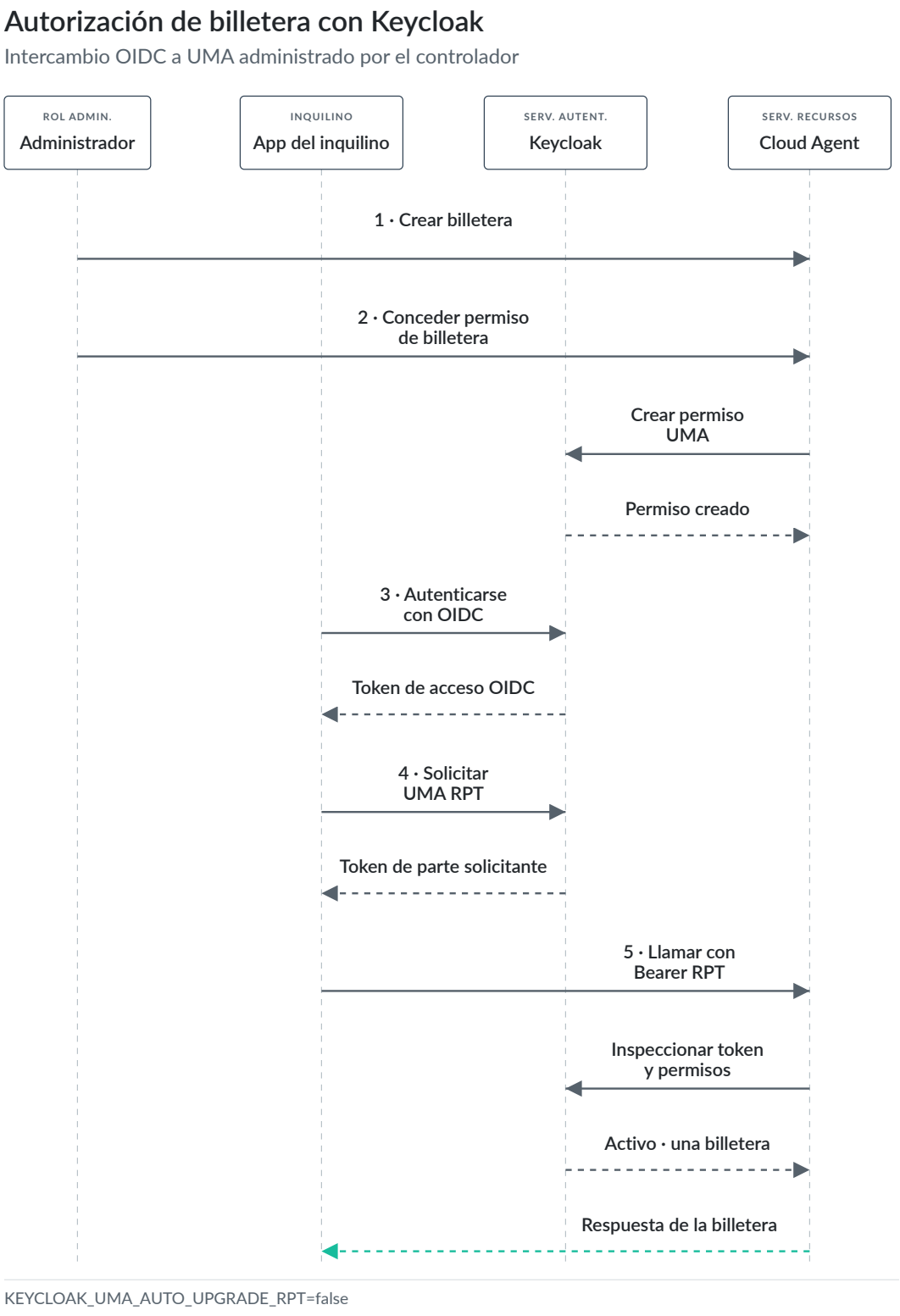


Figura 4: El administrador concede a un sujeto Keycloak acceso a una billetera de Cloud Agent. Con la conversión automática a RPT deshabilitada, el controlador del inquilino obtiene el RPT. El agente acepta una billetera autorizada por solicitud del inquilino.

Operaciones de Cardano Wallet

PRISM Node en modo Cardano tiene tres dependencias externas: su propia base de datos PostgreSQL, un backend Cardano Wallet y Cardano DB Sync. El archivo Compose local de Identus ya crea la base de datos del nodo mediante el servicio PostgreSQL compartido. Añade Cardano Wallet y un nodo Cardano a `infrastructure/shared/docker-compose-preprod.yml`.

Usa variables separadas para el selector de red de PRISM Node y el directorio de configuración de servicios Cardano:

```
NODE_CARDANO_NETWORK=testnet
CARDANO_NETWORK=preprod
```

PRISM Node acepta `testnet` y `mainnet` para `NODE_CARDANO_NETWORK`. Cardano Wallet, el nodo Cardano y DB Sync usan los nombres de configuración `preprod`, `preview` o `mainnet`. Este tutorial usa Cardano `preprod`, que sigue siendo un despliegue `testnet` de PRISM Node.

```
  cardano-node:
    image: cardanofoundation/cardano-wallet:${CARDANO_WALLET_TAG}
    environment:
      CARDANO_NODE_SOCKET_PATH: /ipc/${NODE_SOCKET_NAME}
    volumes:
      - ${NODE_DB}:/data
      - ${NODE_SOCKET_DIR}:/ipc
      - ${NODE_CONFIGS}:/configs
    entrypoint: []
    command: >
      cardano-node run
        --topology /configs/cardano/${CARDANO_NETWORK}/topology.json
        --database-path /data
        --socket-path /ipc/${NODE_SOCKET_NAME}
        --config /configs/cardano/${CARDANO_NETWORK}/config.json
        +RTS -N -A16m -qg -qb -RTS
    restart: on-failure
    logging:
      driver: "json-file"
      options:
        compress: "true"
        max-file: "10"
        max-size: "50m"

  cardano-wallet:
    image: cardanofoundation/cardano-wallet:${CARDANO_WALLET_TAG}
    volumes:
```

```
      - ${WALLET_DB}:/wallet-db
      - ${NODE_SOCKET_DIR}:/ipc
      - ${NODE_CONFIGS}:/configs
    ports:
      - "${WALLET_PORT}:8090"
    entrypoint: []
    command: >
      cardano-wallet serve
        --node-socket /ipc/${NODE_SOCKET_NAME}
        --database /wallet-db
        --listen-address 0.0.0.0
        --testnet /configs/cardano/${CARDANO_NETWORK}/byron-genesis.json
    depends_on:
      - cardano-node
    restart: on-failure
    logging:
      driver: "json-file"
      options:
        compress: "true"
        max-file: "10"
        max-size: "50m"
```

Para `mainnet`, establece ambas variables de red de producción y cambia el comando de billetera:

```
NODE_CARDANO_NETWORK=mainnet
CARDANO_NETWORK=mainnet
```

Cambia:

```
--testnet /configs/cardano/${CARDANO_NETWORK}/byron-genesis.json
```

por:

```
--mainnet
```

Vincula la API de Cardano Wallet a `127.0.0.1`, una red privada de administración o una red interna de servicios.

Crea o restaura la billetera que usa PRISM Node después de que Cardano Wallet informe de un estado de red correcto:

```
curl http://127.0.0.1:8090/v2/network/information
```

Genera la frase mnemónica mediante tu proceso de custodia o la CLI de Cardano Wallet en una máquina segura del operador:

```
cardano-wallet mnemonic generate --size 24
```

Restaura la billetera mediante la API local de Cardano Wallet. Sustituye cada marcador `word-*` antes de ejecutar el comando:

```
curl -X POST "http://127.0.0.1:8090/v2/wallets" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "prism-node-preprod",
    "mnemonic_sentence": [
      "word-01", "word-02", "word-03", "word-04", "word-05", "word-06",
      "word-07", "word-08", "word-09", "word-10", "word-11", "word-12",
      "word-13", "word-14", "word-15", "word-16", "word-17", "word-18",
      "word-19", "word-20", "word-21", "word-22", "word-23", "word-24"
    ],
    "passphrase": "replace-with-wallet-passphrase",
    "address_pool_gap": 20
  }'
```

Registra el `id` devuelto de la billetera como `NODE_CARDANO_WALLET_ID`. Almacena la frase mnemónica y la contraseña de gasto en el sistema de secretos de producción. Mantén la frase mnemónica fuera de `.env-preprod`, los registros de CI, el historial de la terminal y el repositorio.

Solicita a Cardano Wallet una dirección sin usar y añádele fondos:

```
curl "http://127.0.0.1:8090/v2/wallets/${NODE_CARDANO_WALLET_ID}/addresses
↪  ?state=unused"
```

Establece la primera dirección sin usar como `NODE_CARDANO_PAYMENT_ADDRESS` después de añadirle fondos. En `preprod`, usa el faucet de Cardano. En `mainnet`, transfiere ADA real mediante el proceso de tesorería asignado al operador del emisor o verificador. PRISM Node necesita suficiente ADA para las comisiones de transacción y la salida de 1 ADA que describe su guía de despliegue.

Añade las variables Cardano documentadas de PRISM Node al servicio existente `prism-node`:

```
  prism-node:
    image: ghcr.io/input-output-hk/prism-node:${PRISM_NODE_VERSION}
    environment:
      NODE_PSQL_HOST: db:5432
      NODE_PSQL_DATABASE: node_db
      NODE_PSQL_USERNAME: ${AGENT_DB_USER:-postgres}
      NODE_PSQL_PASSWORD: ${AGENT_DB_PASSWORD}
      NODE_LEDGER: ${NODE_LEDGER:-cardano}
      NODE_CARDANO_NETWORK: ${NODE_CARDANO_NETWORK:-testnet}
      NODE_CARDANO_WALLET_ID: ${NODE_CARDANO_WALLET_ID}
      NODE_CARDANO_WALLET_PASSPHRASE: ${NODE_CARDANO_WALLET_PASSPHRASE}
      NODE_CARDANO_PAYMENT_ADDRESS: ${NODE_CARDANO_PAYMENT_ADDRESS}
      NODE_CARDANO_WALLET_API_HOST:
↪   ${NODE_CARDANO_WALLET_API_HOST:-cardano-wallet}
      NODE_CARDANO_WALLET_API_PORT: ${NODE_CARDANO_WALLET_API_PORT:-8090}
      NODE_CARDANO_DB_SYNC_HOST: ${NODE_CARDANO_DB_SYNC_HOST}
      NODE_CARDANO_DB_SYNC_DATABASE:
↪   ${NODE_CARDANO_DB_SYNC_DATABASE:-cexplorer}
      NODE_CARDANO_DB_SYNC_USERNAME: ${NODE_CARDANO_DB_SYNC_USERNAME}
      NODE_CARDANO_DB_SYNC_PASSWORD: ${NODE_CARDANO_DB_SYNC_PASSWORD}
```

La guía de despliegue de PRISM Node indica que `NODE_CARDANO_NETWORK` establece su selector de red. Wallet y DB Sync usan su propia configuración de red. El nodo Cardano, Cardano Wallet, DB Sync y PRISM Node deben apuntar a la misma red Cardano. Una configuración mezclada puede iniciar los procesos y aún fallar al publicar o resolver operaciones DID. En este tutorial, `NODE_CARDANO_NETWORK=testnet` con `CARDANO_NETWORK=preprod` es coherente: PRISM Node usa modo testnet y los servicios Cardano usan archivos de la red de pruebas de preproducción.

Operaciones de Cardano DB Sync

DB Sync requiere su propia base de datos PostgreSQL y una conexión al mismo nodo Cardano. Usa el paquete de la versión actual de DB Sync, sus notas de versión o tu módulo de infraestructura para la definición exacta del servicio. El límite del servicio debe satisfacer estas variables de PRISM Node:

```
NODE_CARDANO_DB_SYNC_HOST=cardano-db-sync-postgres:5432
NODE_CARDANO_DB_SYNC_DATABASE=cexplorer
NODE_CARDANO_DB_SYNC_USERNAME=postgres
NODE_CARDANO_DB_SYNC_PASSWORD=replace-with-db-sync-password
```

Un laboratorio basado en Compose normalmente contiene:

```
  cardano-db-sync-postgres:
    image: postgres:14
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${NODE_CARDANO_DB_SYNC_PASSWORD}
      POSTGRES_DB: ${NODE_CARDANO_DB_SYNC_DATABASE:-cexplorer}
    volumes:
      - cardano_db_sync_postgres:/var/lib/postgresql/data

  cardano-db-sync:
    image: ghcr.io/intersectmbo/cardano-db-sync:${CARDANO_DB_SYNC_VERSION}
    depends_on:
      - cardano-node
      - cardano-db-sync-postgres
    volumes:
      - ${NODE_SOCKET_DIR}:/ipc
      - ${NODE_CONFIGS}:/configs
    environment:
      CARDANO_NODE_SOCKET_PATH: /ipc/${NODE_SOCKET_NAME}
      NETWORK: ${CARDANO_NETWORK}
      POSTGRES_HOST: cardano-db-sync-postgres
      POSTGRES_PORT: 5432
      POSTGRES_DB: ${NODE_CARDANO_DB_SYNC_DATABASE:-cexplorer}
      POSTGRES_USER: ${NODE_CARDANO_DB_SYNC_USERNAME:-postgres}
      POSTGRES_PASSWORD: ${NODE_CARDANO_DB_SYNC_PASSWORD}
```

Las opciones de comandos y los pasos de restauración de instantáneas de DB Sync cambian entre versiones. Fija la imagen, lee sus notas de versión y valida la sincronización en `preprod` antes de cambiar a `mainnet`. Para `mainnet`, restaura desde una instantánea de confianza o planifica una sincronización inicial larga.

Añade el volumen de la base de datos DB Sync:

```
volumes:
  cardano_db_sync_postgres:
```

Ejecuta DB Sync contra el mismo socket de nodo que usa Cardano Wallet. DB Sync publica el progreso de sincronización en sus registros y en la base de datos `cexplorer`. Antes de comprobar la publicación `did:prism`, confirma que DB Sync está cerca del último bloque de la cadena. Después crea, actualiza y resuelve un DID mediante Cloud Agent.

Para actualizar DB Sync, lee las notas de versión antes de cambiar la etiqueta de imagen. Las notas de 13.7.1.0 indican que actualizar desde 13.7.0.x ejecuta una migración de la tabla `epoch` y que existen instantáneas mainnet compatibles de 13.7 y 13.6. Trata la compatibilidad de instantáneas como específica de cada versión.

Selección de red preprod y mainnet

Usa `preprod` para el primer despliegue con características de producción. La documentación Cardano describe preproducción como la red de pruebas madura que se parece a mainnet para probar versiones. Usa ADA de prueba, por lo que los equipos de emisores y verificadores pueden comprobar publicación de DIDs, emisión, revocación y verificación de credenciales antes de transferir fondos reales.

Para preprod:

```
NODE_CARDANO_NETWORK=testnet
CARDANO_NETWORK=preprod
```

Para mainnet:

```
NODE_CARDANO_NETWORK=mainnet
CARDANO_NETWORK=mainnet
```

Mainnet cambia el perfil de riesgo. El operador debe financiar la billetera de PRISM Node con ADA real, proteger la frase mnemónica y contraseña de Cardano Wallet, supervisar el saldo y aprobar el gasto en comisiones. Ejecuta al menos un flujo completo del emisor en preprod antes de mainnet: crea un `did:prism` de emisor, emite una credencial a una billetera de titular, revócala o suspéndela si tu caso usa listas de estado, preséntala al software del verificador y confirma que este separa las comprobaciones criptográficas de las comprobaciones del emisor y de la política de confianza.

Copiar la configuración de red Cardano

El repositorio Cardano Wallet almacena la configuración de red bajo `configs/cardano`. Copia esos archivos al directorio de trabajo del proyecto que usa el volumen Compose:

```
mkdir -p ./cardano
git clone --depth 1 https://github.com/cardano-foundation/cardano-wallet
 ↪  ./cardano/cardano-wallet
cp -R ./cardano/cardano-wallet/configs ./cardano/configs
```

Ejecuta el script de actualización desde `configs/cardano`. El script entra en cada directorio de red y ejecuta su `download.sh`:

```
(
  cd ./cardano/configs/cardano
  ./refresh.sh
)
```

Confirma que el directorio `preprod` contiene los archivos de red necesarios para `cardano-node` y `cardano-wallet`:

```
ls ./cardano/configs/cardano/preprod
```

Los archivos esperados incluyen:

```
alonzo-genesis.json
byron-genesis.json
config.json
conway-genesis.json
download.sh
shelley-genesis.json
topology.json
```

Crea los directorios de datos que usa el archivo Compose:

```
mkdir -p ./cardano/node-db ./cardano/wallet-db ./cardano/ipc
```

Iniciar preprod

Valida la configuración Compose antes de iniciar servicios:

```
docker compose \
  -p identus-preprod \
  -f ./infrastructure/shared/docker-compose-preprod.yml \
  --env-file ./infrastructure/preprod/.env-preprod \
  config
```

Inicia el entorno:

```
./infrastructure/preprod/run.sh \
  -n preprod \
  -b \
  -w \
  -e ./infrastructure/preprod/.env-preprod \
  -p 8000 \
  -d agent.example.test
```

Comprueba el punto de acceso de estado de Cloud Agent mediante la misma URL base pública configurada en `REST_SERVICE_URL`:

```
curl https://agent.example.test/cloud-agent/_system/health
```

Para un laboratorio antes de terminar TLS, usa el puerto del host de APISIX:

```
curl http://localhost:8000/cloud-agent/_system/health
```

Comprueba el estado de red de Cardano Wallet:

```
curl http://127.0.0.1:8090/v2/network/information
```

Crea o restaura la billetera Cardano de PRISM Node, añádele fondos en `preprod` y actualiza estos valores en `.env-preprod`:

```
NODE_CARDANO_WALLET_ID=replace-after-wallet-create
NODE_CARDANO_WALLET_PASSPHRASE=replace-after-wallet-create
NODE_CARDANO_PAYMENT_ADDRESS=replace-after-wallet-create
```

Reinicia PRISM Node después de cambiar esos valores.

Comprobaciones de producción

Usa estas comprobaciones antes de que un emisor, verificador, billetera de titular o aplicación controladora dependa del despliegue:

- El punto de acceso de estado de Cloud Agent devuelve la versión esperada y la `REST_SERVICE_URL` pública usa HTTPS.
- Las billeteras de titulares y los servicios de mediador pueden acceder a `DIDCOMM_SERVICE_URL`.
- La autenticación por clave de API está habilitada. `ADMIN_TOKEN` y `API_KEY_SALT` son valores largos y aleatorios almacenados en el sistema de secretos de producción.
- `DEFAULT_WALLET_ENABLED=false` para despliegues de varios inquilinos.
- `SECRET_STORAGE_BACKEND=vault` para producción. Vault usa configuración de producción y la copia de seguridad de semillas de billetera tiene un procedimiento del operador.
- Los puertos PostgreSQL permanecen en redes privadas. Las bases de datos de Cloud Agent, PRISM Node y DB Sync tienen copias de seguridad y pruebas de restauración.
- PRISM Node usa `NODE_CARDANO_NETWORK=testnet` para preprod y `mainnet` para mainnet. El nodo Cardano, Cardano Wallet y DB Sync usan el valor correspondiente de `CARDANO_NETWORK`, `preprod` o `mainnet`.
- Cardano Wallet tiene suficiente ADA para transacciones de PRISM Node. En `preprod`, usa un faucet. En `mainnet`, supervisa el saldo y las comisiones de transacción.

- DB Sync está cerca del último bloque de la cadena antes de comprobar publicaciones de PRISM Node.
- Una operación de creación `did:prism` alcanza el estado confirmado y después un resolvedor puede resolver el DID de forma corta.
- El software del verificador separa las comprobaciones técnicas de credenciales de las decisiones sobre emisores, esquemas, registros de confianza y política de negocio.

Mantén los archivos Docker de ejemplo bajo control de versiones para el aprendizaje de desarrolladores y trabajo de laboratorio reproducible. Para mainnet, lleva secretos, almacenamiento persistente, redes, TLS, supervisión, copias de seguridad y política de despliegue al sistema que tu equipo usa para operar en producción.

Mantenimiento

Dominar Identus implica mantener la aplicación después de lanzarla. El despliegue de producción del capítulo no es un solo proceso. Es un conjunto de servicios de larga ejecución: Cloud Agent, PRISM Node o NeoPRISM, HashiCorp Vault, PostgreSQL y, en modo Cardano, un nodo Cardano, Cardano Wallet y Cardano DB Sync. Cada uno tiene su propio ciclo de vida, almacenamiento y modos de fallo. Este capítulo cubre el trabajo posterior al lanzamiento para mantener el entorno en buen estado: reiniciarlo correctamente, observarlo, actualizarlo sin romper la compatibilidad y proteger sus secretos y claves.

Este capítulo presupone el despliegue descrito en el capítulo de instalación de producción. Las indicaciones son concretas donde ese despliegue las sustenta. Donde las prácticas operativas de Identus aún son inmaduras o no tienen documentación, la sección incluye un marcador para ampliarla en un borrador posterior.

Reinicio y limpieza

Los servicios de un despliegue en modo Cardano tienen un orden de dependencias que los reinicios deben respetar. Cardano Wallet y Cardano DB Sync se conectan al nodo Cardano mediante su socket. PRISM Node depende de Cardano Wallet y DB Sync, y Cloud Agent depende de PRISM Node y PostgreSQL. Un reinicio correcto normalmente avanza desde el registro distribuido hacia la aplicación:

1. Inicia o comprueba PostgreSQL y abre Vault. Cloud Agent no puede leer semillas de billetera mientras Vault está sellado.
2. Inicia `cardano-node` y después `cardano-wallet` y `cardano-db-sync` cuando el socket del nodo esté disponible.
3. Espera a que DB Sync se acerque al último bloque de la cadena antes de depender de la publicación o resolución `did:prism`.
4. Inicia PRISM Node, después Cloud Agent y después la aplicación controladora.

Un reinicio no vuelve a anclar los DIDs existentes. Cloud Agent reconstruye el material de claves DID a partir de la semilla de billetera y la ruta de derivación almacenada. Por ello, disponer de la semilla —con Vault abierto y accesible— es el requisito previo para los flujos de emisión y actualización DID tras reiniciar.

La limpieza trata principalmente del disco y los registros. Los servicios Cardano generan registros continuos durante la sincronización. Mantén configurada la rotación para todos los contenedores de larga ejecución. La base de datos PostgreSQL de DB Sync crece continuamente y consume más disco que el resto del entorno. A diferencia de las bases de datos de Cloud Agent y PRISM Node, los datos DB Sync son estado derivado de la cadena:

pueden reconstruirse mediante una nueva sincronización o restaurarse desde una instantánea de confianza. Por ello pueden depurarse o reconstruirse en lugar de mantener copias de seguridad a largo plazo.

> TODO: Añadir procedimientos concretos de limpieza: eliminar imágenes y volúmenes Docker, recuperar disco de DB Sync y decidir entre nueva sincronización e instantánea con tiempos medidos en `preprod` y `mainnet`.

Observabilidad

Operar bien Identus requiere observar la capa de aplicación —Cloud Agent y controlador— y la capa del registro distribuido —servicios Cardano—. Un problema en cualquiera de ellas puede detener la emisión de credenciales mientras Cloud Agent sigue «activo».

Comprobaciones de estado y servicios

La lista de comprobaciones de producción del capítulo es el punto de partida para la supervisión continua. Las comprobaciones que conviene ejecutar continuamente, además del lanzamiento, incluyen:

- El estado de Cloud Agent devuelve la versión esperada y la `REST_SERVICE_URL` pública responde mediante HTTPS.
- Las billeteras de titulares y servicios de mediador pueden acceder a `DIDCOMM_SERVICE_URL`.
- Cardano DB Sync está cerca del último bloque de la cadena. Si se retrasa, demora silenciosamente la publicación y resolución de DIDs.
- Cardano Wallet informa de un estado de red correcto (`/v2/network/information`) y, en mainnet, la dirección de pago de PRISM Node tiene suficiente ADA para comisiones.
- Una comprobación de extremo a extremo: una operación de creación `did:prism` alcanza el estado confirmado y después se resuelve en forma corta.

Administración de nodos y memoria

Cardano DB Sync consume más recursos que el resto del entorno y sus requisitos dominan el dimensionamiento. El capítulo de producción registra cifras mainnet del orden de 64 GB de RAM, 4 o más núcleos de CPU, SSD con 60 000 IOPS o más y varios cientos de GB de disco que crecen con el tiempo. `preprod` es mucho menor. Ejecuta DB Sync y su PostgreSQL cerca uno del otro, idealmente juntos, para reducir la latencia de sincronización. Supervisa las IOPS si separas servicios entre máquinas.

Durante la operación, observa:

- El crecimiento de disco PostgreSQL de DB Sync y el espacio restante.
- La memoria de `cardano-node` y `cardano-db-sync` frente a los límites del host.
- El tamaño y la latencia de consultas de las bases de datos de Cloud Agent y PRISM Node, que afectan al rendimiento de emisión y verificación.

Pruebas de rendimiento

TODO: Marcador de posición. El libro aún no define una metodología de pruebas de rendimiento para Identus. Ampliar esta sección con una prueba de carga repetible —rendimiento de emisión, verificación y resolución DID—, métricas, valores objetivo y cómo la profundidad de confirmación Cardano limita la latencia de emisión de extremo a extremo.

Análisis con BlockTrust Analytics

TODO: Marcador de posición. BlockTrust Analytics es una herramienta de análisis de terceros para Identus/PRISM. Ampliar esta sección con lo que observa, cómo conectarla a un despliegue en ejecución y qué preguntas operativas responde que las comprobaciones de estado básicas no responden.

Actualización de agentes

Selección de versiones y compatibilidad

Trata las notas de versiones de Identus Platform como la primera fuente de compatibilidad. Una versión de plataforma fija componentes que se sabe que funcionan juntos: por ejemplo, versiones de Cloud Agent, Mediator, NeoPRISM y SDK. Pueden existir versiones independientes más recientes, como una nueva imagen de Cloud Agent o PRISM Node, pero requieren comprobar la compatibilidad con la versión fijada de plataforma. Los pares de imágenes Cloud Agent y PRISM Node sin comprobar pueden fallar en las comprobaciones de compatibilidad.

En la práctica, las actualizaciones cambian una o más de estas versiones fijas:

- `AGENT_VERSION` (Cloud Agent).
- `PRISM_NODE_VERSION` (PRISM Node) o la versión del backend NeoPRISM.
- `CARDANO_WALLET_TAG` y su versión asociada de `cardano-node`.
- `CARDANO_DB_SYNC_VERSION`.

Cuando cambies cualquier imagen Identus, vuelve a ejecutar la lista completa de comprobaciones de producción antes de depender del despliegue.

Procedimiento de actualización y notas de compatibilidad

Las actualizaciones de Cardano DB Sync necesitan especial atención: lee las notas de versión antes de cambiar la etiqueta de imagen. Pueden ejecutar migraciones de esquemas, como una migración de la tabla `epoch` entre versiones secundarias, y la compatibilidad de instantáneas depende de cada versión. Valida una actualización DB Sync en `preprod` antes de aplicarla a `mainnet`. Confirma que la nueva versión aún llega al último bloque de la cadena y que PRISM Node puede publicar y resolver un DID de prueba después.

Para Cloud Agent y PRISM Node, confirma que el par es una combinación probada. Después actualiza y ejecuta un flujo completo del emisor —crear DID, emitir, revocar o suspender si corresponde, presentar y verificar— antes de cambiar el tráfico.

Reducir el tiempo de inactividad

> TODO: Marcador de posición. Documentar una estrategia de actualización con poco tiempo de inactividad. Resolver antes estas preguntas: si Cloud Agent permite ejecutar versiones antiguas y nuevas en paralelo con una base de datos compartida, cómo aplica las migraciones durante una actualización y cómo terminar los flujos DIDComm y de emisión en curso. Hasta entonces, planificar una ventana de mantenimiento.

HashiCorp Vault y administración de claves

Administración de claves

En producción, Cloud Agent almacena material de semillas de billetera en Vault (`SECRET_STORAGE_BACKEND=vault`), bajo rutas específicas como `/secret/<wallet-id>/seed`, junto con rutas de claves de DIDs de pares y otros secretos. La semilla de billetera del inquilino permite volver a derivar claves DID después de reiniciar o desplegar de nuevo. Perderla puede inutilizar DIDs existentes para actualizaciones, desactivación o emisión futuras. Por tanto, Vault es el componente más importante que debes proteger y respaldar en el despliegue.

Aspectos operativos esenciales del capítulo de producción:

- Usa AppRole (`VAULT_APPROLE_ROLE_ID` / `VAULT_APPROLE_SECRET_ID`) para despliegues automáticos. Reserva `VAULT_TOKEN` para laboratorio o emergencias.
- Ejecuta Vault con refuerzo de producción: TLS, usuario de ejecución sin privilegios, una decisión explícita sobre bloqueo de memoria o intercambio cifrado, separación de almacenamiento y rutas de red restringidas. Vault con `server -dev` es solo almacenamiento de laboratorio.
- Crea copias de seguridad de Vault, comprueba la restauración y documenta el procedimiento de apertura o claves de recuperación. Tras cualquier reinicio de Vault, debes abrir el almacén antes de que Cloud Agent lea semillas.

Copias de seguridad y restauración

Crea copias de seguridad según el tipo de datos, porque los servicios requieren tratamientos distintos:

- **Las bases de datos de Cloud Agent y PRISM Node** contienen estado de aplicación. Crea copias de seguridad y comprueba las restauraciones.

- **El almacenamiento Vault** contiene semillas de billetera y material de claves. Crea copias de seguridad, comprueba la restauración y protege las claves de apertura y recuperación.
- **La frase mnemónica y contraseña de gasto de Cardano Wallet**, junto con la dirección de pago de PRISM Node, pertenecen al sistema de secretos de producción. Nunca deben estar en archivos `.env`, registros de CI, historial de la terminal o el repositorio.
- **La base de datos Cardano DB Sync** contiene estado derivado de la cadena. Puede reconstruirse mediante sincronización o restaurarse desde una instantánea de confianza. Su copia de seguridad tiene menor prioridad que el estado de aplicación.

Rotación de claves y credenciales

La rotación abarca dos tipos de secretos. Las credenciales operativas —`ADMIN_TOKEN`, claves de API de inquilinos, contraseñas de bases de datos, credenciales de Vault, secretos de cliente Keycloak y contraseña de Cardano Wallet— protegen el acceso a los servicios. Se rotan mediante el mismo proceso que el resto de secretos de producción. Las claves DID son distintas: son material a nivel de protocolo del que dependen los verificadores, y rotarlas cambia lo que otras partes pueden verificar. Los procedimientos siguientes se basan en Cloud Agent 2.2.0. Comprueba las notas de la versión al actualizar, porque el comportamiento de selección de claves aún evoluciona en el agente.

Credenciales operativas. La mayoría se rotan con un reinicio, pero debes conocer algunos comportamientos antes de empezar:

Credencial	Procedimiento	Efecto
`ADMIN_TOKEN`	Genera un valor nuevo, actualiza el almacén de secretos y reinicia Cloud Agent.	El agente acepta un solo token de administrador. La automatización administrativa falla hasta que use el nuevo valor; las llamadas de API de los inquilinos no se ven afectadas.
Claves de API de inquilinos	Registra una clave nueva con `POST /iam/apikey-authentication`, cambia la aplicación del inquilino para que la use y después elimina el registro de la clave antigua con `DELETE /iam/apikey-authentication` y el mismo cuerpo `entityId`/`apiKey`.	Ninguno si sigues ese orden: una entidad puede tener varias claves de API a la vez.

Credencial	Procedimiento	Efecto
`API_KEY_SALT`	No lo rotes como mantenimiento rutinario.	Cloud Agent almacena las claves de API como hashes con sal. Cambiar la sal invalida todas las claves de API registradas a la vez. Trátalo como un nuevo aprovisionamiento de todos los inquilinos.
`secret_id` de AppRole de Vault	Consulta el procedimiento siguiente.	Ninguno si el `secret_id` antiguo sigue siendo válido hasta que todas las instancias del agente se hayan reiniciado.

Genera siempre un valor aleatorio nuevo para una clave de API de reemplazo. Cloud Agent conserva un registro de las claves cuyo registro se ha eliminado y rechaza registrar el mismo valor de nuevo. Si alguien presenta una clave ya registrada para una entidad *distinta*, el agente la considera comprometida y la desactiva.

Rotar el `secret_id` de AppRole de Vault. Cloud Agent inicia sesión en Vault con `VAULT_APPROLE_ROLE_ID` y `VAULT_APPROLE_SECRET_ID` al arrancar, y vuelve a iniciar sesión con el mismo par antes de que caduque cada concesión de token. Revocar el `secret_id` antiguo mientras un agente aún lo usa no causa un fallo inmediato. El fallo ocurre en el siguiente inicio de sesión, cuando el agente ya no puede leer las semillas de billetera. Rota en este orden:

1. Genera un `secret_id` nuevo para el rol y registra su accessor:

   ```
   vault write -f auth/approle/role/cloud-agent/secret-id
   ```

2. Actualiza `VAULT_APPROLE_SECRET_ID` en el almacén de secretos de producción y reinicia o sustituye de forma gradual todas las instancias de Cloud Agent.
3. Confirma que cada instancia está en buen estado y puede acceder al material de billetera, por ejemplo al enumerar los DIDs de un inquilino.
4. Destruye el `secret_id` antiguo mediante su accessor:

   ```
   vault write auth/approle/role/cloud-agent/secret-id-accessor/destroy \
     secret_id_accessor=replace-with-old-accessor
   ```

Si el rol establece `secret_id_ttl` o `secret_id_num_uses`, recuerda que los inicios de sesión periódicos del agente los consumen. Un `secret_id` que caduca o agota sus usos mientras el agente está en ejecución tiene el mismo efecto que revocarlo.

Rotar claves `did:prism`. Un controlador `did:prism` rota claves de verificación al publicar una operación DID firmada de **actualización** mediante PRISM Node. Esto cambia los

métodos y relaciones de verificación del documento DID en la cadena, como explica el capítulo . En Cloud Agent, la operación es `POST /did-registrar/dids/{didRef}/updates`, con acciones `ADD_KEY` y `REMOVE_KEY`. El agente deriva la clave nueva de la semilla de billetera y registra su ruta de derivación.

La restricción principal afecta al verificador. Cuando Cloud Agent verifica una credencial JWT, resuelve el documento DID *actual* del emisor y comprueba la firma con las claves incluidas en `assertionMethod`. Por tanto, eliminar una clave impide verificar todas las credenciales que firmó esa clave, no solo las futuras. Rota una clave de emisión con un período de solapamiento:

1. **Añade la clave nueva.** Envía una actualización que añada una clave con el mismo propósito y curva que la clave que sustituyes:

```
curl -X POST "https://agent.example.test/cloud-agent/did-registrar/di
↪  ds/$ISSUER_DID/updates" \
  -H "apikey: replace-with-tenant-api-key" \
  -H "Content-Type: application/json" \
  -d '{
    "actions": [
      {
        "actionType": "ADD_KEY",
        "addKey": { "id": "assertion-2", "purpose": "assertionMethod",
↪  "curve": "secp256k1" }
      }
    ]
  }'
```

   El agente acepta una sola actualización pendiente por DID. Espera hasta que la operación se confirme en la cadena y el DID se resuelva con la clave nueva antes de enviar otra actualización.

2. **Indica la clave de firma en cada solicitud de emisión.** Mientras el DID tenga dos claves `assertionMethod`, Cloud Agent rechaza elegir una: falla la emisión cuando firma una credencial JWT para una oferta que no estableció un ID de clave de emisión. La búsqueda de la clave ocurre al firmar, por lo que esto también afecta a ofertas creadas antes de añadir la clave nueva que aún esperan la solicitud del titular. Establece `issuingKid` en `jwtVcPropertiesV1`, por ejemplo `"issuingKid": "assertion-2"`, en cada oferta. Actualiza la aplicación del emisor *antes* de que se confirme el paso 1 y deja que las ofertas en curso terminen primero. En esta versión, la emisión OID4VCI no acepta un ID de clave, por lo que falla mientras el DID tenga más de una clave `assertionMethod`. Si la usas, planifica el solapamiento con esta restricción.

3. **Vuelve a emitir las credenciales** que deban seguir siendo verificables después de eliminar la clave antigua, y fírmalas con la nueva. Puedes conservar sin cambios las credenciales que caduquen o se revoquen antes de la transición.

4. **Elimina la clave antigua** mediante una acción `REMOVE_KEY` (`"removeKey": { "id": "assertion-1" }`) cuando ninguna credencial que aún necesites dependa de ella.

> Después de eliminarla, confirma que una credencial recién emitida se verifica y que la credencial de lista de estado aún se verifica. El agente firma las credenciales de listas de estado con la primera clave `assertionMethod` que encuentra. Por tanto, una lista de estado cuya última firma usó la clave eliminada puede necesitar una actualización —una revocación o suspensión— para firmarse de nuevo.

La clave `master0` del DID, que firma las propias operaciones de actualización, está reservada. Cloud Agent rechaza acciones `ADD_KEY` y `REMOVE_KEY` que la nombren, por lo que no puede rotarse mediante la API.

La semilla de billetera no puede rotarse. La semilla se fija al crear la billetera. Cloud Agent no tiene un punto de acceso para sustituirla, y todas las claves PRISM de la billetera, incluida `master0`, se derivan de ella. «Rotar» una semilla implica crear una billetera nueva con una semilla nueva, crear y publicar DIDs nuevos en ella, volver a emitir credenciales desde esos DIDs y trasladar a los inquilinos y las partes que confían en el resultado a los nuevos DIDs. Planifícalo como una migración.

Esto también define el alcance de una exposición de la semilla. Cualquiera que tenga la semilla puede derivar `master0` y publicar operaciones de actualización o desactivación para cada DID PRISM de esa billetera. Por tanto, la respuesta es urgente: desactiva los DIDs afectados, crea una billetera y DIDs nuevos y vuelve a emitir las credenciales. Al desactivar un DID, los verificadores ya no pueden resolver claves utilizables para él, por lo que dejan de verificar todas las credenciales que emitió. Ese es el resultado previsto cuando ya no se puede confiar en las claves de firma, pero debes comunicarlo a los titulares y las partes que confían en el resultado antes de que ocurra, cuando sea posible. Por eso, la semilla y el almacenamiento Vault que la conserva son los elementos más importantes que debes proteger y respaldar en el despliegue.

Sección V: adenda

Registros de confianza

A lo largo del libro hemos seguido las credenciales entre tres roles: un emisor que firma la credencial, un titular que la almacena y presenta, y un verificador que la comprueba. La criptografía del capítulo responde a una pregunta precisa: *¿la clave que indica la credencial realmente la firmó, y alguien la ha alterado?* Es una garantía sólida, pero no cubre todo. Una firma válida solo indica al verificador que una clave firmó la credencial. No indica si debe creer a la entidad detrás de esa clave.

Considera un verificador que recibe una credencial de título universitario. La firma es correcta. Pero cualquiera puede instalar Identus Cloud Agent, crear un DID publicado y emitir una credencial que afirme ser un título de esa universidad. La criptografía de esa credencial falsificada es igual de válida. La diferencia entre el emisor real y el impostor es la *autoridad*: una comunidad reconoce a la universidad auténtica como entidad autorizada para emitir títulos. Un registro de confianza recoge esa autoridad y responde preguntas sobre ella.

Qué es un registro de confianza

Un registro de confianza es una lista oficial legible por máquina que registra qué entidades tienen la confianza para desempeñar qué roles en una comunidad o *ecosistema* concreto. En su forma más simple responde a una pregunta como:

> ¿El DID `did:prism:abc…` tiene autorización para emitir una credencial de tipo `UniversityDegree` según las reglas de este ecosistema?

El registro acompaña a la verificación criptográfica. La verificación demuestra *quién firmó* la credencial. El registro de confianza demuestra *si ese firmante tiene reconocimiento* como autoridad para lo que afirma la credencial. El verificador necesita ambas respuestas antes de actuar con seguridad.

Conviene delimitar con precisión el registro de confianza. No es una autoridad de certificación y no emite ni conserva credenciales. No es una cadena de bloques ni un registro de datos verificables, aunque puede publicar sus datos en uno. Es una lista con gobernanza que mantiene el organismo que define las reglas del ecosistema, o alguien en su nombre. En el lenguaje de identidad autosoberana, ese organismo es la *autoridad de gobernanza* y el documento que describe las reglas es el *marco de gobernanza*. El registro de confianza expresa ese marco durante la ejecución y permite consultarlo.

Algunos ejemplos concretos ayudan a comprender la idea:

- Un ministerio nacional de educación mantiene un registro de universidades acreditadas que pueden emitir credenciales de títulos. Un verificador que comprueba un diploma pregunta al registro si el DID emisor pertenece a una institución acreditada.
- Un colegio farmacéutico mantiene un registro de farmacéuticos habilitados. Un sistema que dispensa medicamentos controlados pregunta si el DID del profesional que los presenta tiene habilitación vigente.
- Una autoridad aeronáutica mantiene un registro de organizaciones de mantenimiento aprobadas cuyos técnicos pueden firmar credenciales de aeronavegabilidad.

En cada caso se repite el patrón: una comunidad con reglas, un organismo que mantiene la lista y verificadores que la consultan al decidir.

> **Nota**
>
> Los registros de confianza aún son una parte emergente de SSI. Muchos sistemas de producción usan mecanismos más simples, como una lista fija de DIDs de emisores de confianza configurada directamente en el verificador. Identus refleja esta etapa: como vimos en el capítulo , sus políticas de verificación contienen una lista `trustedIssuers` limitada a un esquema. Un registro de confianza generaliza esa idea en un servicio compartido, con gobernanza y consultas externas. Por ello, buena parte de lo que sigue describe arquitectura y perspectivas futuras, más que software ampliamente desplegado.

Posición del registro de confianza en la arquitectura

Para comprender un registro de confianza, observa cómo encaja entre los componentes que ya hemos construido. Es un servicio externo que el verificador consulta por separado del intercambio de credenciales. No forma parte de la conexión DIDComm entre titular y verificador, y normalmente el titular nunca lo usa.

El siguiente diagrama muestra un despliegue SSI con calidad de producción. El emisor, el titular y el verificador ejecutan cada uno un Identus Cloud Agent. Intercambian credenciales mediante DIDComm y usan mediadores cuando hace falta. Resuelven DIDs contra un registro de datos verificables. El registro de confianza permanece aparte y el verificador lo consulta cuando necesita decidir sobre la confianza.

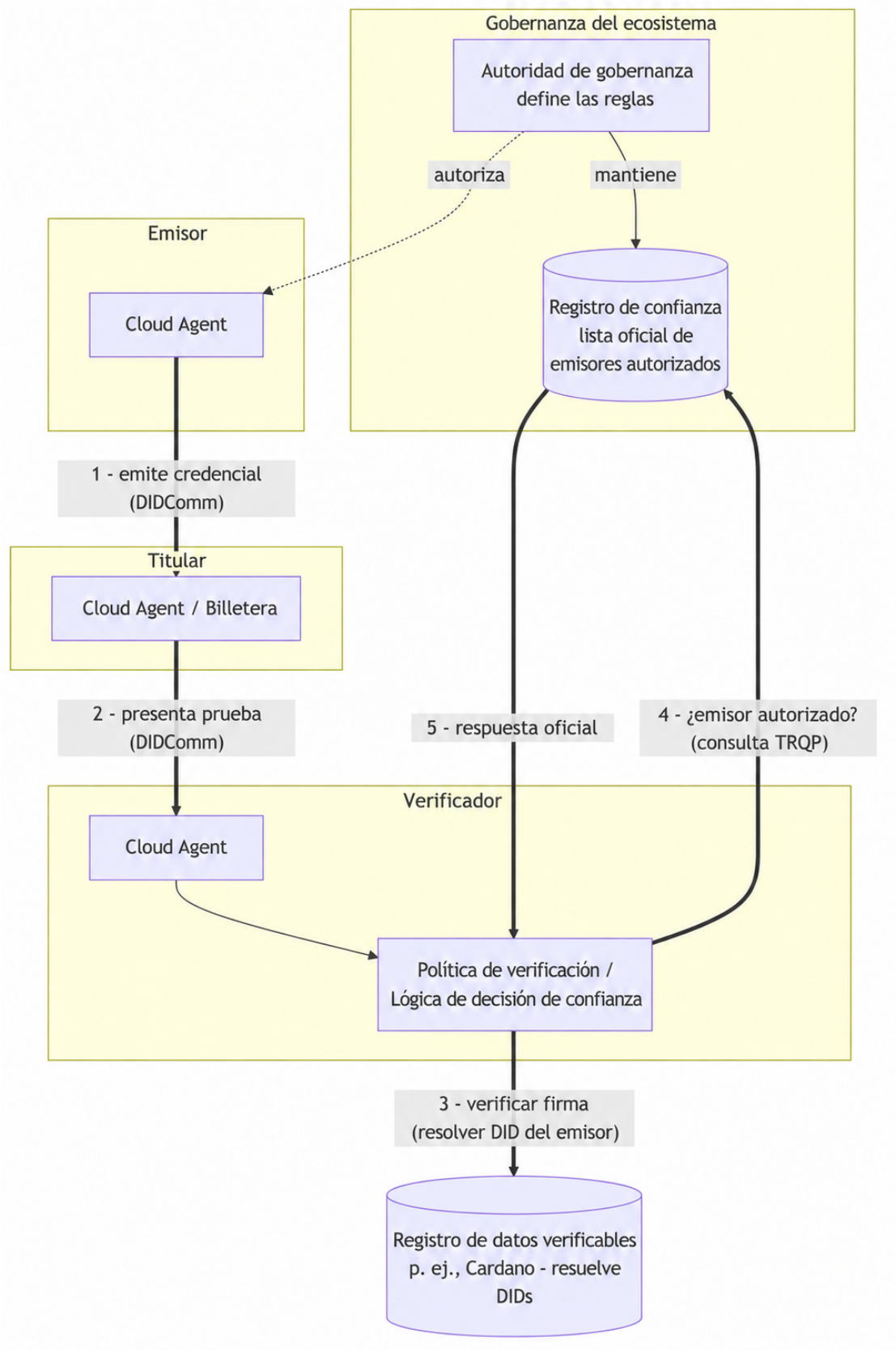


Figura 1: Posición de un registro de confianza en una arquitectura SSI de producción

Desde el punto de vista del verificador, el flujo es:

1. El emisor emite una credencial al titular mediante DIDComm, como describe el capítulo .
2. Más adelante, el titular presenta una prueba al verificador mediante una conexión DIDComm existente.
3. El verificador verifica criptográficamente la presentación y resuelve el DID del emisor contra el registro de datos verificables para comprobar la firma.
4. El verificador pregunta al registro de confianza si el DID del emisor tiene autorización para emitir este tipo de credencial según el marco de gobernanza correspondiente.
5. El registro devuelve una respuesta oficial que el verificador incorpora a su decisión final de aceptar o rechazar.

Dos propiedades de este diseño importan. Primero, la consulta del registro ocurre *fuera de banda* respecto al intercambio de credenciales. Ocurre entre el verificador y el registro; el titular no participa y ni siquiera necesita saber qué registro consulta el verificador. Segundo, la verificación de confianza y la verificación de firma son pasos separados. Como señala el capítulo , un despliegue que usa un registro externo debe separar su consulta de la verificación de pruebas: el verificador resuelve el conjunto de emisores de confianza para el esquema solicitado e incorpora ese resultado a su decisión de política. Una credencial puede ser criptográficamente perfecta y aun así recibir un rechazo porque su emisor no está en el registro.

Trust Registry Query Protocol (TRQP) 2.0 de ToIP

Si cada ecosistema inventara su propia forma de exponer un registro de confianza, los verificadores necesitarían código de integración específico para cada comunidad. La Trust Over IP Foundation (ToIP), que presentamos en el capítulo , aborda esto con **Trust Registry Query Protocol (TRQP)**. Su especificación 2.0 define una forma estándar de solo lectura para preguntar a un registro si una entidad tiene una autorización concreta en un ecosistema.

Una analogía útil de la especificación compara TRQP para registros de confianza con DNS para servidores de nombres: una capa de consulta ligera y universal que permite obtener respuestas oficiales sin conocer la implementación interna del registro. El protocolo se centra en dos tipos de preguntas:

- **Consultas de autorización**: *¿La entidad X tiene la autorización Y según el marco de gobernanza del ecosistema Z?* Es la pregunta de nuestro verificador: ¿puede este emisor emitir este tipo de credencial?
- **Consultas de reconocimiento**: *¿Este ecosistema reconoce a otro ecosistema o registro como autoridad?* Esto permite que comunidades distintas se reconozcan entre sí y que la confianza atraviese límites de organizaciones.

El protocolo estándar y mínimo permite que aplicaciones y organizaciones SSI distintas interoperen. Un verificador basado en Identus y un registro mantenido por otra organización pueden interactuar si ambos usan TRQP, sin adoptar las tecnologías de la otra parte. La

especificación separa el protocolo básico de sus adaptaciones de transporte. La adaptación inicial define TRQP mediante HTTPS con una definición OpenAPI y deja espacio para adaptaciones futuras como DIDComm. TRQP es de solo lectura: consulta datos de confianza, pero nunca crea ni modifica el contenido del registro. La autoridad de gobernanza del ecosistema conserva la responsabilidad de añadir contenidos y gobernarlos.

Para obtener los detalles completos, incluidos el modelo de datos y la definición de API, consulta la especificación aprobada: ToIP Trust Registry Query Protocol (TRQP) v2.0.

Registros de confianza e Identus hoy

Identus no incluye un registro de confianza integrado y, en el momento de escribir este texto, muy pocos funcionan en producción. Identus proporciona los componentes donde incorporar la decisión de un registro. Como explica el capítulo , la `verification policy` del verificador ya expresa un conjunto `trustedIssuers` limitado a `schemaId`. En un ecosistema maduro, un registro externo puede proporcionar ese conjunto: en lugar de fijar los DIDs de emisores de confianza en cada verificador, su controlador consulta un registro mediante TRQP, resuelve los emisores actualmente autorizados para el tipo de credencial y aplica el resultado a su decisión de política.

Esto mantiene una arquitectura clara y adaptable. Las comprobaciones criptográficas del verificador conservan su lugar. La decisión de confianza se convierte en una consulta separada que puede sustituirse. Puede comenzar como una lista estática durante el desarrollo y crecer hasta un registro con gobernanza que use TRQP a medida que madura el ecosistema de la aplicación.

> **Nota**
>
> Como los registros de confianza están en una etapa inicial de adopción, trata este capítulo como una guía de la dirección del ecosistema. Si construyes hoy, mantén aislada la lógica de decisión de confianza de la lógica de verificación, para que sustituir una lista estática de emisores por un registro real sea un cambio localizado.

Continúa tu recorrido

Información sobre cómo convertirse en colaborador de Identus

- Cómo contribuir al código fuente o a la documentación.

Información sobre cómo participar en la comunidad SSI fuera de Identus

Trust over IP (ToIP)

Trust Over IP (ToIP) reúne más de 300 organizaciones de distintos sectores que colaboran para impulsar la identidad descentralizada mediante ideas y software. Según ToIP: «Desarrollamos herramientas y especificaciones para ayudar a comunidades de cualquier tamaño a usar redes digitales para construir y fortalecer la confianza entre participantes».

The ToIP Stack, publicado en 2019, describe cómo las cuatro capas —DIDs, protocolo DIDComm, protocolos de intercambio de datos y ecosistemas de aplicaciones— forman un marco de confianza digital seguro, escalable e interoperable. Introduction to ToIP, Version 2.0, publicado en 2021, es un recurso útil para conocer el problema de identidad digital y su solución. Las opciones de afiliación incluyen cuentas gratuitas de colaborador y membresías de pago que proporcionan acceso a los recursos y canales Slack de ToIP para colaborar.

Para unirte a Trust Over IP, debes crear una cuenta gratuita de Linux Foundation y completar la solicitud ToIP aquí.

Todos los miembros acuerdan una participación abierta y no competitiva. Pide a tu equipo jurídico que revise todos los documentos asociados durante la incorporación.

Para obtener más detalles, consulta The ToIP Stack y la descripción general.

Decentralized Identity Foundation (DIF)

Apéndices

Erratas

Las erratas aparecerán aquí.

Enumeraremos los errores que puedan haberse «impreso» en determinadas ediciones del libro.

Glosario

Este glosario reúne los términos de identidad autosoberana (SSI), identidad descentralizada y específicos de Identus que usa el libro. Las definiciones proporcionan al desarrollador información suficiente para seguir los capítulos, con referencias a los estándares que definen formalmente cada término. Los términos aparecen en orden alfabético.

ADA La criptomoneda nativa de la cadena de bloques Cardano. PRISM Node gasta ADA en comisiones cuando ancla operaciones `did:prism` en la cadena: ADA de prueba en redes como `preprod` y ADA real en `mainnet`.

Afirmación Una declaración o atributo individual sobre un sujeto dentro de una credencial, por ejemplo un nombre, fecha de nacimiento o categoría de licencia. Una credencial verificable es un conjunto de afirmaciones de un emisor.

Almacenamiento de secretos El almacén configurable de Cloud Agent para semillas de billetera y claves privadas. Un backend `postgres` guarda secretos sin cifrar y sirve solo para laboratorio. Se recomienda HashiCorp Vault para producción.

Anclaje en el registro distribuido Registrar operaciones DID como transacciones en una cadena de bloques, como Cardano, para que las alteraciones sean detectables y permitan resolución pública.

AnonCreds (credenciales anónimas) Un formato de credencial verificable que preserva la privacidad y procede del ecosistema Hyperledger Indy. Usa pruebas de conocimiento cero y pruebas de predicados para que un titular demuestre hechos, como «edad superior a 21», sin revelar los valores originales de los atributos. Identus Cloud Agent lo admite.

Apollo El componente básico de Identus que proporciona primitivas criptográficas —hashing, firmas digitales, cifrado y generación de claves— usadas en toda la plataforma para garantizar integridad, autenticidad y confidencialidad de datos.

AppRole Un método de autenticación de HashiCorp Vault, formado por un ID de rol y un ID secreto, que se prefiere a un token raíz estático para despliegues automáticos de producción de Cloud Agent.

Aries (Hyperledger Aries) Una familia de protocolos de interoperabilidad Hyperledger (RFC) para agentes SSI que cubre intercambio de DIDs, invitaciones fuera de banda y flujos de emisión de credenciales y presentación de pruebas. Identus Cloud Agent busca la compatibilidad con W3C y Aries.

assertionMethod Una relación de verificación en un documento DID que enumera las claves aprobadas para hacer afirmaciones, incluida la firma o emisión de credenciales verificables. El verificador comprueba que la clave firmante de la credencial aparece aquí.

authentication Una relación de verificación en un documento DID que enumera las claves aprobadas para demostrar el control del DID mediante autenticación de desafío y respuesta.

Autoridad de certificación (CA) El modelo centralizado tradicional de confianza digital, donde una autoridad central garantiza identidades. El libro lo compara con un registro de confianza, que registra la autoridad dentro de un ecosistema con gobernanza, pero no emite ni conserva credenciales.

Autoridad de gobernanza El organismo que define las reglas de un ecosistema y mantiene o supervisa su registro de confianza.

Billetera En SSI, software que almacena y protege los DIDs, claves, conexiones y credenciales de una parte, y crea presentaciones. En Identus Cloud Agent, también es la unidad de aislamiento del inquilino. La billetera del titular suele ser un Edge Agent; una billetera de servidor se ejecuta en Cloud Agent. Es distinta de Cardano Wallet.

Cardano La cadena de bloques pública que Identus usa como registro de datos verificables de producción. Ancla operaciones `did:prism` para que cualquiera pueda resolver un DID PRISM publicado. El anclaje requiere ADA para pagar comisiones.

Cardano DB Sync Un componente Cardano que sigue la cadena e indexa bloques en PostgreSQL para que PRISM Node lea datos del registro distribuido de forma eficiente. Normalmente consume más recursos que el resto de un entorno Cardano alojado por el propio operador.

Cardano Wallet El servicio del ecosistema Cardano que PRISM Node usa para administrar fondos y enviar transacciones Cardano al publicar operaciones DID. Es distinto de una billetera SSI.

Castor El componente básico de Identus que crea, administra y resuelve identificadores descentralizados. Actualmente admite los métodos `did:prism` y `did:peer`.

Clave de API Una credencial por inquilino, enviada en la cabecera `apikey`, que limita las llamadas de API de Cloud Agent a la billetera de ese inquilino. Las operaciones privilegiadas de administración usan un token de administrador.

Claves de destinatario Las claves que una billetera registra con un mediador para que sepa qué mensajes entrantes debe encolar para ese destinatario.

Cloud Agent El agente de Identus en el servidor, escrito en Scala, que administra billeteras, DIDs y mensajería DIDComm, y puede emitir, conservar y verificar credenciales verificables. Una aplicación controladora lo dirige mediante su API REST. Se espera que esté siempre conectado.

cnf (clave de confirmación) Un elemento de vinculación de clave incorporado en una credencial SD-JWT. Vincula la credencial a una clave del titular para que este demuestre después su control al firmar el desafío y el dominio del verificador.

Componentes básicos El conjunto de bibliotecas modulares especializadas de Identus que administran un área de funciones SSI cada una: Apollo, criptografía; Castor, DIDs; Pollux, credenciales; Mercury, DIDComm; y Pluto, almacenamiento. Cloud Agent y los SDK de Edge Agent implementan los mismos componentes básicos.

Conexión Una relación DIDComm establecida, segura y continua entre dos agentes. Normalmente usa DIDs de pares y cada parte la mantiene como registro de conexión en su billetera.

Controlador (aplicación) La aplicación que dirige un Cloud Agent mediante su API REST y callbacks webhook. El controlador contiene la lógica de negocio y el agente administra los estándares SSI. No debe confundirse con un controlador DID.

Controlador DID La entidad autorizada según las reglas del método DID para cambiar un documento DID: rotar claves, añadir servicios o desactivar. Puede ser el mismo sujeto DID u otra entidad que actúe en su nombre.

Coordinate Mediation El protocolo DIDComm `coordinate-mediation/2.0` que una billetera usa para solicitar mediación a un mediador y registrar las claves de destinatario para las que debe aceptar mensajes.

Correlación La vinculación de la actividad de una persona u organización entre contextos mediante un identificador compartido o una clave reutilizada. Los DIDs públicos y el material reutilizado de documentos DID crean riesgo de correlación; los DIDs específicos de cada relación lo reducen.

COSE La familia de estándares criptográficos basada en CBOR para firma y cifrado, equivalente binario de JOSE, que referencia la recomendación W3C VC «Securing with JOSE/COSE».

Credencial verificable (VC) Un conjunto de afirmaciones de un emisor, protegido con un mecanismo criptográfico para que el software detecte alteraciones y confirme la autoría. El modelo de datos de credenciales verificables de W3C la define. El modelo de datos es independiente del formato: JWT-VC, SD-JWT-VC o AnonCreds.

credentialStatus La propiedad de una credencial revocable que apunta a su entrada en la lista de estado. Permite que el verificador descubra si está revocada o suspendida. El modelo de datos VC de W3C la define.

DID de pares Un DID del método `did:peer` que las partes de una relación comparten de forma privada y no publican en ningún registro público. Identus lo usa para conexiones DIDComm y mediación, donde los identificadores específicos de cada relación reducen la correlación.

DID del emisor Un DID bajo el control de un emisor que usa para firmar sus credenciales, de modo que los verificadores resuelvan sus claves y comprueben la autoría. Normalmente es público, como los DIDs PRISM.

DID específico de una relación Un DID único creado para una sola relación, de modo que cada relación use un identificador distinto con seudónimo. Reduce la correlación entre contextos. Los DIDs de pares suelen usarse de esta forma.

DID PRISM Un DID que usa el método `did:prism` de Identus y cuyas operaciones de creación, actualización y desactivación se anclan al registro Cardano mediante PRISM Node. Sirve para identidades públicas de emisores y verificadores que necesitan resolución respaldada por el registro distribuido. Consulta también forma larga / forma corta.

DID publicado Un DID cuyas operaciones se han anclado al registro distribuido para que cualquier parte lo resuelva. Se diferencia de un DID no publicado: por ejemplo, normalmente el titular no necesita publicarlo y el emisor sí.

`did:peer` Consulta DID de pares.

`did:prism` Consulta DID PRISM.

DIDComm (DIDComm Messaging V2) Un protocolo de mensajería seguro, cifrado e independiente del transporte para comunicar agentes identificados por DIDs. DIDComm V2, la versión de Identus, oculta el contenido a todos salvo los destinatarios autorizados, demuestra el remitente en modo autenticado y funciona mediante cualquier transporte, como HTTP, WebSockets o Bluetooth.

DIDPair El emparejamiento criptográfico de un DID remitente y un DID destinatario que juntos identifican un canal DIDComm. Un mediador mantiene una cola de mensajes por DIDPair.

DIF (Decentralized Identity Foundation) Una organización del sector que desarrolla estándares de identidad descentralizada, incluidos DIDComm y Presentation Exchange.

DIF Presentation Exchange Una especificación DIF para expresar qué prueba requiere un verificador y cómo responde la billetera del titular. Identus Cloud Agent la implementa para solicitudes y entregas de credenciales.

Divulgación (SD-JWT) En una credencial SD-JWT, el valor de afirmación con sal que el titular revela durante la presentación para que el verificador calcule su hash y lo compare con el resumen firmado en la credencial. Las afirmaciones no divulgadas permanecen ocultas.

Divulgación selectiva La capacidad del titular de revelar solo las afirmaciones que necesita el verificador, en lugar de toda la credencial. Por ejemplo, demostrar «más de 21» sin revelar toda la fecha de nacimiento. Depende del formato: SD-JWT y AnonCreds la admiten; JWT-VC simple no. Apoya la minimización de datos.

Documento DID El documento en el que se resuelve un DID. Publica el material público necesario para interactuar con el sujeto DID: métodos de verificación, relaciones de verificación y puntos de acceso de servicios. Nunca debe contener claves privadas, secretos o un perfil personal.

Ecosistema Una comunidad de participantes que opera según un marco de gobernanza compartido, donde se definen relaciones de confianza y autorizaciones. Un registro de confianza responde preguntas sobre la autoridad en un ecosistema específico.

Ed25519 Un algoritmo de firma digital y tipo de clave de curva elíptica de uso común para autenticación y firma de credenciales. Por ejemplo, Identus lo requiere para las claves del emisor y titular de SD-JWT.

Edge Agent Un agente Identus que se ejecuta dentro de una aplicación para usuarios, web, móvil o de escritorio, implementado como SDK. A diferencia de Cloud Agent, no se puede suponer que siempre está conectado y depende de un mediador para enviar y recibir mensajes. Normalmente actúa como billetera del usuario.

Edge SDK Las bibliotecas cliente Identus, disponibles en TypeScript, Swift y Kotlin Multiplatform, que incorporan funciones de Edge Agent en una aplicación. Implementan los mismos componentes básicos que Cloud Agent.

Emisión de credenciales El flujo de protocolo por el que un emisor crea, firma y entrega una credencial verificable en la billetera del titular. En Identus sigue el protocolo Issue Credential mediante DIDComm, u OpenID4VCI.

Emisor El rol que hace afirmaciones sobre un sujeto, las empaqueta como credencial verificable y la firma criptográficamente. Su autoridad procede del contexto: ley, acreditación, contrato o gobernanza.

Emisores de confianza El conjunto de DIDs de emisores, normalmente limitado a un esquema de credencial, que un verificador acepta como autoridades. En Identus se configura actualmente en una política de verificación. Un registro de confianza generaliza este concepto.

Entidad En Cloud Agent, el objeto administrativo que representa a un inquilino y se vincula a una billetera. La autenticación, mediante clave de API o permiso Keycloak, se asocia a una entidad. En modo de un solo inquilino, usa automáticamente una entidad predeterminada con ID de ceros.

Esquema de credencial Una definición registrada y compartida de la estructura de un tipo de credencial: campos, tipos y atributos obligatorios. Los emisores emiten conforme a ella, y los verificadores y sus políticas de verificación se limitan a ella. Identus admite JSON Schema para JWT y SD-JWT y esquemas AnonCreds.

Forma larga / forma corta de DID PRISM Dos codificaciones de un DID PRISM. La forma larga incorpora todo el estado inicial del DID en el identificador y se puede resolver antes de publicarlo. La forma corta es el identificador compacto que se usa después de publicarlo en el registro distribuido.

HashiCorp Vault Un servicio de administración de secretos que Cloud Agent puede usar como backend de almacenamiento de secretos de producción para semillas de billetera y material de claves, con rutas por billetera en despliegues de varios inquilinos.

Hyperledger Indy Un ecosistema Hyperledger de cadena de bloques y SSI del que procede el formato de credencial AnonCreds.

Identidad autosoberana (SSI) Un modelo de identidad digital basado en credenciales bajo custodia del usuario y pruebas criptográficas, en lugar de un proveedor central de identidad. Un sujeto recibe una credencial de un emisor, la almacena como titular y presenta pruebas a un verificador que aplica su propia política. SSI cambia la custodia y el intercambio de datos de identidad; la gobernanza, la ley y la reputación aún determinan la confianza.

Identificador descentralizado (DID) Un identificador único global que una entidad controla sin un registrador central. La especificación DID Core de W3C lo define como un URI de tres partes: el esquema `did:`, un nombre de método y un identificador específico del método. Se resuelve en un documento DID.

Identus (Hyperledger Identus) La plataforma de identidad descentralizada de código abierto que explica este libro, alojada bajo el marco de confianza descentralizada de Linux Foundation. Proporciona Cloud Agent, PRISM Node, SDK de Edge y Mediator para emitir, conservar y verificar credenciales.

Inquilino Un usuario lógico u organización que atiende Cloud Agent, representado por una entidad y respaldado por una billetera aislada. Consulta el modo de varios inquilinos.

Invitación fuera de banda (OOB) Una invitación DIDComm autónoma, que suele entregarse como URL o código QR con un parámetro de consulta `_oob`. Permite que una nueva parte inicie una conexión, mediación o emisión o presentación sin conexión previa, sin un canal existente.

JOSE La familia de estándares criptográficos basada en JSON —JSON Web Signature, JSON Web Encryption, JSON Web Key y JSON Web Token— que suele proteger credenciales. Consulta también COSE.

JSON Web Key (JWK) Un formato JSON para representar una clave criptográfica. Los documentos DID y el material de claves DIDComm suelen expresarse como JWK.

JWT-VC Un formato de credencial verificable que empaqueta afirmaciones en un JSON Web Token firmado. Normalmente se presenta completo y revela todas sus afirmaciones, sin divulgación selectiva.

keyAgreement Una relación de verificación en un documento DID que enumera claves aprobadas para el acuerdo de claves; por ejemplo, establecer el cifrado de DIDComm. Normalmente usa una clave X25519.

Keycloak Un servidor de administración de identidad y acceso que Identus puede integrar para que los inquilinos se autentiquen mediante OIDC y reciban permisos de billetera mediante UMA, en lugar de claves estáticas de API.

Lista de estado Un mecanismo de revocación publicado y eficiente en espacio, como Bitstring Status List o StatusList2021, donde cada bit representa el estado de revocación o suspensión de una credencial. El verificador lee el bit que referencia credentialStatus. En Identus lo proporciona Pollux.

Marco de gobernanza Las reglas que publica una autoridad de gobernanza para describir quién puede hacer qué en un ecosistema. Un registro de confianza las expresa durante la ejecución y permite consultarlas. La autoridad procede del marco de gobernanza, no de la tecnología del registro.

Mediación El acuerdo que establece la billetera del titular con un mediador para que reciba y retransmita mensajes DIDComm en su nombre. Produce información de enrutamiento que la billetera anuncia después en su documento DID.

Mediador Un servicio DIDComm V2 que actúa como retransmisor estable y siempre conectado para agentes, como billeteras móviles, que no siempre son accesibles. Encola mensajes cifrados por DIDPair y los entrega cuando el destinatario consulta o se conecta, sin poder leer el contenido cifrado. *No* es un VDR.

Mensaje `forward` Un mensaje de enrutamiento DIDComm del protocolo `routing/2.0` que envuelve un mensaje ya cifrado para el destinatario final en una envoltura externa dirigida a un mediador. Así puede enrutarlo sin leer el contenido interno.

Mercury El componente básico de Identus que proporciona la interfaz DIDComm V2 para mensajería segura entre agentes, independiente del transporte subyacente.

Message Pickup El protocolo DIDComm `messagepickup/3.0` que una billetera usa para consultar mensajes en cola de un mediador, solicitar su entrega y confirmar su recepción.

Método de verificación Una entrada de un documento DID que especifica una clave pública, por ejemplo como JWK, su tipo y su controlador. Las relaciones de verificación lo referencian para fines específicos.

Método DID El esquema —la parte posterior a `did:`, como `prism` o `peer`— que define cómo crear, resolver, actualizar y desactivar una clase de DID. W3C define el modelo de datos compartido; cada especificación de método define su propio registro, operaciones y ciclo de vida.

Modo de varios inquilinos Un modo de Cloud Agent en el que una instancia aloja muchos inquilinos aislados, cada uno con su propia billetera, DIDs, credenciales y conexiones. El modo de un solo inquilino atiende una billetera predeterminada.

NeoPRISM Un backend de nodo DID más reciente para Identus, recomendado para producción, que puede sustituir al PRISM Node heredado para publicar y resolver operaciones `did:prism`.

Oferta de credencial Un mensaje de un emisor que propone emitir una credencial específica a un titular e inicia el flujo de emisión de credenciales.

OIDC (OpenID Connect) Un protocolo estándar de autenticación sobre OAuth 2.0 que, mediante Keycloak, autentica inquilinos y emite tokens de acceso.

OpenID4VCI (OID4VCI) OpenID for Verifiable Credential Issuance: un protocolo basado en OAuth 2.0 para que una billetera obtenga credenciales desde el punto de acceso del emisor. Es una vía alternativa de emisión a DIDComm.

OpenID4VP OpenID for Verifiable Presentations: un protocolo basado en OAuth 2.0 para solicitar y presentar credenciales. Es una vía alternativa de presentación a DIDComm.

Parte que confía en el resultado La parte que consume un resultado de verificación y aplica políticas de negocio y confianza —emisores, esquemas y tipos de credenciales aceptados— para decidir si actúa. A menudo es el mismo actor que el verificador.

Pluto El componente básico de Identus que define la interfaz de almacenamiento de datos de identidad: DIDs, claves, credenciales y estado de conexión. La aplicación proporciona la implementación concreta del almacenamiento.

Política de verificación La configuración del lado del verificador en Identus que expresa qué esquemas de credenciales y emisores de confianza acepta. Se aplica después de las comprobaciones criptográficas y es la forma práctica de la política de aceptación o validación.

Pollux El componente básico y subsistema de Cloud Agent responsable de operaciones de credenciales verificables: emisión, verificación, divulgación selectiva y estado de credenciales.

Presentación verificable (VP) Datos derivados de una o más credenciales verificables que se comparten con un verificador específico como respuesta a una solicitud. Según el formato, una presentación puede incluir una credencial completa, partes de una —divulgación selectiva— o datos de varias.

PRISM Node El componente Identus que implementa `did:prism` y actúa como nodo de segunda capa sobre el registro distribuido. Publica y resuelve DIDs PRISM y mantiene un estado interno indexado, sincronizado con la cadena subyacente. Sirve como VDR para DIDs PRISM y se espera que esté siempre conectado.

Profundidad de confirmación El número de bloques que deben añadirse después de una transacción DID antes de tratar su estado como final y seguro para resolver. Es un parámetro ajustable al anclar DIDs en un registro distribuido.

Protocolo Issue Credential El protocolo DIDComm, Issue Credential 3.0 en Identus, que regula el intercambio de mensajes de oferta, solicitud y emisión de credenciales entre emisor y titular.

Protocolo Present Proof El protocolo DIDComm independiente del formato, Present Proof 3.0 en Identus, que transporta una solicitud de presentación del verificador y una presentación de respuesta del titular.

Prueba de conocimiento cero (ZKP) Una técnica criptográfica que demuestra que una declaración es verdadera sin revelar los datos subyacentes. Es la base de las pruebas de predicados AnonCreds, como «más de 21» sin divulgar una fecha de nacimiento. Prueba de predicado

Una prueba de que un atributo satisface una condición, como `age >= 21`, sin divulgar el valor original. Es una función esencial de AnonCreds y las pruebas de conocimiento cero.

Punto de acceso de servicio Una entrada de un documento DID que anuncia dónde y cómo interactuar con el sujeto DID. Por ejemplo, un punto de acceso `DIDCommMessaging` que indica la URL, el perfil DIDComm aceptado y las claves de enrutamiento.

Registro de confianza Un registro oficial, con gobernanza y legible por máquina de las entidades autorizadas para cada rol de un ecosistema. El verificador lo consulta para confirmar la *autoridad* del emisor, una pregunta separada de la *validez* criptográfica de la credencial. No es una autoridad de certificación ni un VDR.

Registro de datos verificables (VDR) Un sistema, a menudo una cadena de bloques, pero también una base de datos, red distribuida o almacén en memoria, donde se publican y resuelven DIDs y datos relacionados. Permite que partes independientes verifiquen autenticidad e integridad sin autoridad central. Identus accede a VDR mediante drivers intercambiables, como los de PRISM Node y NeoPRISM. Un mediador no es un VDR.

Relación de verificación La asociación en un documento DID entre un método de verificación y un fin permitido: authentication, assertionMethod, keyAgreement, `capabilityInvocation` o `capabilityDelegation`. Estas relaciones evitan reutilizar una clave entre distintos fines de prueba.

Resolución DID El proceso de buscar un DID y devolver su documento DID junto con metadatos de resolución. Un resolvedor que comprende el método DID correspondiente lo ejecuta.

Resolvedor Software que realiza la resolución DID: recibe un DID y devuelve su documento DID. Un resolvedor universal dirige muchos métodos a drivers específicos. Elegir un resolvedor es una decisión de confianza.

Resolvedor universal Un servicio público de resolución que puede resolver muchos métodos DID al dirigir cada uno a un driver específico.

Revocación / suspensión Invalidar una credencial emitida antes de su caducidad natural, de forma permanente —revocación— o temporal —suspensión—. El cambio aparece en una lista de estado que los verificadores consultan mediante credentialStatus.

Ruta de derivación La ruta determinista que, junto con una semilla de billetera, regenera las claves de un DID específico mediante derivación jerárquica determinista. Cloud Agent almacena la ruta en lugar de las claves derivadas.

SD-JWT / SD-JWT-VC Selective Disclosure JWT: una variante JWT que usa hashes o resúmenes con sal para que el titular revele solo afirmaciones seleccionadas mientras el verificador compara cada valor divulgado con el resumen firmado. SD-JWT-VC es el formato de credencial verificable basado en ella. Identus lo admite. Consulta divulgación y cnf.

secp256k1 Un algoritmo de clave y firma de curva elíptica, también usado por Bitcoin y Cardano, que Identus admite para ciertas pruebas de credenciales y listas de estado.

Secreto de vinculación Un secreto bajo el control del titular que AnonCreds usa para vincular credenciales emitidas al titular y permitir pruebas de conocimiento cero entre varias credenciales.

Semilla de billetera El material secreto de clave raíz del que una billetera deriva determinísticamente sus claves DID mediante una ruta de derivación almacenada. Es un secreto de gran valor: perderlo puede impedir de forma permanente usar o actualizar DIDs existentes.

Solicitud de presentación Un mensaje, también llamado solicitud de prueba, en el que el verificador especifica qué prueba necesita del titular. El protocolo Present Proof lo transporta.

Sujeto La persona, organización, dispositivo, cuenta o cosa que describen las afirmaciones de una credencial. A menudo es el titular, pero no necesariamente: un progenitor puede conservar una credencial de su hijo, o un administrador de flota una de un dispositivo.

Titular El rol que recibe credenciales verificables, las almacena normalmente en una billetera y crea presentaciones en respuesta a solicitudes del verificador. A menudo es el sujeto de la credencial, pero no siempre.

ToIP (Trust over IP Foundation) Un organismo de Linux Foundation que desarrolla estándares de confianza digital descentralizada, incluido ToIP Stack —un modelo por capas de DIDs, DIDComm, protocolos de intercambio de datos y ecosistemas de aplicaciones— y Trust Registry Query Protocol.

Triángulo de confianza Una forma común de describir los tres roles SSI —emisor, titular y verificador— y los flujos de credenciales y presentaciones entre ellos. Los roles dependen de la interacción: una entidad puede emitir una credencial, conservar otra y verificar una tercera.

TRQP (Trust Registry Query Protocol) Un protocolo de solo lectura de ToIP para preguntar a un registro de confianza si una entidad tiene una autorización según la gobernanza de un ecosistema. En términos simples: «¿La entidad X tiene la autorización Y según el marco de gobernanza Z?». A veces se describe como «DNS para registros de confianza».

UMA (User-Managed Access) Un estándar de autorización basado en OAuth 2.0 que Keycloak usa para conceder a un sujeto permisos sobre un recurso de billetera específico de Cloud Agent. Emite un token de parte solicitante (RPT) que el inquilino presenta al agente.

Validación La comprobación de reglas de negocio del verificador para determinar si una credencial es *adecuada* para una decisión; por ejemplo, si el emisor resulta aceptable según una política de contratación. Es distinta de la verificación: una credencial puede ser criptográficamente válida y no superar la validación.

Verificación La comprobación técnica de que una credencial o presentación es criptográficamente correcta: firmas válidas, claves del emisor resueltas correctamente, uso adecuado de claves según las relaciones de verificación, estado actual y vinculación del titular. Es distinta de la validación, que aplica políticas de negocio.

Verificador El rol que solicita una presentación al titular y decide si la acepta, mediante verificación criptográfica y validación de política de negocio. Consulta también parte que confía en el resultado.

Vinculación de clave Vincular criptográficamente una credencial a una clave del titular para que demuestre su control al presentarla. Consulta también vinculación del titular y cnf.

Vinculación del titular Prueba criptográfica de que la parte que presenta una credencial controla la clave asociada a su sujeto. Consulta también vinculación de clave.

Webhook El mecanismo de callback HTTP por el que Cloud Agent notifica al controlador cambios de estado, como una conexión recibida, una credencial emitida o recibida, o una presentación verificada.

X25519 Un tipo de clave de curva elíptica para el acuerdo de claves Diffie-Hellman que establece el cifrado de DIDComm. Aparece en los documentos DID bajo la relación keyAgreement.